

# Database SQL (Structure Query Language)



**[https://github.com/up1/  
workshop-database](https://github.com/up1/workshop-database)**



# Topics

Introduction to Database management

Database modeling

Relational Database vs NoSQL

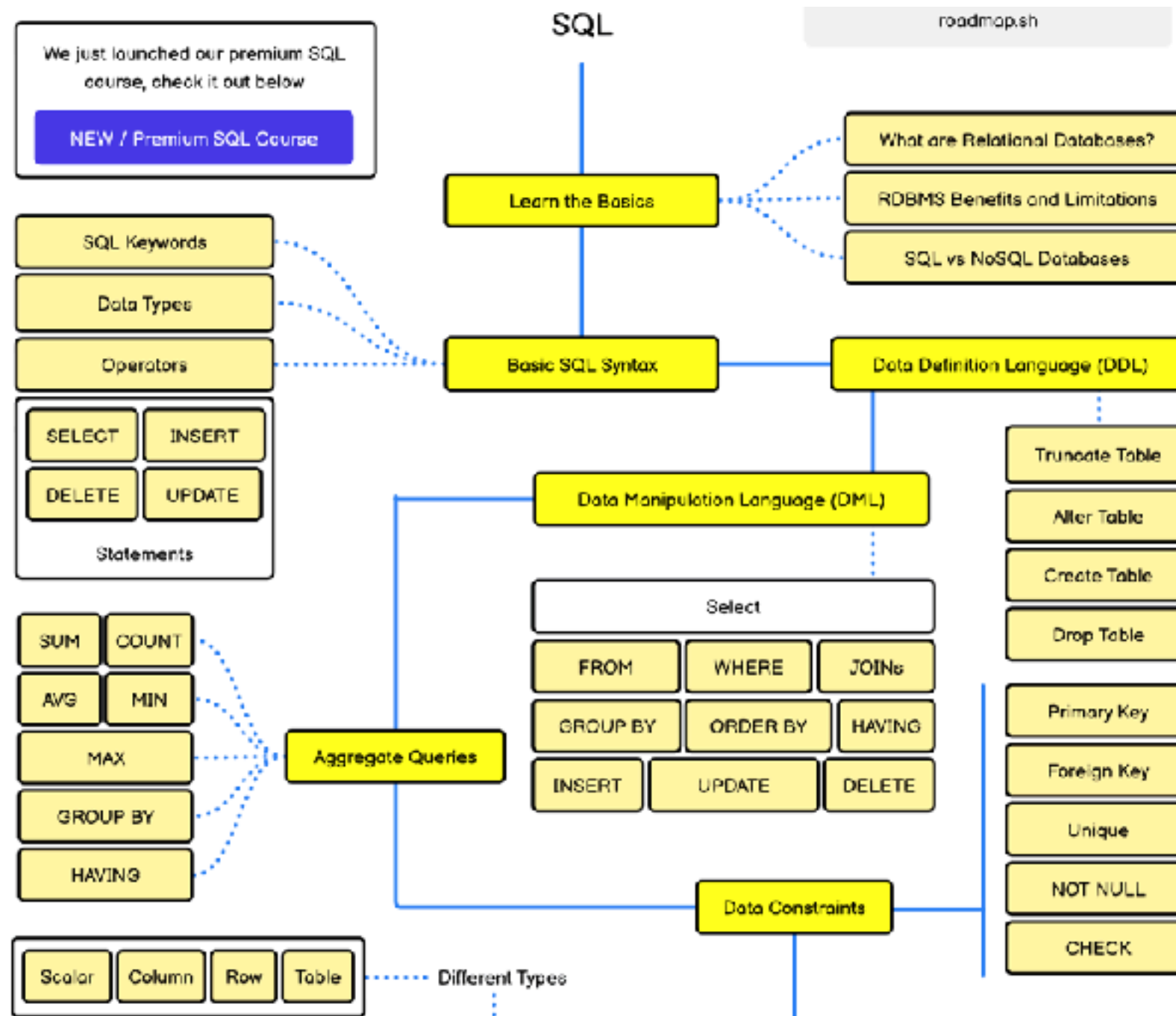
Basic of Relational Database

SQL (Structure Query Language)

Workshop with SQL and PostgreSQL



# SQL Roadmap



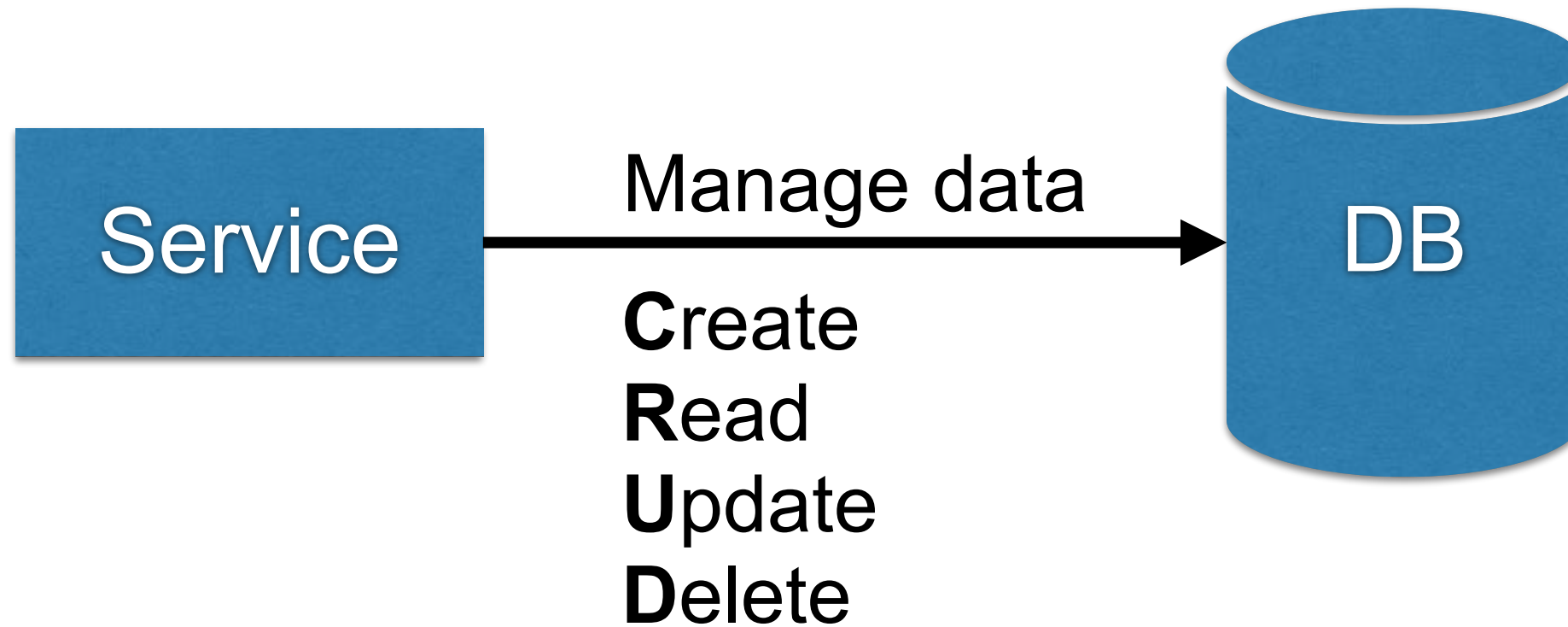
<https://roadmap.sh/sql>



# Database

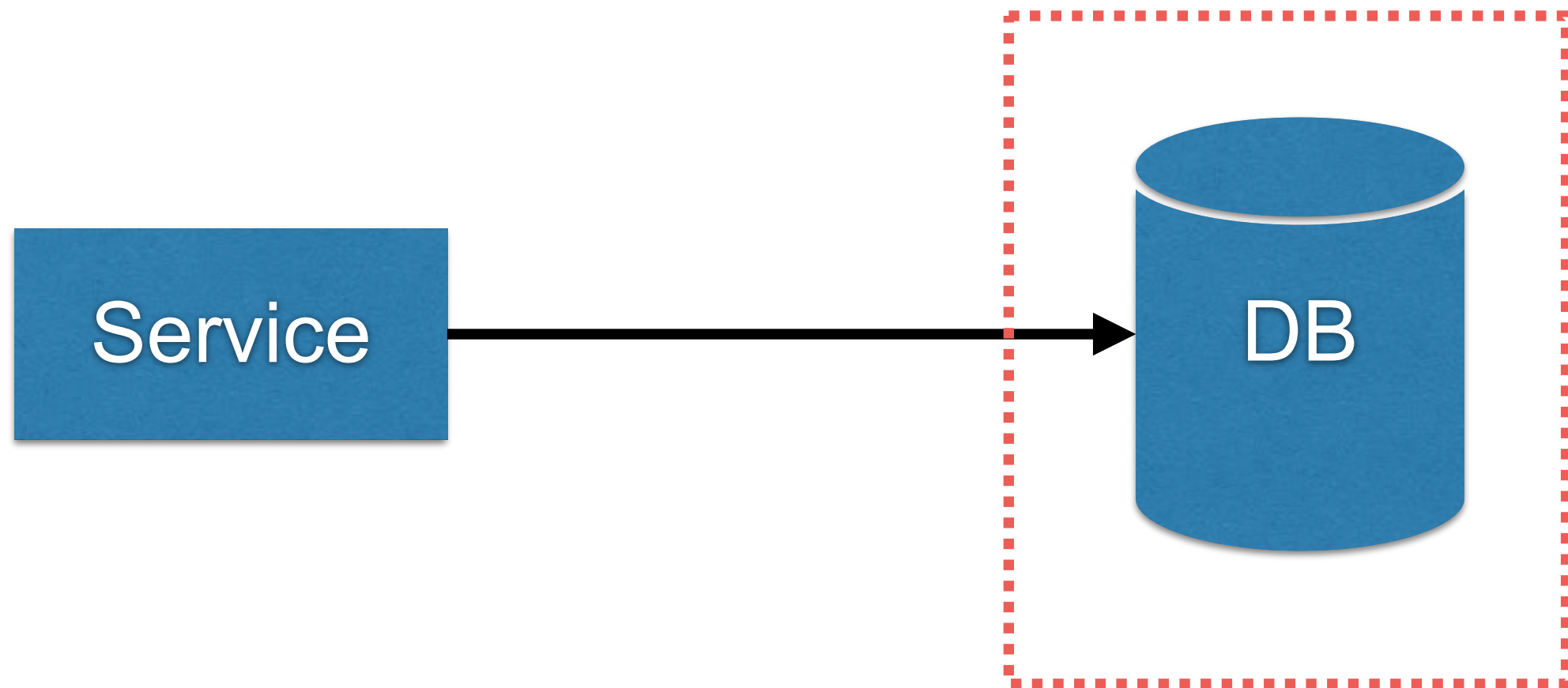


# Database



# Working with Database

Directly to database



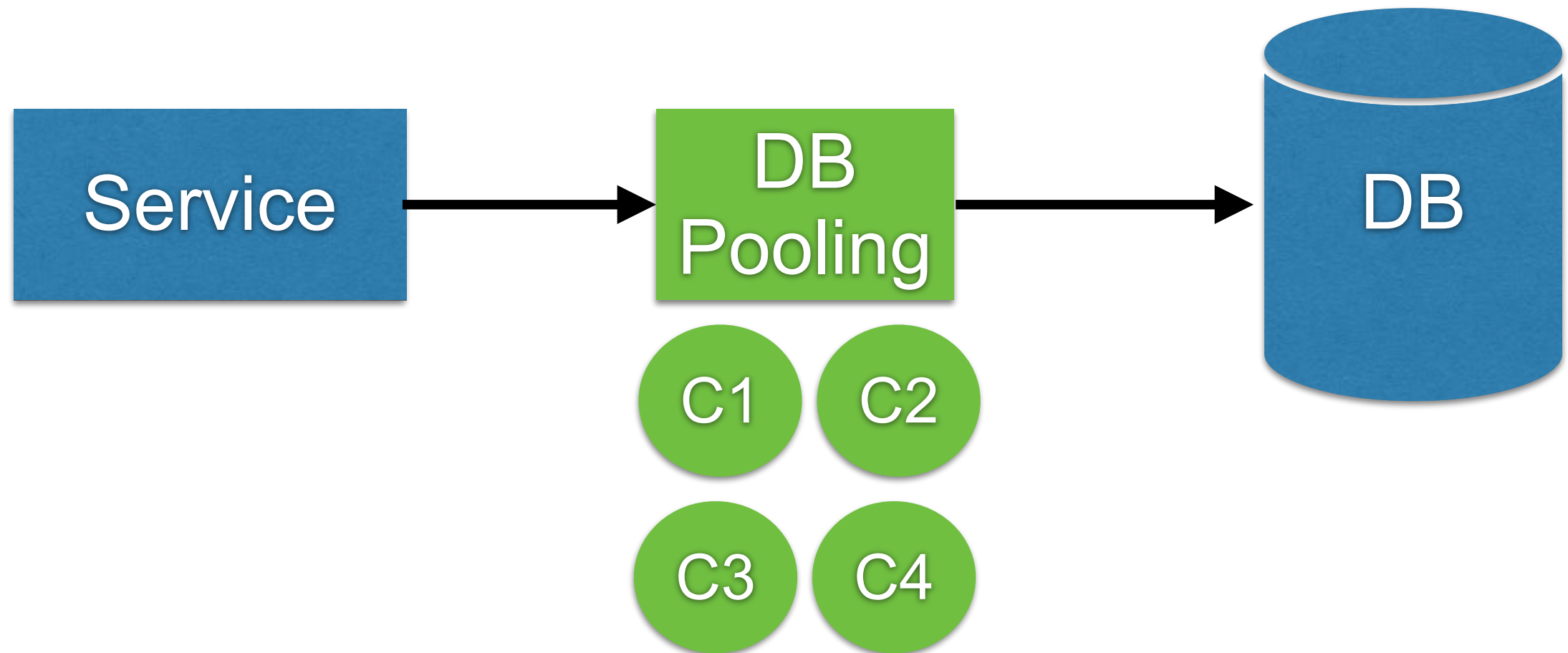
Data modeling  
Database architecture





# Working with Database

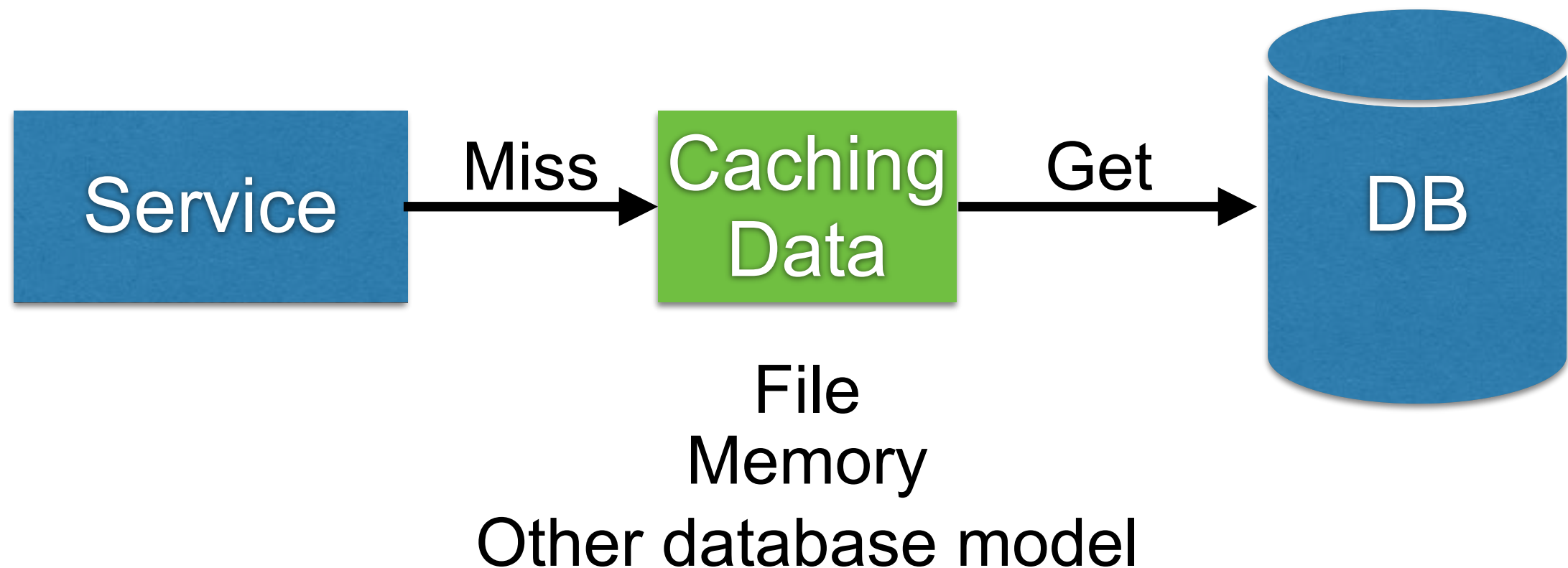
## Connection pool





# Working with Database

## Caching data layer



# Scalability metrics

Throughput (transactions per second)

Latency (response time)

Availability (uptime)

Durability (data safety)



# Database ?

Structured data collection

Records

Relationships



# Database Management System ?

Software systems for storing and querying databases



# User's perspective

Data modeling

Querying data

Store data

Read

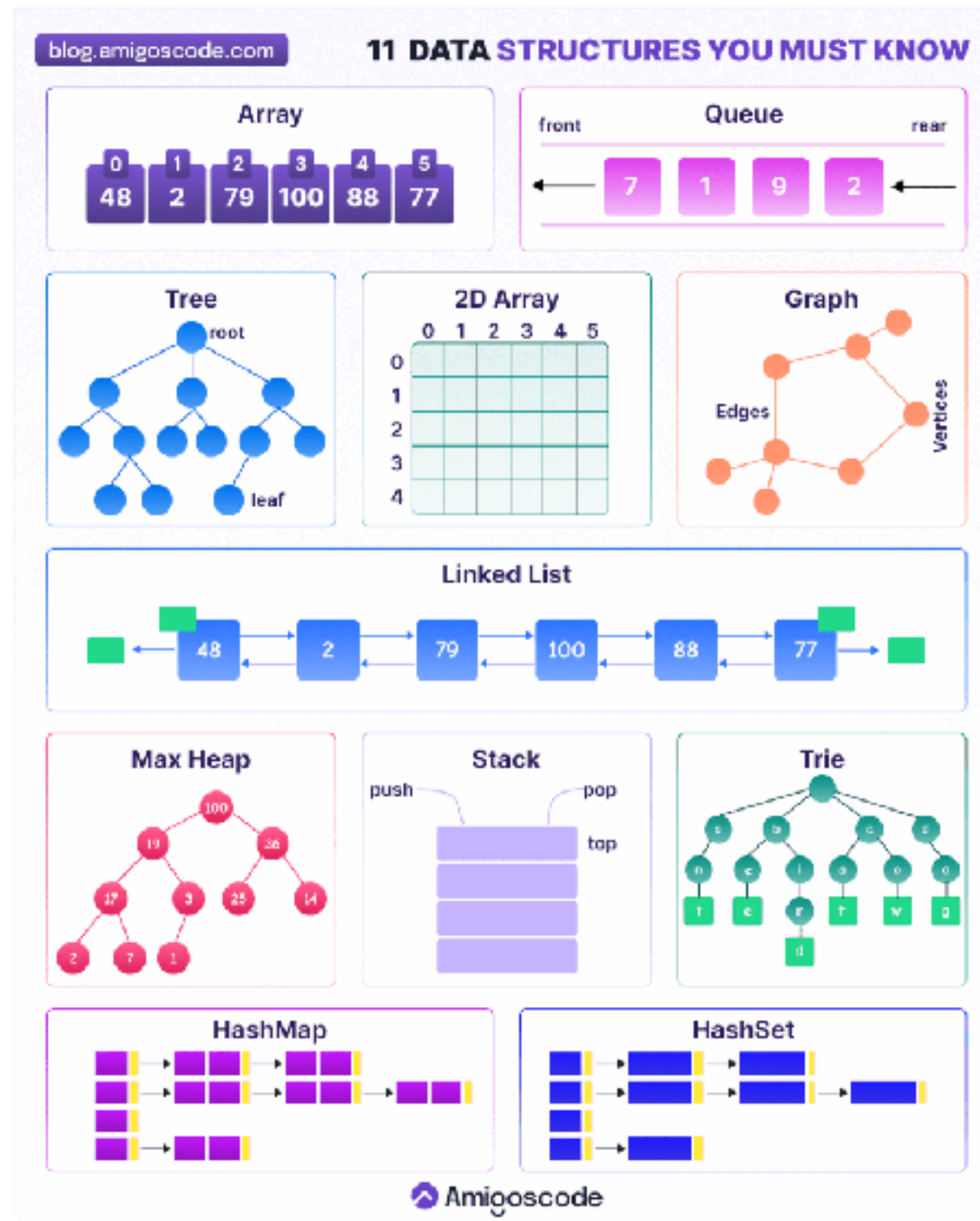
Write



# Database Models ?



# Data Structure



<https://blog.amigoscode.com/p/11-data-structures-every-developer>





# Database Models ?

424 systems in ranking, July 2025

| Rank     |          |                                                                                         | DBMS                                                                                        | Database Model                                                                                                 | Score    |          |          |
|----------|----------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|----------|----------|----------|
| Jul 2025 | Jun 2025 | Jul 2024                                                                                |                                                                                             |                                                                                                                | Jul 2025 | Jun 2025 | Jul 2024 |
| 1.       | 1.       | 1.                                                                                      | Oracle                                                                                      | Relational, Multi-model     | 1217.05  | -13.33   | -23.31   |
| 2.       | 2.       | 2.                                                                                      | MySQL                                                                                       | Relational, Multi-model     | 940.73   | -12.85   | -98.73   |
| 3.       | 3.       | 3.                                                                                      | Microsoft SQL Server                                                                        | Relational, Multi-model     | 771.14   | -5.61    | -36.51   |
| 4.       | 4.       | 4.                                                                                      | PostgreSQL                                                                                  | Relational, Multi-model     | 680.89   | +0.23    | +41.98   |
| 5.       | 5.       | 5.                                                                                      | MongoDB    | Document, Multi-model       | 403.83   | +0.99    | -25.99   |
| 6.       | 6.       |  7.    | Snowflake                                                                                   | Relational                                                                                                     | 176.17   | +1.68    | +39.64   |
| 7.       | 7.       |  6.  | Redis                                                                                       | Key-value, Multi-model    | 149.72   | -2.01    | -7.05    |
| 8.       | 8.       |  9.  | IBM Db2                                                                                     | Relational, Multi-model   | 127.51   | +2.38    | +3.11    |
| 9.       | 9.       |  8.  | Elasticsearch                                                                               | Multi-model               | 118.83   | -2.45    | -12.00   |
| 10.      | 10.      | 10.                                                                                     | SQLite                                                                                      | Relational                                                                                                     | 115.44   | -1.60    | +5.49    |
| 11.      | 11.      |  12. | Apache Cassandra                                                                            | Wide column, Multi-model  | 108.76   | +0.49    | +9.63    |
| 12.      | 12.      |  15. | Databricks                                                                                  | Multi-model               | 108.04   | +3.36    | +24.74   |
| 13.      | 13.      |  14. | MariaDB  | Relational, Multi-model   | 95.44    | +0.91    | +4.85    |
| 14.      | 14.      |  11. | Microsoft Access                                                                            | Relational                                                                                                     | 90.47    | +2.19    | -10.17   |
| 15.      | 15.      |  17. | Amazon DynamoDB                                                                             | Multi-model               | 84.15    | +0.81    | +13.20   |

<https://db-engines.com/en/ranking>





# Database Model

**Relational**

Document

Key-value

Search engine

Wide column

Graph

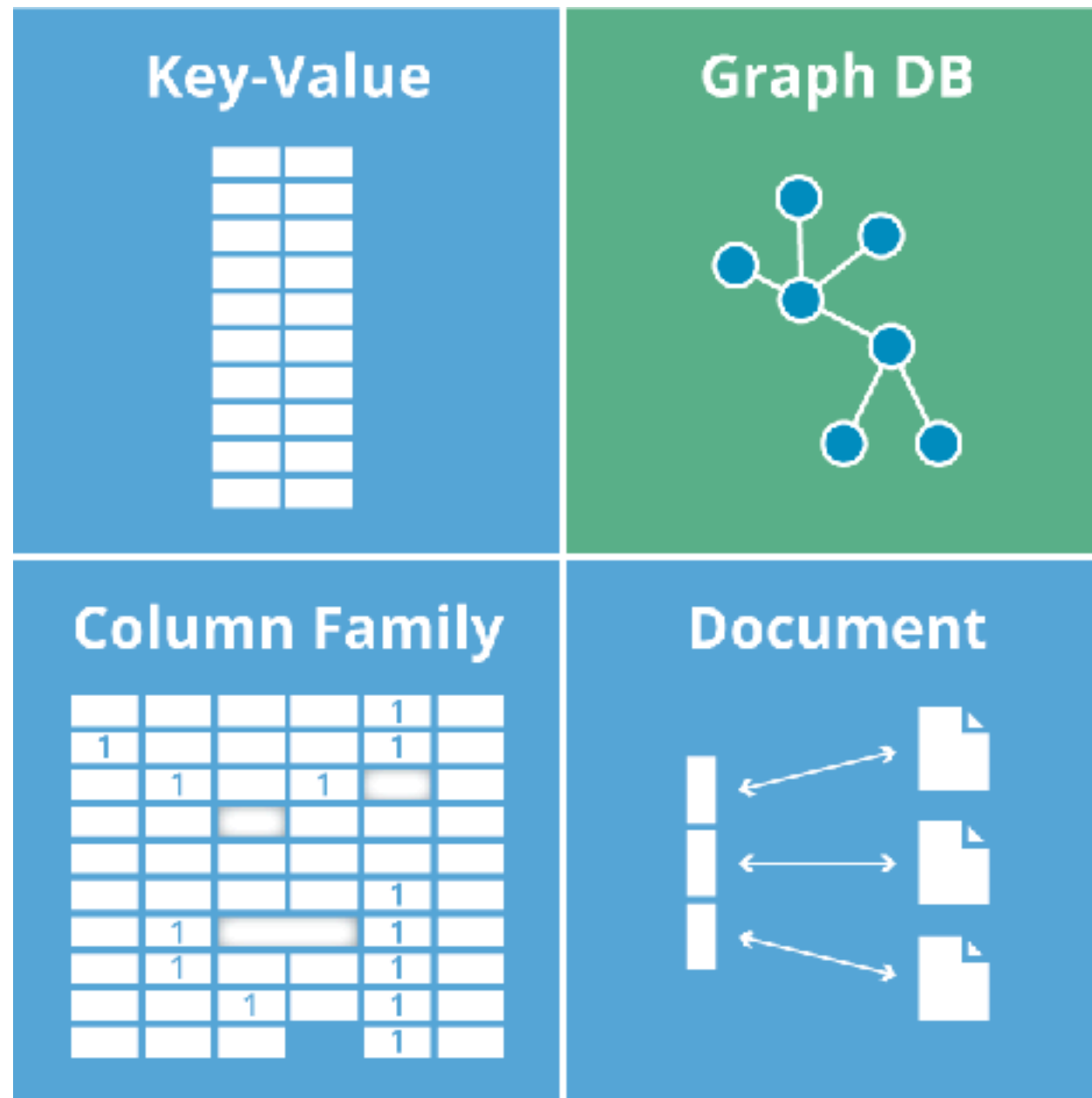
Time series

Vector

**Non-Relational**  
**NoSQL**



# NoSQL



# Relational Database



PostgreSQL



**ORACLE®**  
DATABASE

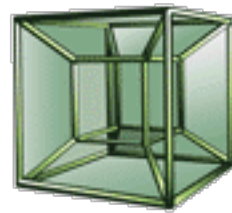


# NoSQL

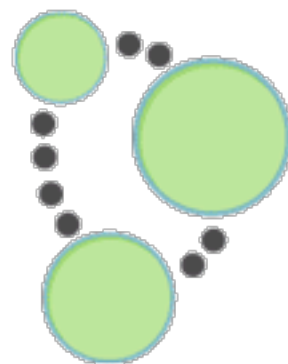
APACHE  
**HBASE**



**riak**



**HYPERTABLE** INC

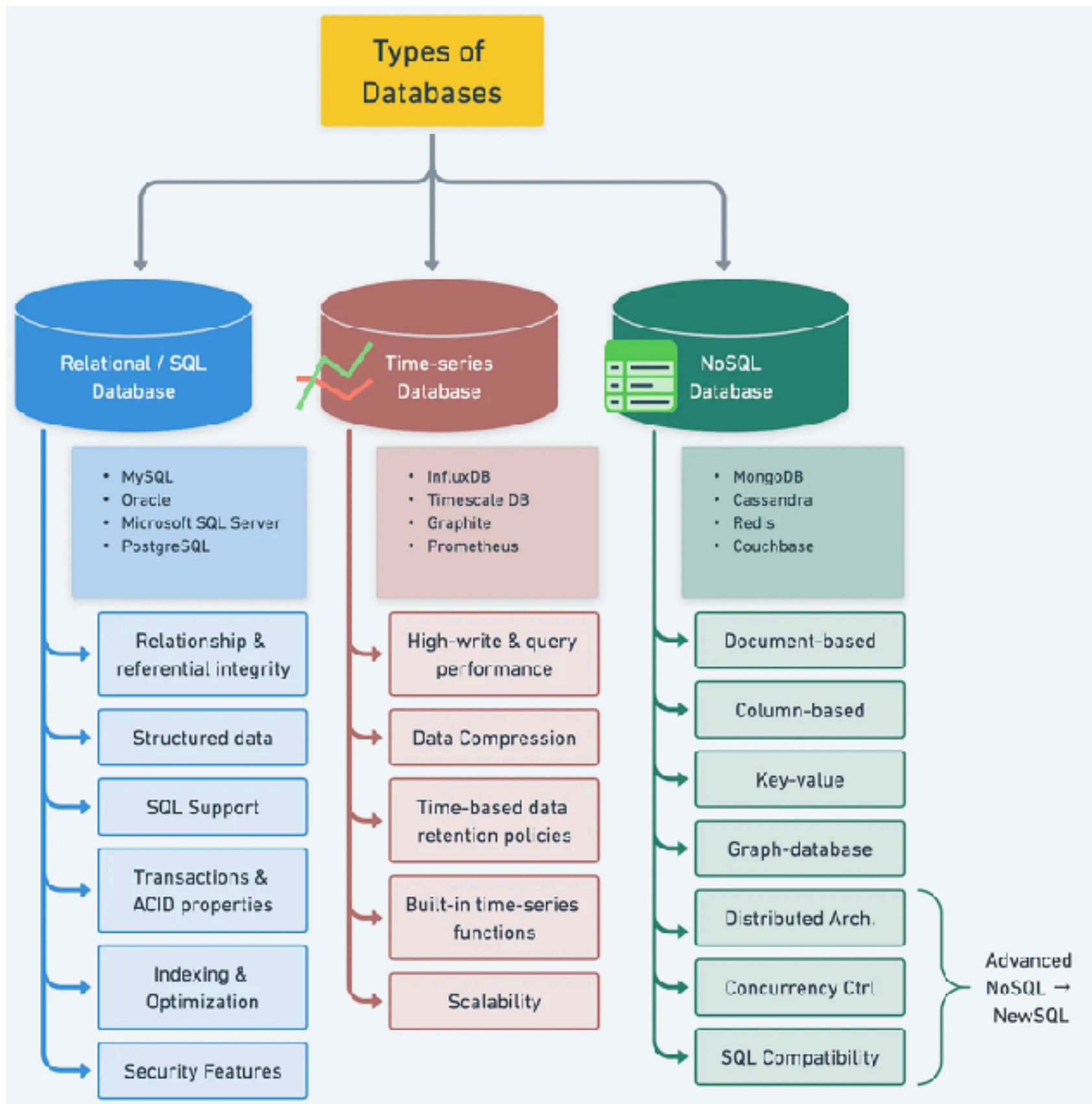


**Neo4j**



**redis**





<https://blog.bytebytego.com/p/understanding-database-types>



# Relational Database ?



# Relational Database ?

**Relational DataBase Managment System**

Maintain data in **tables**

**Pre-define** structure

Each table has **Primary key**

Many **columns**

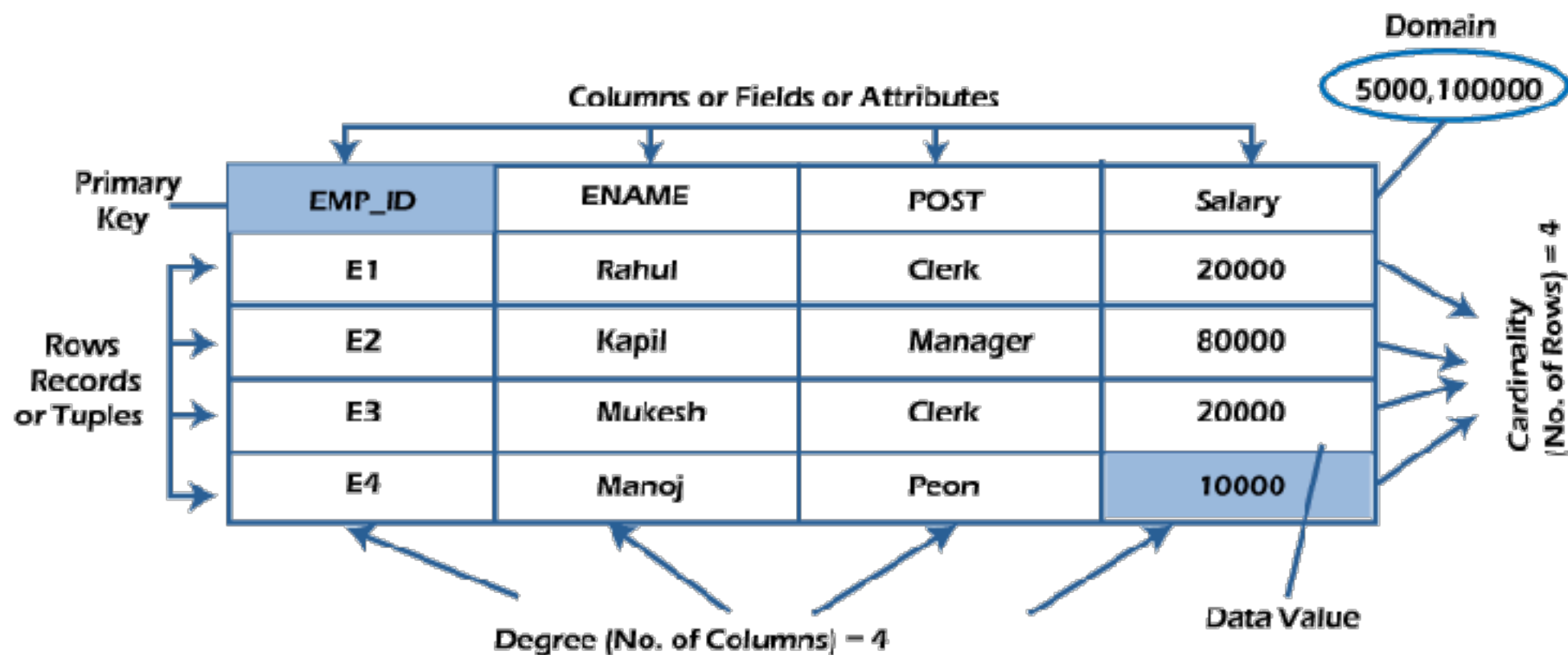
Data is represented in term of **row**





# Relational Database ?

Table = Employee



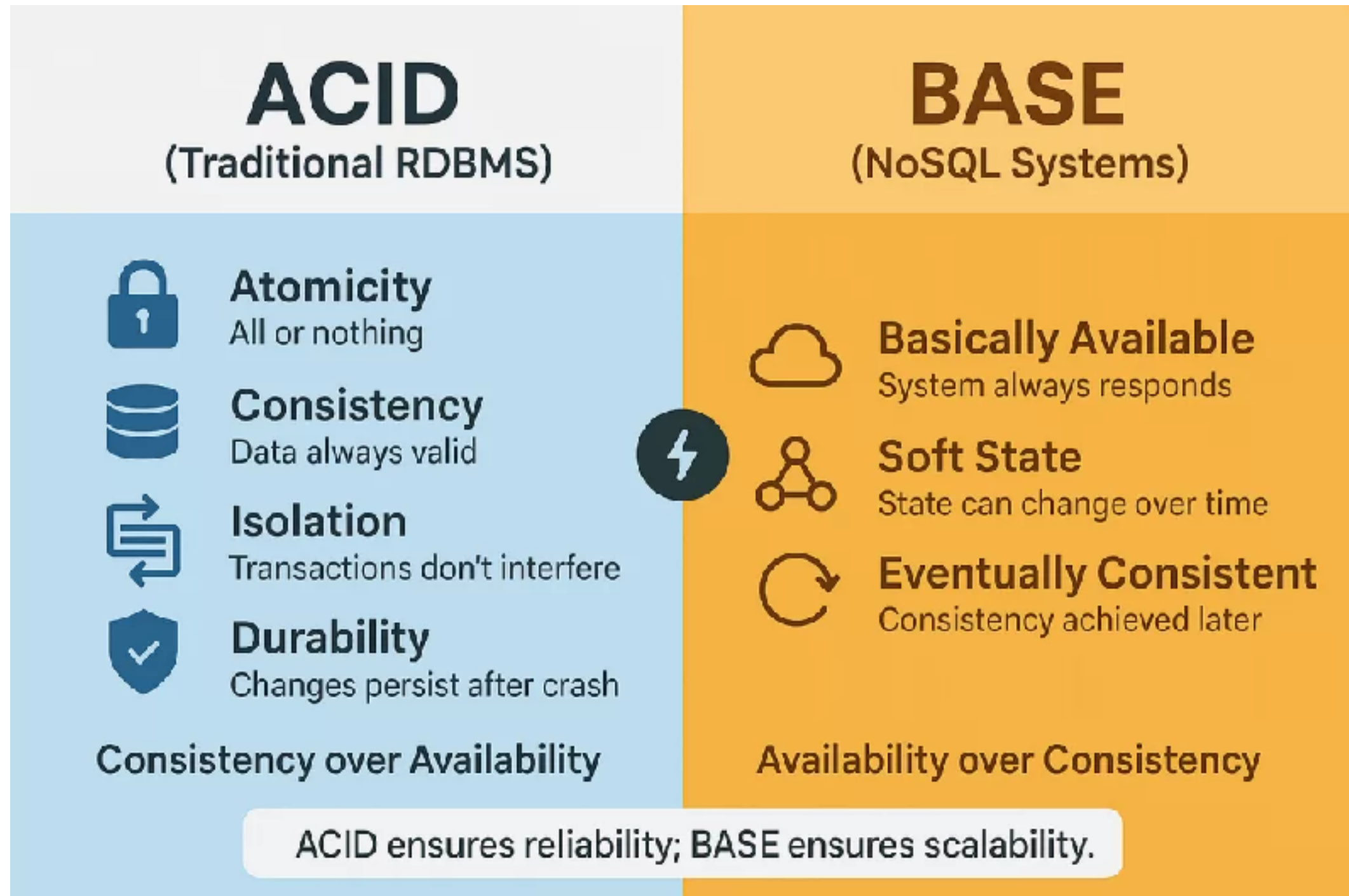


# Relational Database

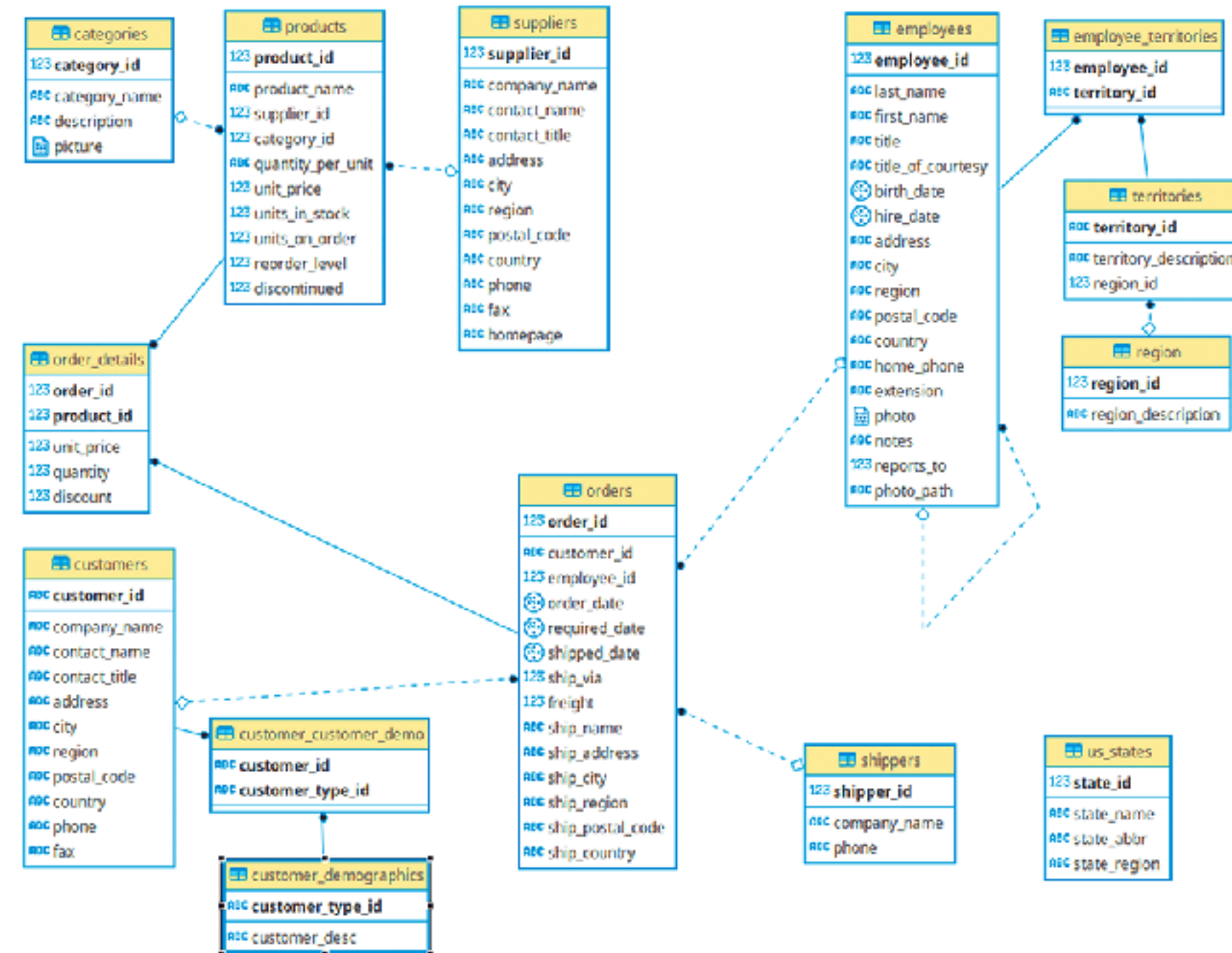
| Database     | Ease of use | Support complex query | Community support | Licence     |
|--------------|-------------|-----------------------|-------------------|-------------|
| MySQL        | High        | High                  | Strong            | Open source |
| PostgreSQL   | Medium      | Very high             | Strong            | Open source |
| MSSQL Server | Medium      | Very high             | Strong            | Proprietary |
| Oracle       | Low         | Very high             | Moderate          | Proprietary |



# ACID vs BASE



# Real Design of RDBMS ?



# Questions ?

How to write data ?

How to read data ?

Aggregate data ?

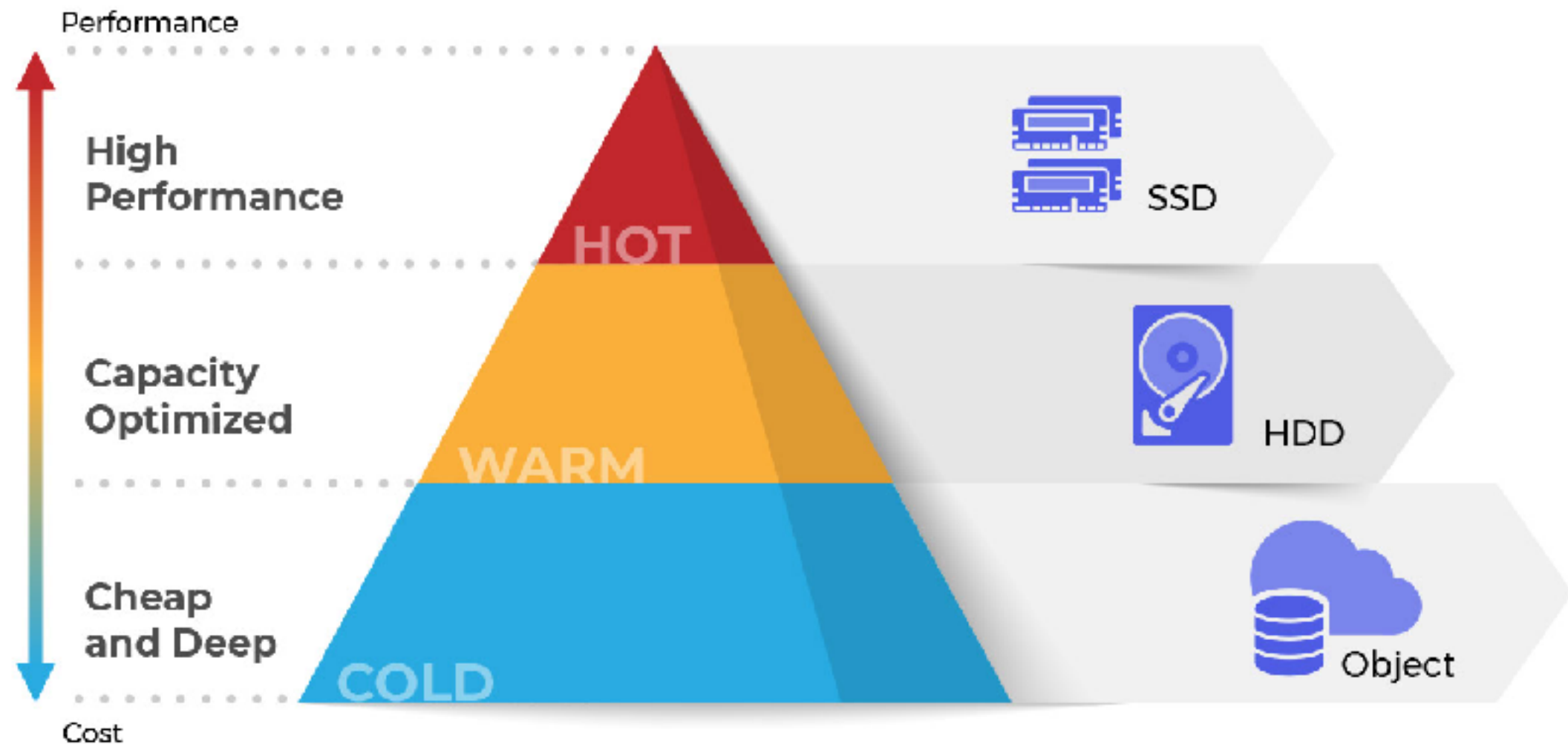
Reporting

Data retention ?



# Questions ?

Hot vs Warm vs Cold data ?

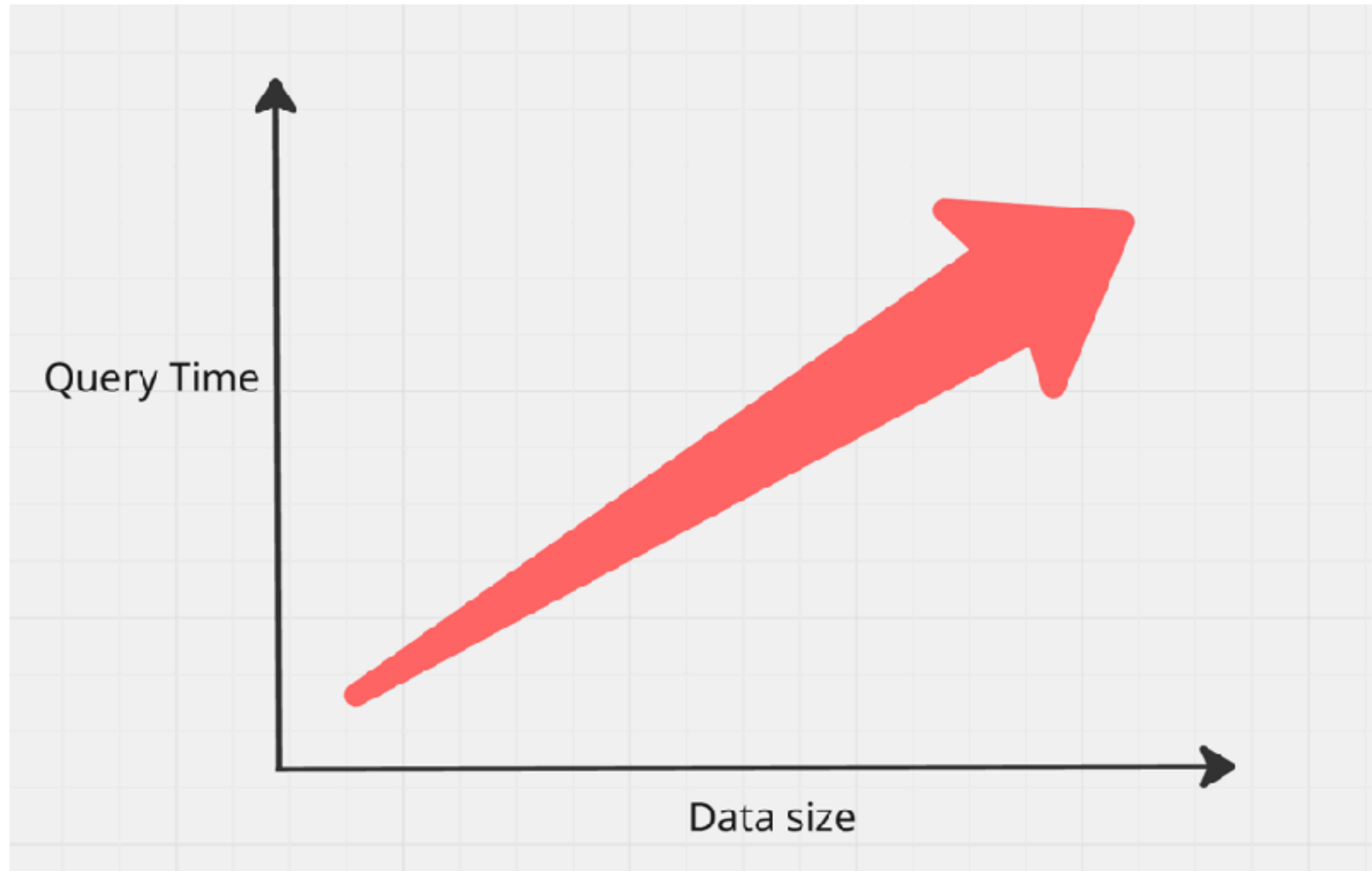


# Complex Query ?

```
1  WITH tmp_1 AS
2  (
3  SELECT Calc1 =
4  ( (SELECT TOP 1 DataValue
5  FROM (
6  SELECT TOP 50 PERCENT DataValue
7  FROM SOMEDATA
8  WHERE DataValue IS NOT NULL
9  ORDER BY DataValue
10 ) AS A
11 ORDER BY DataValue DESC
12 ) + (SELECT TOP 1 DataValue
13 FROM (
14 SELECT TOP 50 PERCENT DataValue
15 FROM SOMEDATA
16 WHERE DataValue Is NOT NULL
17 ORDER BY DataValue DESC
18 )AS A
19 ORDER BY DataValue ASC))/2
20 ),tmp_2 AS
21 (SELECT AVG(DataValue) Mean, MAX(DataValue) - MIN(DataValue) AS MaxMinRange
22 FROM SOMEDATA
23 ),tmp_3 AS
24 (
25 SELECT TOP 1 DataValue AS Calc2, COUNT(*) AS Calc2Count
26 FROM SOMEDATA
27 GROUP BY DataValue
28 ORDER BY Calc2Count DESC
29 )
30 SELECT Mean, Calc1, Calc2 , MaxMinRange AS [Range]
31 FROM tmp_1 CROSS JOIN tmp_2 CROSS JOIN tmp_3;
```

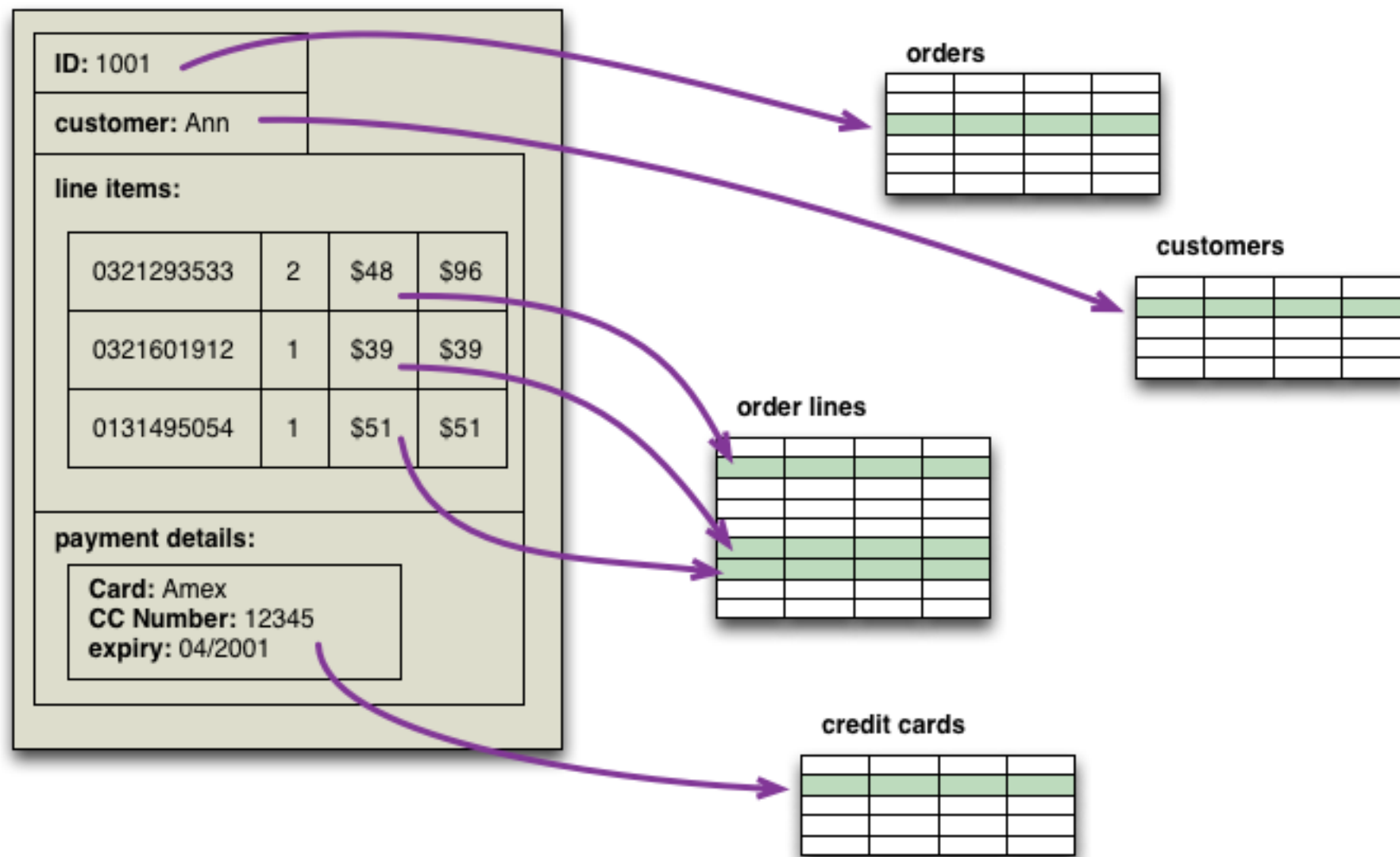


# Problems ?





# NoSQL



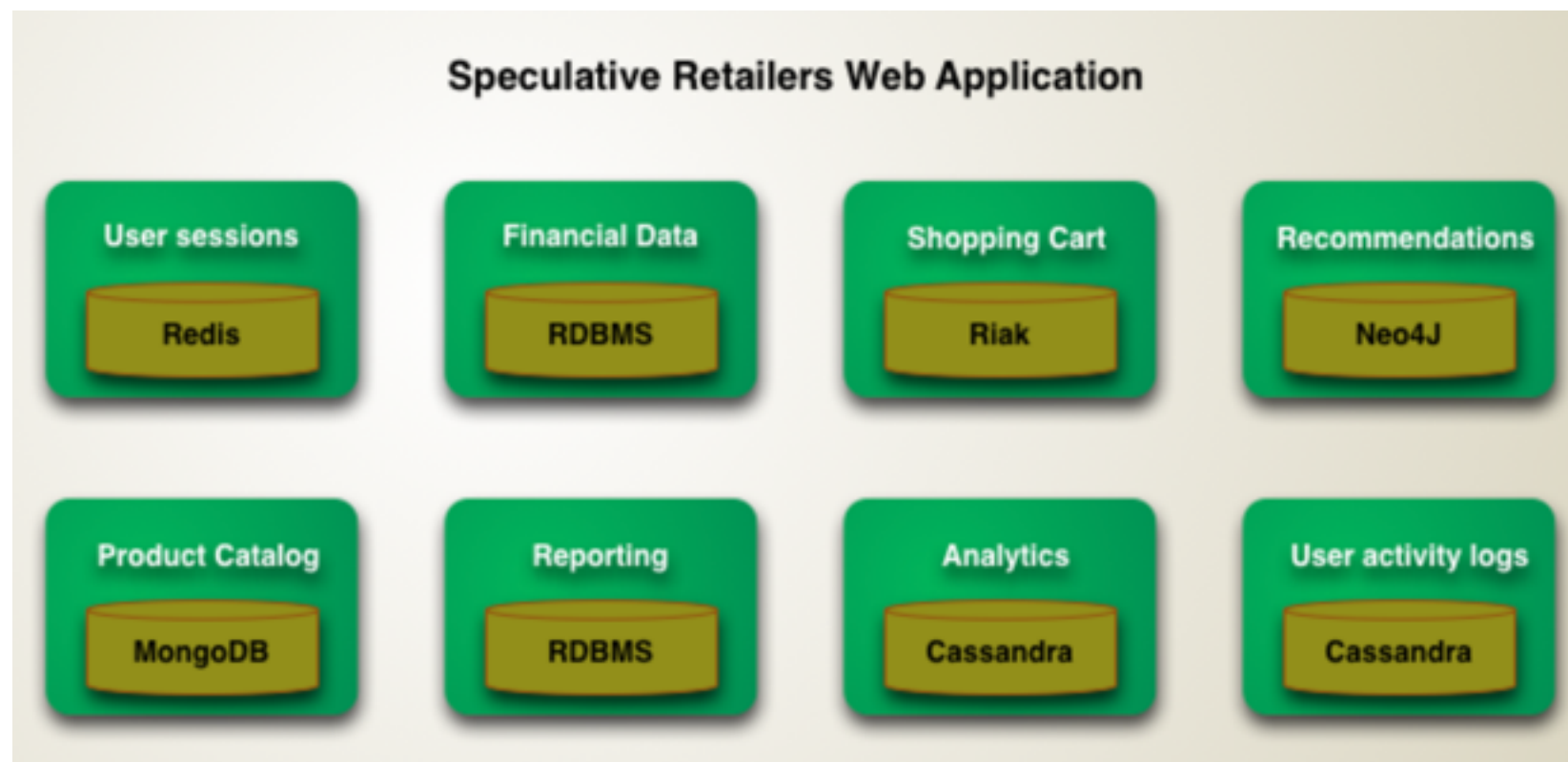
<https://martinfowler.com/bliki/AggregateOrientedDatabase.html>





# Polyglot Persistence

Right tool for the right job



<https://martinfowler.com/bliki/PolyglotPersistence.html>



# Data Modeling

Read

Write



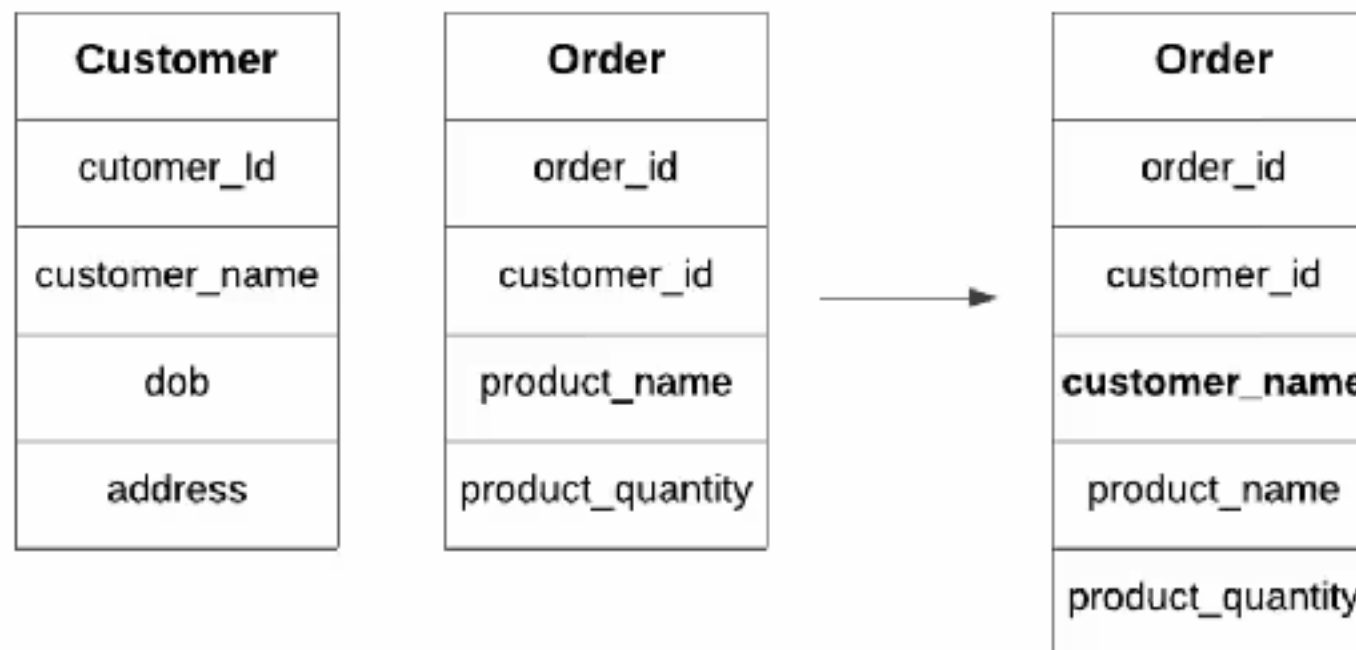
# Data Modeling

Normalization

Denormalization

Pre-joins, pre-aggregation

Caching strategies



# Normalization ?

Cleansing data

Reorganized data

Remove redundant data

Improve efficient for store data

***Organize data in consistent way***



# Normalization

Divide in to several tables  
Link tables with **relationships !!**

Design for write or read ?

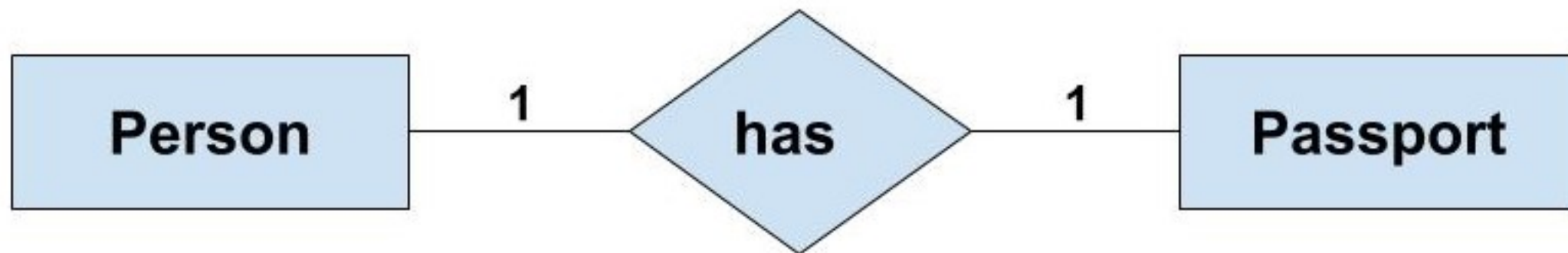


# Relationships



# Relationship between table

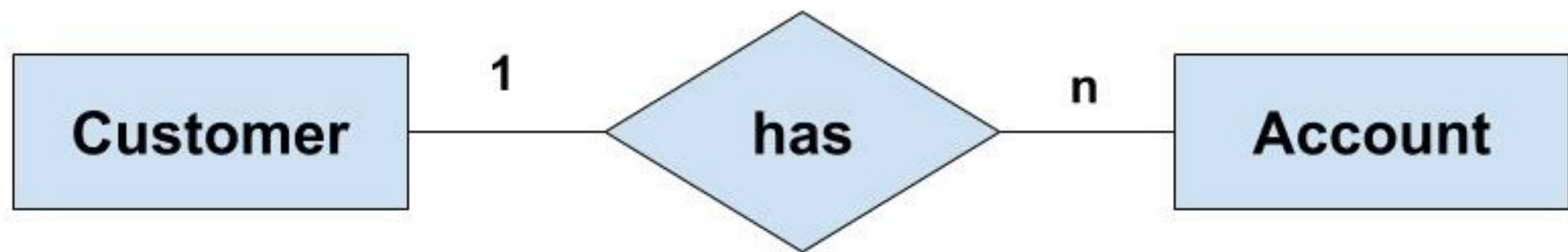
One-to-one  
One-to-many  
Many-to-many





# Relationship between table

One-to-one  
**One-to-many**  
Many-to-many





# Relationship between table

One-to-one  
One-to-many  
**Many-to-many**

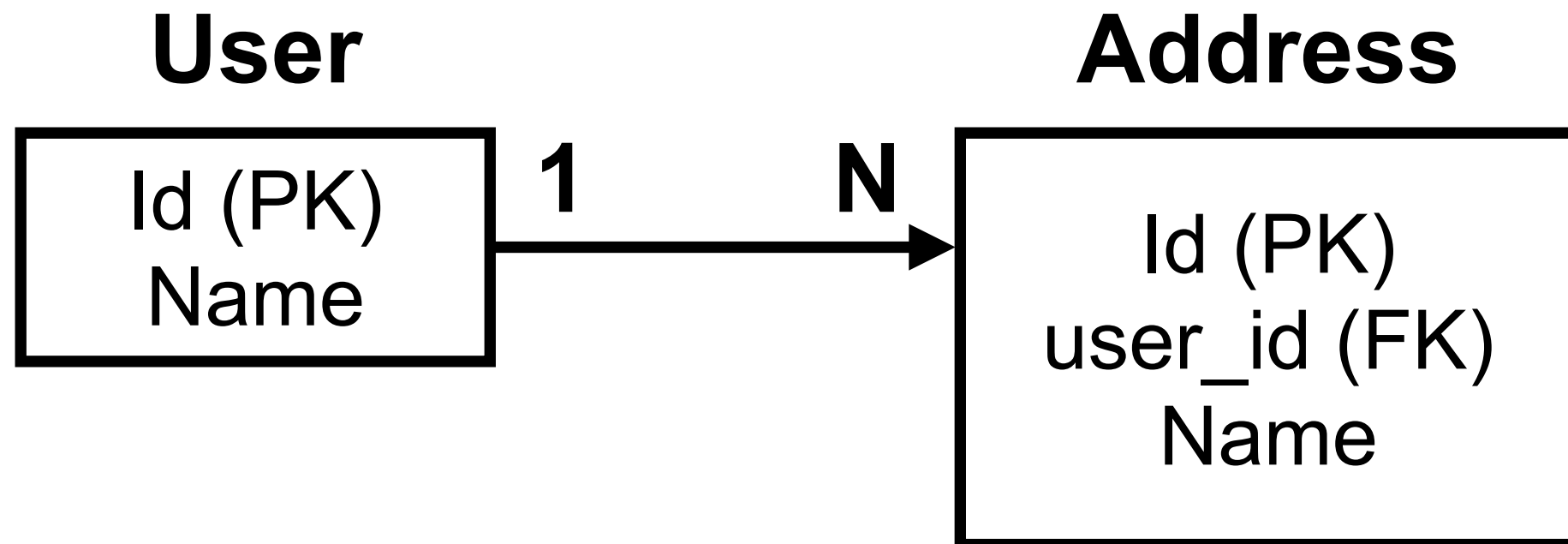


# Relationships

Primary key (PK)

Composite key

Foreign key (FK)



# Normalization forms

**First Normal form (1NF)**

**2NF**

**3NF**

**4NF**

**5NF**

**6NF**



# First Normal Form (1NF)

No repeat data in table

Every column should only have single value

Every row should be unique



# Sample data in 1NF

| Name   | Address   | Gender | T-shirt Size |
|--------|-----------|--------|--------------|
| User 1 | Address 1 | Male   | Large        |
| User 2 | Address 2 | Female | Small        |
| User 2 | Address 2 | Female | Medium       |
| User 3 | Address 3 | Male   | Medium       |



# Second Normal Form (2NF)

Apply 1NF

Have one primary key



# Problem in 1NF

| Name   | Address   | Gender | T-shirt Size |
|--------|-----------|--------|--------------|
| User 1 | Address 1 | Male   | Large        |
| User 2 | Address 2 | Female | Small        |
| User 2 | Address 2 | Female | Medium       |
| User 3 | Address 3 | Male   | Medium       |





# Sample data in 2NF

| User ID | Name   | Address   | Gender |
|---------|--------|-----------|--------|
| 1       | User 1 | Address 1 | Male   |
| 2       | User 2 | Address 2 | Female |
| 3       | User 3 | Address 3 | Male   |





# Sample data in 2NF

| User ID | T-shirt Size |
|---------|--------------|
| 1       | Large        |
| 2       | Small        |
| 2       | Medium       |
| 3       | Medium       |



# Third Normal Form (3NF)

Apply 2NF

It should only be dependent on the primary key

if any changes to the primary key are made,  
all data that is impacted must be put into a  
new table



# Problem in 2NF

| User ID | Name   | Address   | Gender |
|---------|--------|-----------|--------|
| 1       | User 1 | Address 1 | Male   |
| 2       | User 2 | Address 2 | Female |
| 3       | User 3 | Address 3 | Male   |



# Sample data in 3NF

| User ID | Name   | Address   | Gender |
|---------|--------|-----------|--------|
| 1       | User 1 | Address 1 | 1      |
| 2       | User 2 | Address 2 | 2      |
| 3       | User 3 | Address 3 | 1      |



# Sample data in 3NF

| Gender ID | Gender name |
|-----------|-------------|
| 1         | Male        |
| 2         | Female      |



# Sample data in 3NF

| User ID | T-shirt Size |
|---------|--------------|
| 1       | Large        |
| 2       | Small        |
| 2       | Medium       |
| 3       | Medium       |



# 3NF is normal form ...





# Benefits of data normalization

Freeing up space  
Improve query response time  
Reduce data inconsistency  
Easier to use and analyze





# Challenges of data normalization

Slower query response rates

Accurate knowledge is required

Add complexity for teams

Denormalization as an alternative !!



# Primary Key (PK)



# Foreign Key (FK)



# Other Constraints

Ensure data quality

Prevent inconsistency at the database level

Not Null

Unique

Check

Default



# Data Types

<https://www.postgresql.org/docs/current/datatype.html>



# Data Types ?

| Data types           | Memory (storage) | Use cases                           |
|----------------------|------------------|-------------------------------------|
| <b>SMALLINT</b>      | 2 bytes          | Small length data<br>Status code    |
| <b>INTEGER</b>       | 4 bytes          | Standard choice for<br>ID, count    |
| <b>BIGINT/SERIAL</b> | 8 bytes          | Large data set<br>(Billion records) |
| <b>UUID</b>          | 16 bytes         | Unique id in distributed<br>system  |
| <b>JSONB</b>         | Variable         | Binary-parsed JSON                  |



# UUID ?

## Universal Unique ID

| Version   | Basis          | Sortable | Use cases                                                            |
|-----------|----------------|----------|----------------------------------------------------------------------|
| <b>V4</b> | Random         | No       | Non-ordered identifier<br>Temporary key                              |
| V5        | Namespace hash | No       | Deterministic/reproducible ID                                        |
| V6        | Time (ordered) | Yes      | Sequential ID in distributed system<br>with network card information |
| <b>V7</b> | Time (UNIX)    | Yes      | Sequential ID in distributed system<br>with unit timestamp           |

<https://www.postgresql.org/docs/current/datatype-uuid.html>





# UUID V4 (Random)

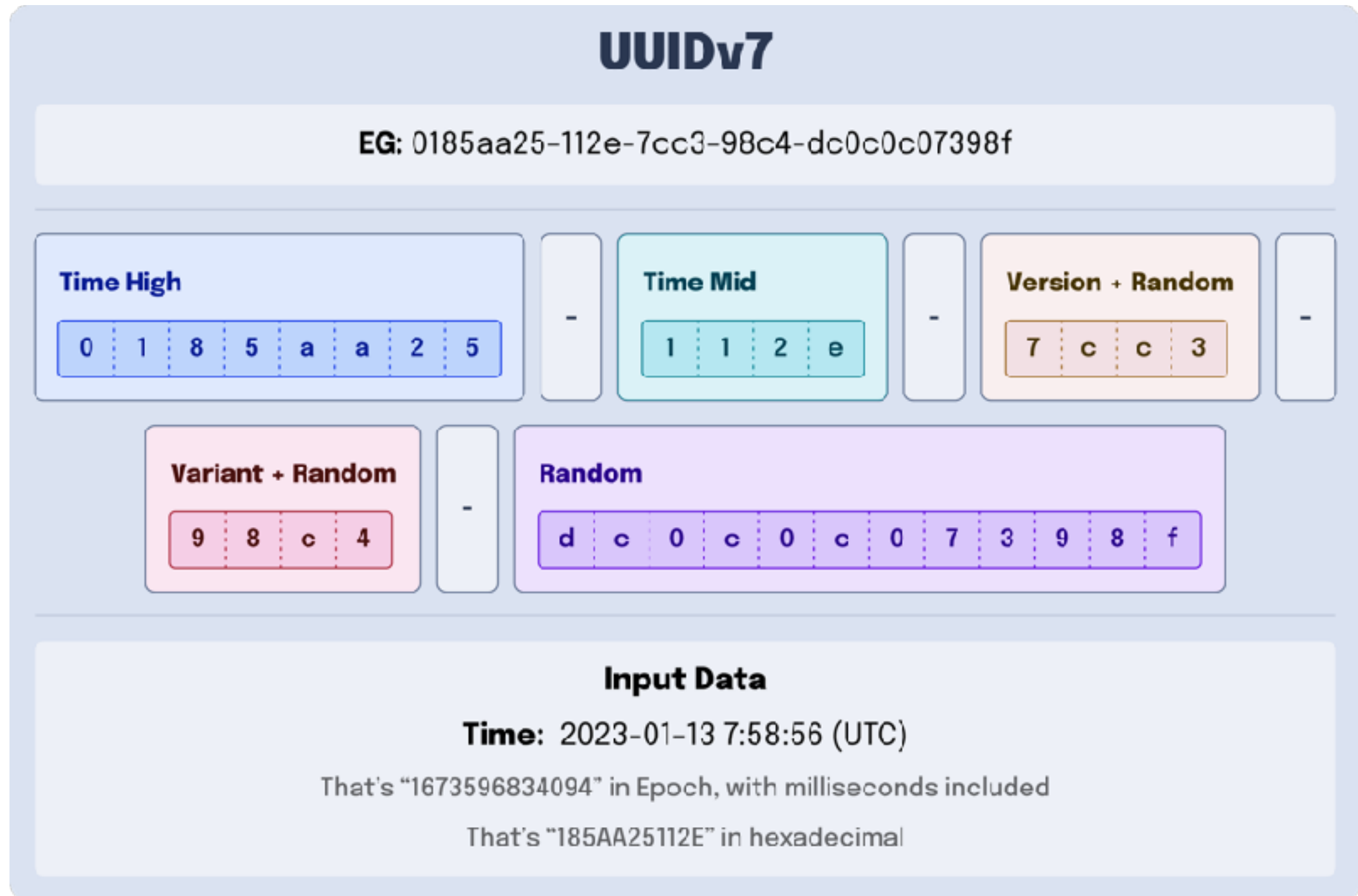


<https://playfulprogramming.com/posts/uuid-v6-8/>





# UUID V7

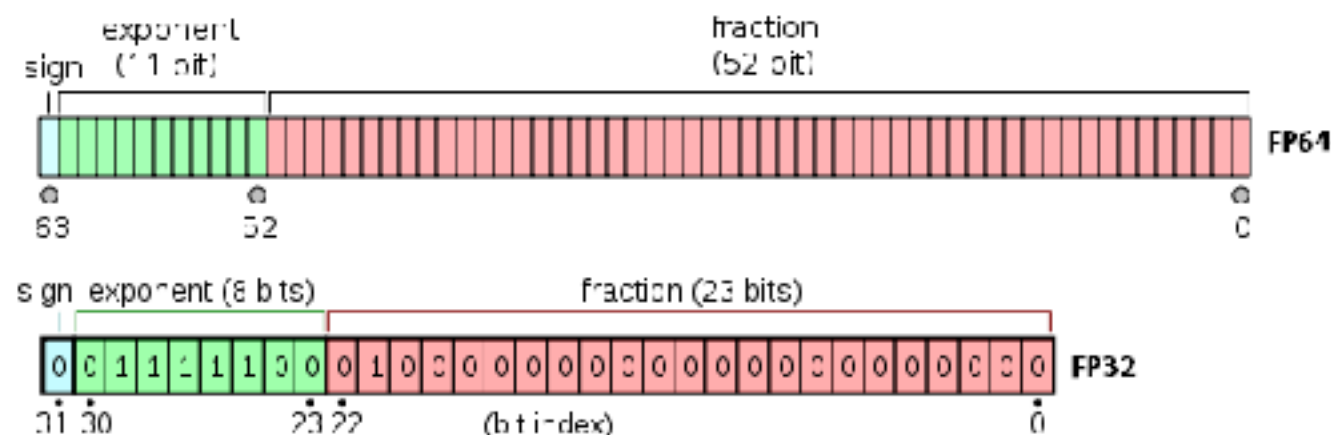


<https://playfulprogramming.com/posts/uuid-v6-8/>



# Data Types ?

| Data types     | Memory (storage) | Use cases                        |
|----------------|------------------|----------------------------------|
| <b>REAL</b>    | 4 bytes          | Single precision floating-point  |
| <b>DOUBLE</b>  | 8 bytes          | Double precision floating-point  |
| <b>NUMERIC</b> | Variable         | Exact arithmetic in banking      |
| <b>Money</b>   | 8 bytes          | Fixed-fractional currency amount |



# String Data Types ?

| Features   | Char(n)           | Varchar(n)        | Text/CLOB                  |
|------------|-------------------|-------------------|----------------------------|
| Max length | N characters      | Physical          | ~ 1 GB (Unlimited)         |
| Storage    | Blank-padded to n | Content only      | Content only + compression |
| Use cases  | Fixed-length data | General use cases | Long-form content, logs    |



# Workshop with PostgreSQL



PostgreSQL



# PostgreSQL Database

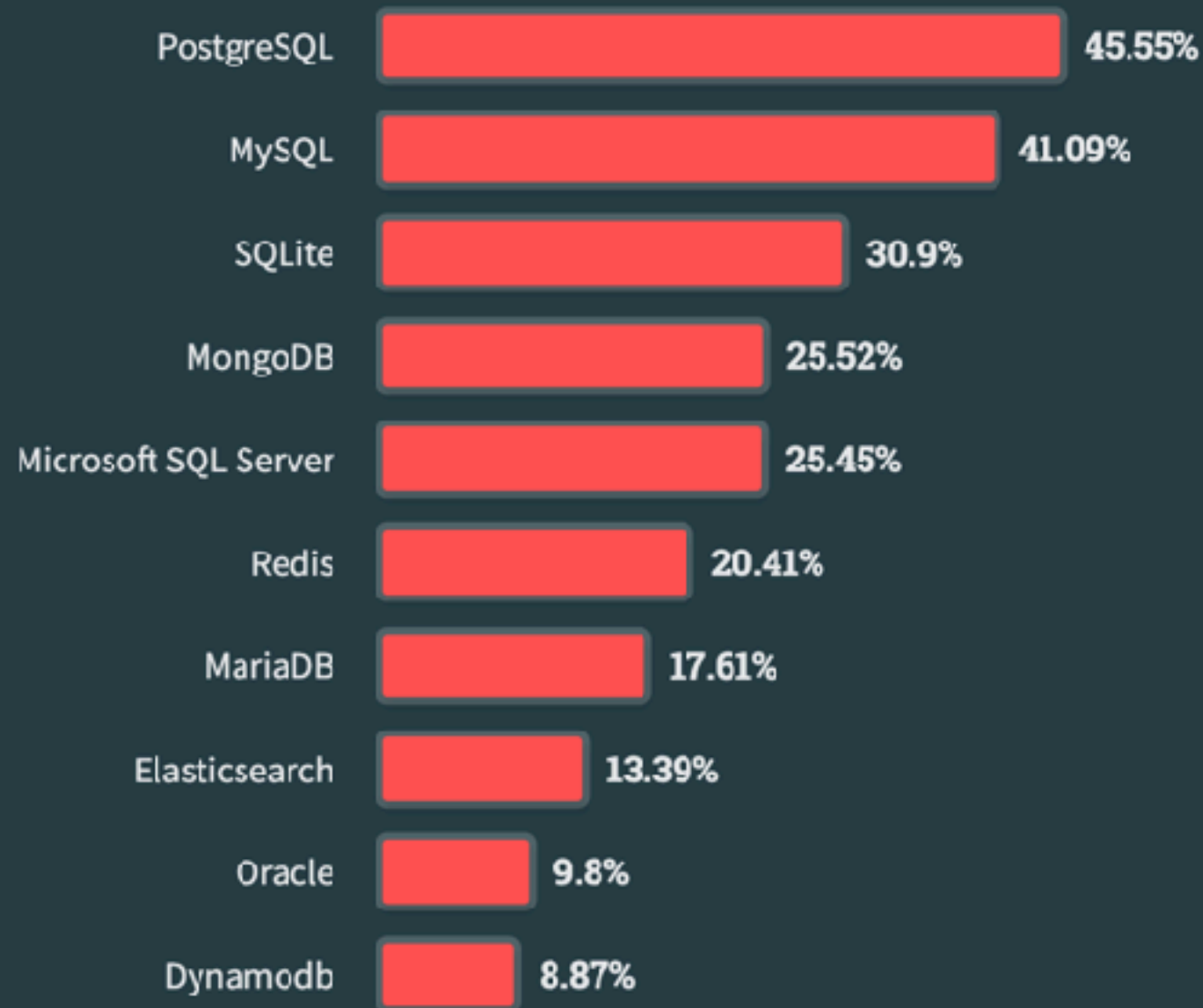
All Respondents

Professional Developers

Learning to Code

Other Coders

76,634 responses



<https://survey.stackoverflow.co/2023/#most-popular-technologies-database>



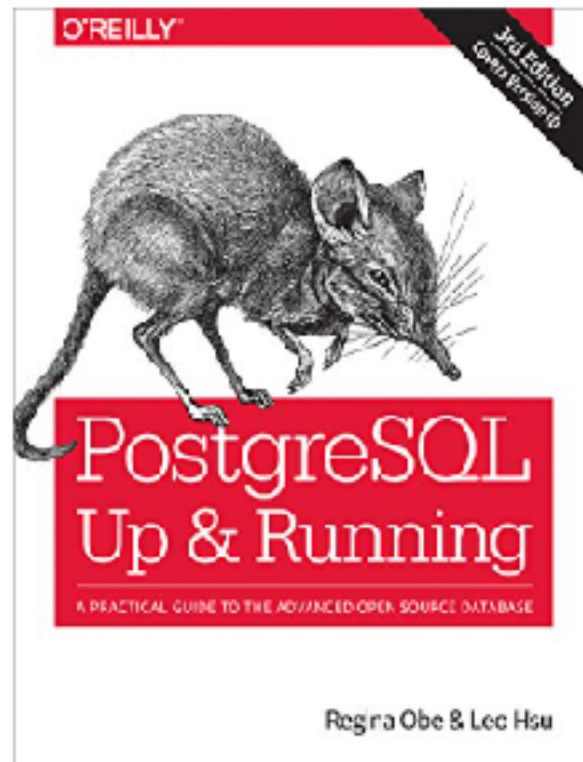
# Book Store

QuickStart  
Guides

Master SQL and Manage, Analyze and Manipulate Data

\$25<sup>19</sup>

Kindle Store › Kindle eBooks › Computers & Technology



Read sample

Follow the Author



Regina Obe

Follow

## PostgreSQL: Up and Running: A Practical Guide to the Advanced Open Source Database 3rd Edition, Kindle Edition

by Regina O. Obe (Author), Leo S. Hsu (Author) | Format: Kindle Edition

4.4 ★★★★★ 99 ratings

[See all formats and editions](#)

Kindle  
\$10.09 - \$28.99

Read with Our Free App

Paperback  
\$44.99

6 Used from \$36.00  
20 New from \$34.18

Thinking of migrating to PostgreSQL? This clear, fast-paced introduction helps you understand and use this open source database system. Not only will you learn about the enterprise class features in versions 9.5 to 10, you'll also discover that PostgreSQL is more than a database system—it's an impressive application platform as well.

With examples throughout, this book shows you how to achieve tasks that are difficult or impossible in other databases. This third edition covers new features, such as ANSI-SQL constructs found only in proprietary databases until now: foreign data wrapper (FDW) enhancements, new full-text search and partitioning introduced in version 9.5, and more. [Read more](#)

ISBN-13



978-1491963418

Edition



3rd

Sticky notes



[On Kindle Scribe](#)

Publisher



O'Reilly Media

Publication date



October 10, 2017

Language



English

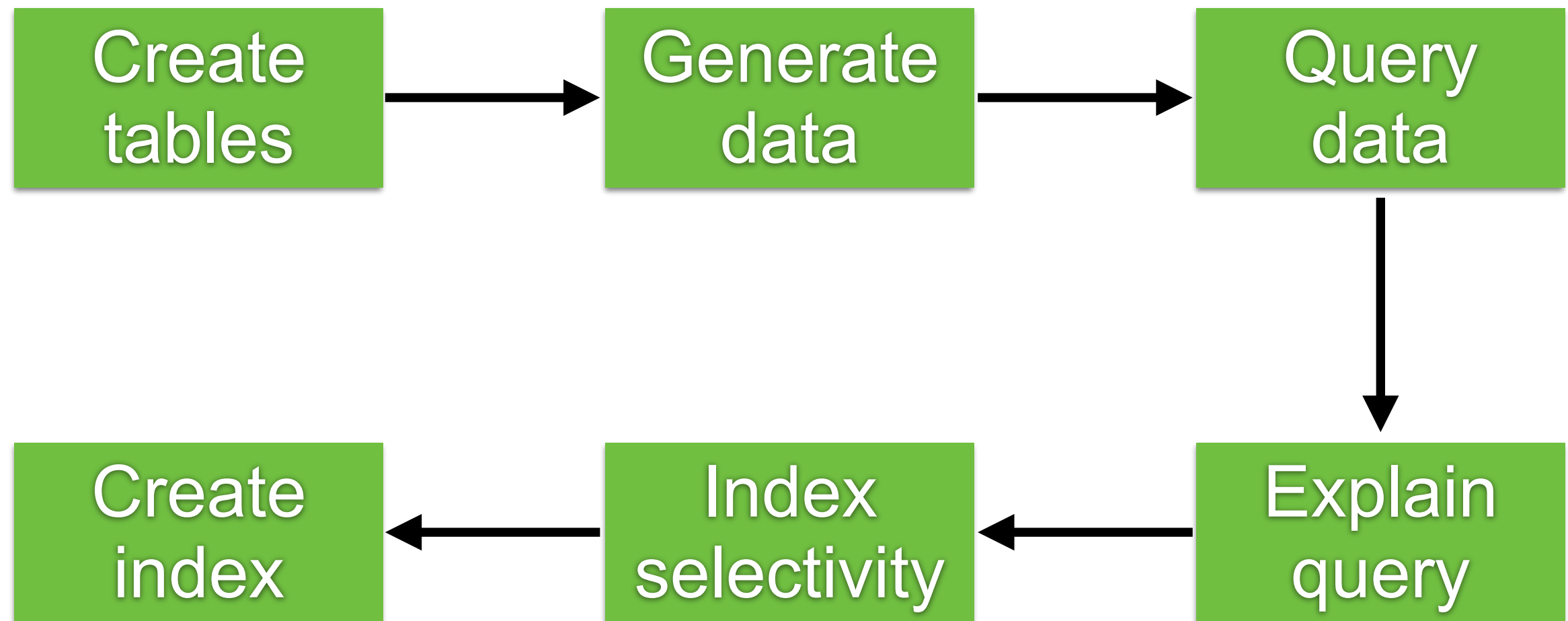


# Book Store





# Step in workshop



<https://github.com/up1/workshop-database/tree/main/workshop/basic-workshop>



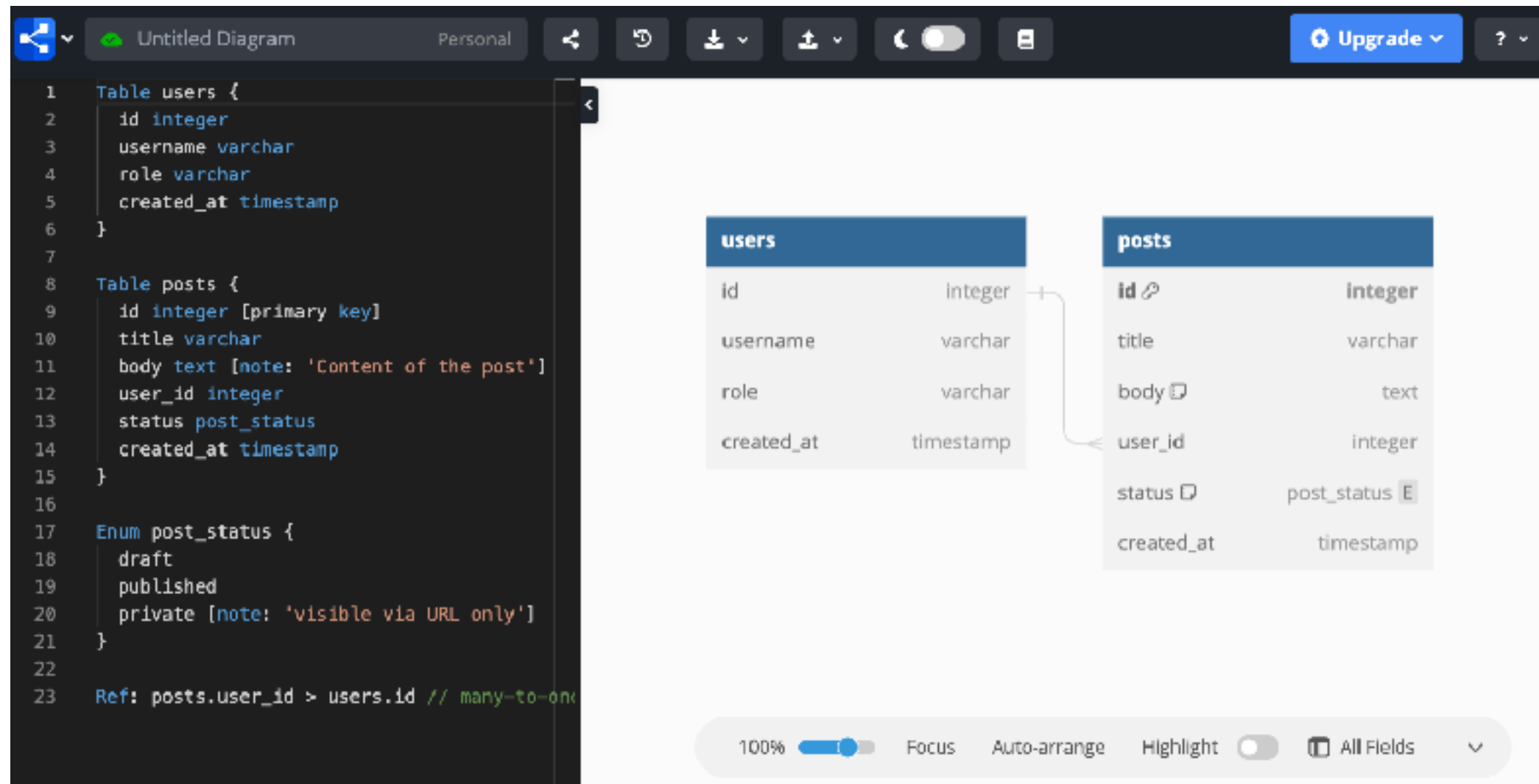


# Design



# Design table with DBML

## Database Markup Language

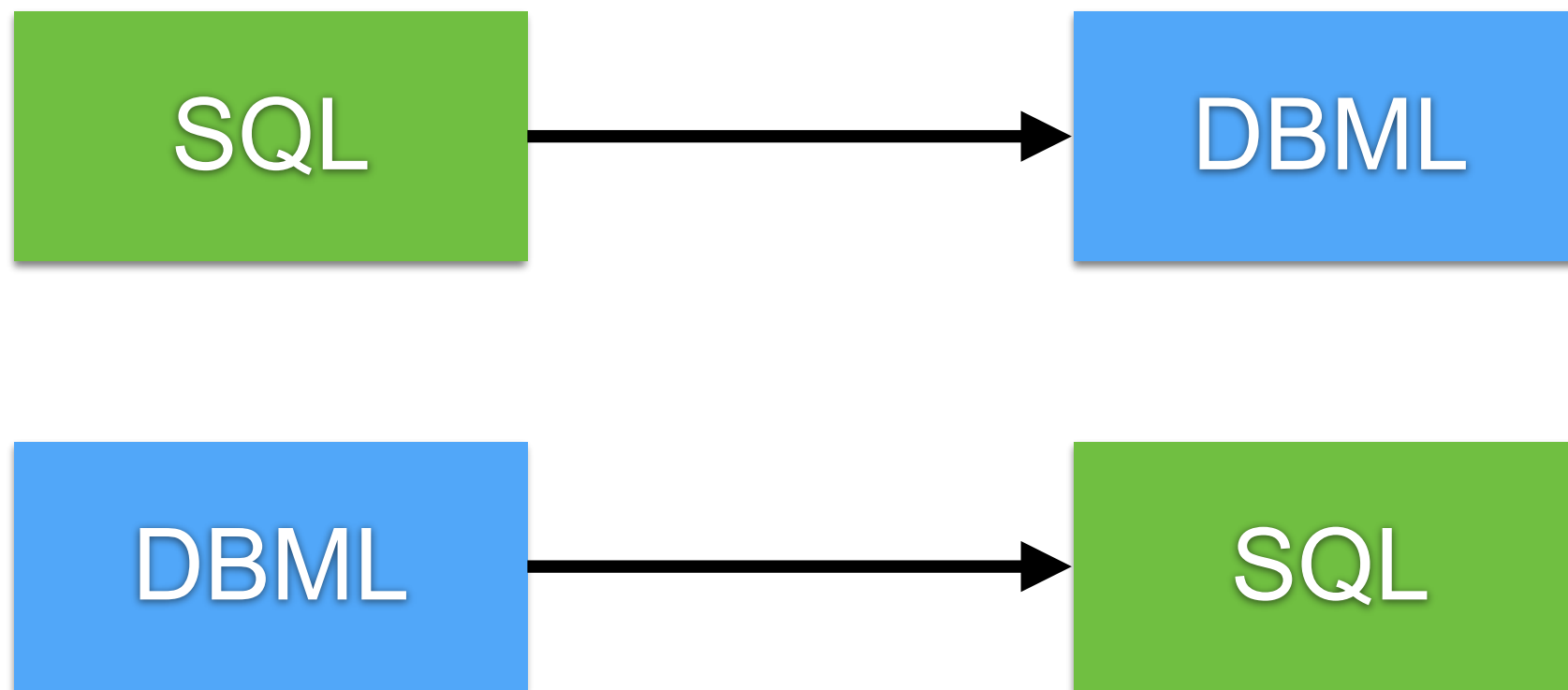


<https://dbml.dbdiagram.io/home/>



# Convert SQL to DBML

```
$npm install -g @dbml/cli
```

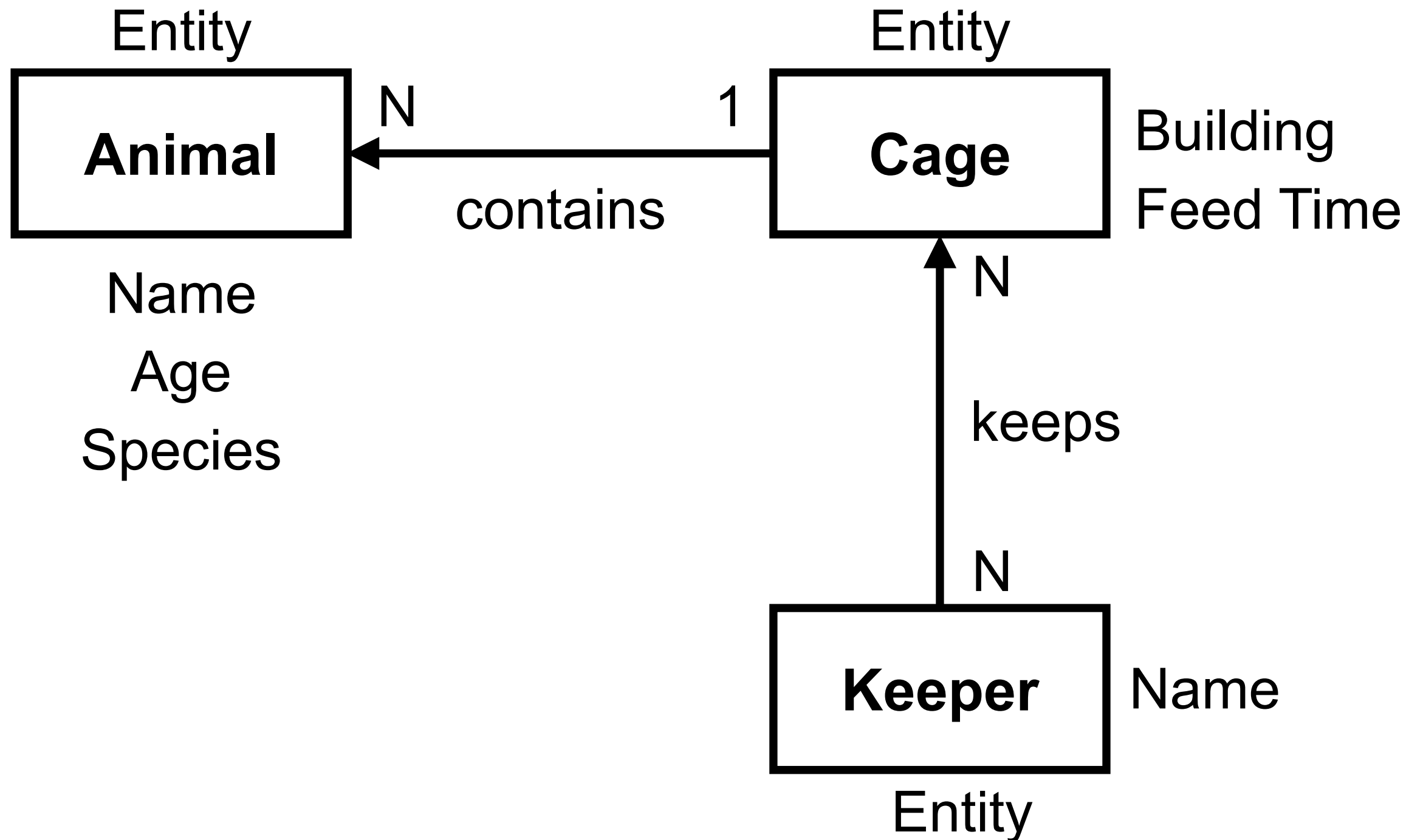


<https://dbml.dbdiagram.io/cli/#convert-a-sql-file-to-dbml>



# Design with entity relation diagram





# How to manage RDBMS ?

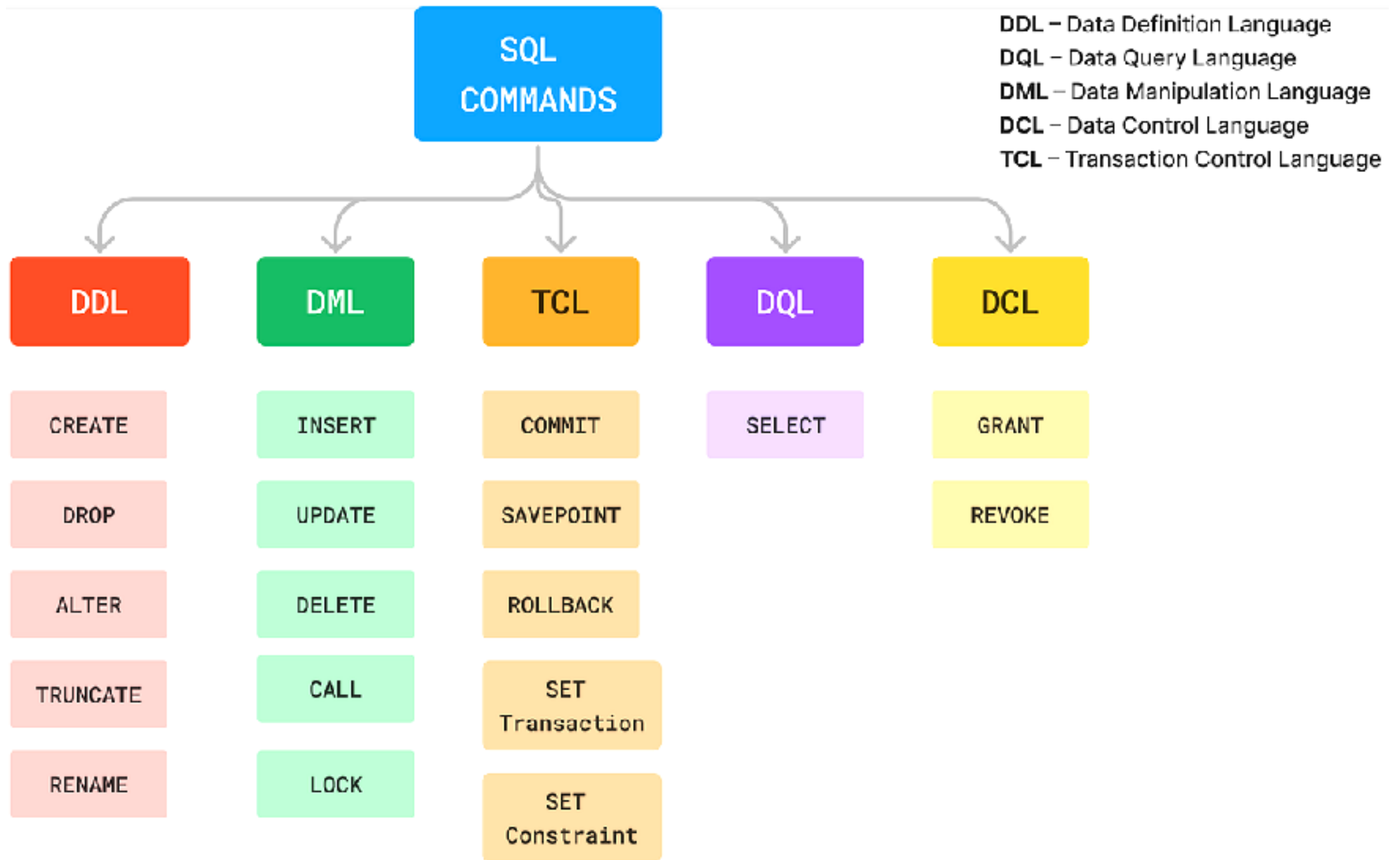


# Database Language

**Data Definition Language (DDL)**  
**Data Manipulation Language (DML)**  
Data Control Language (DCL)  
Transaction Control Language (TCL)









# Data Definition Language (DDL)

Create  
Alter  
Drop  
Truncate



# Data Manipulation Language (DML)

Select  
Insert  
Update  
Delete



# Delete vs Truncate ?



# Delete vs Truncate

## Delete

Row-level operation

Mark every row matching the WHERE-clause as deleted

Slow operation

## Truncate

Table operation

Empty the table

Start a new data file



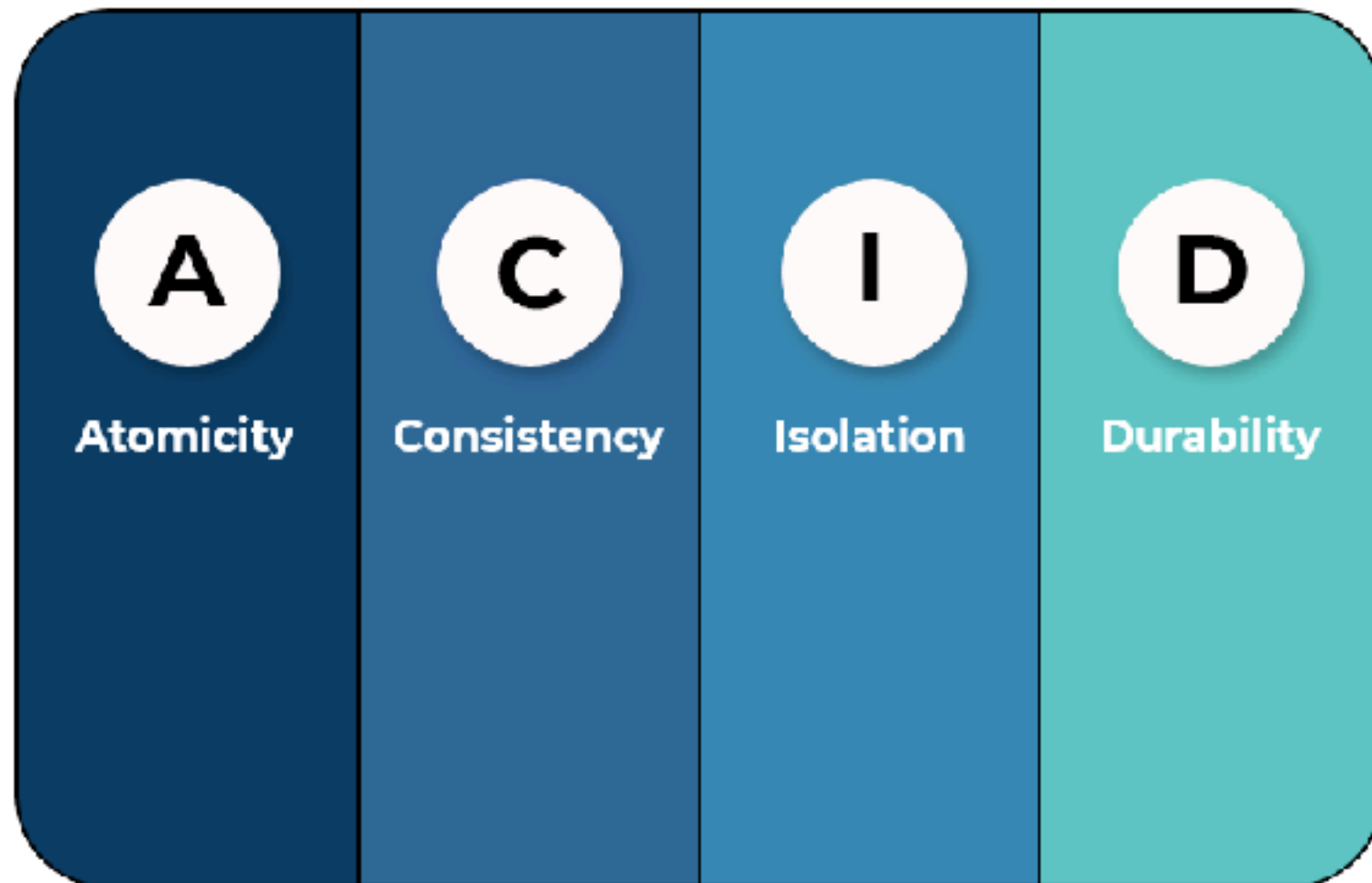
# ACID in RDBMS

<https://en.wikipedia.org/wiki/ACID>

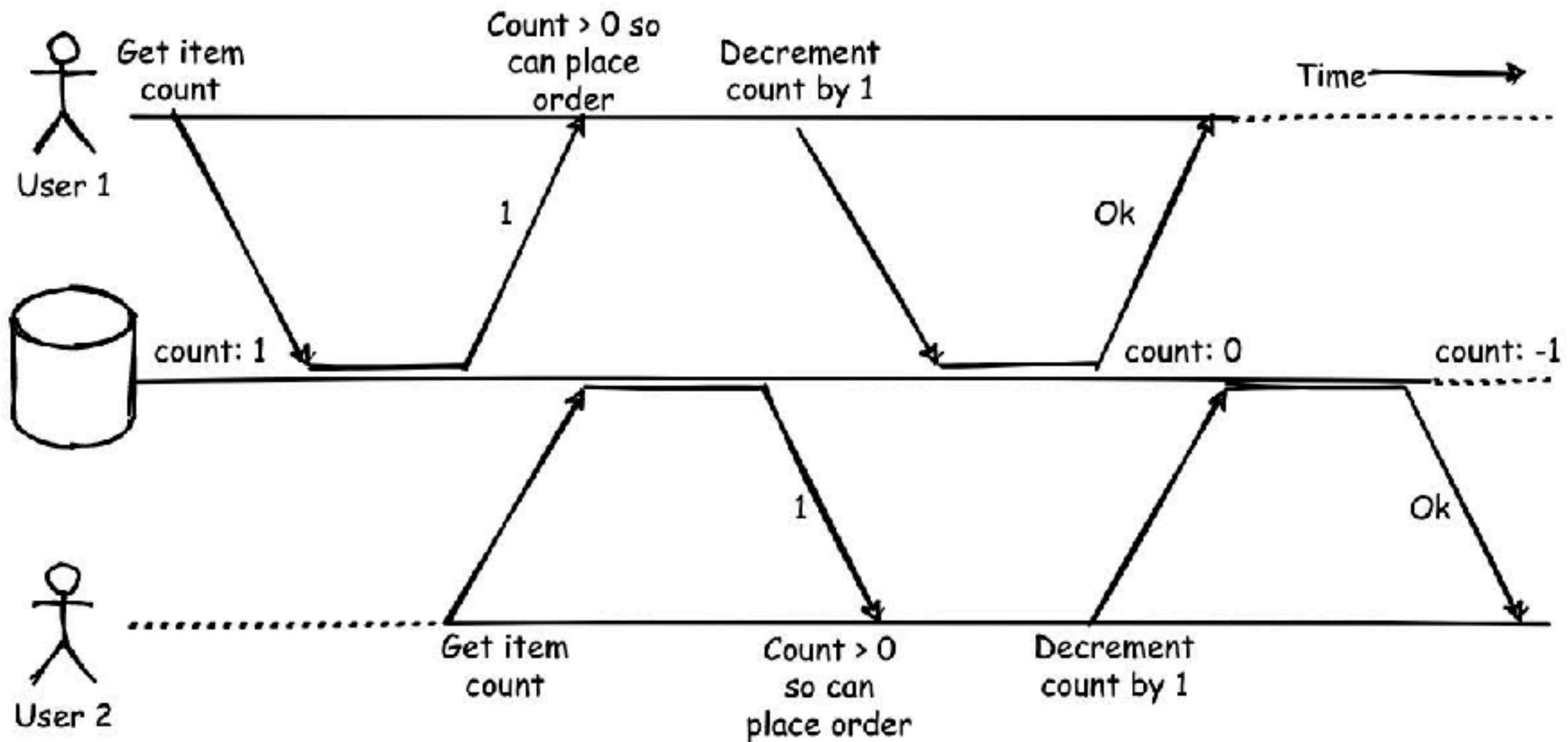


# ACID in RDBMS

Ensure database in valid state even in the event of unexpected errors



# Race condition

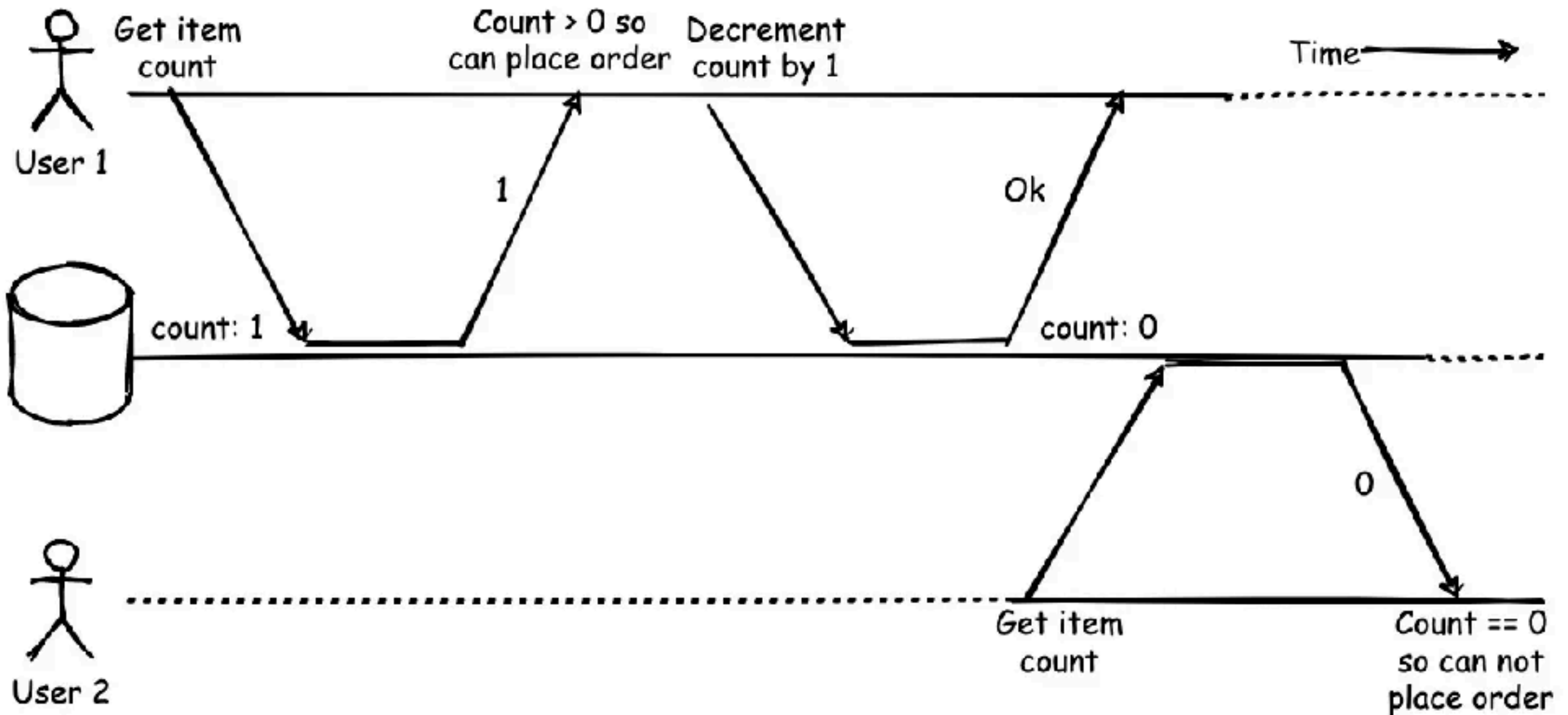


[https://en.wikipedia.org/wiki/Race\\_condition#Example](https://en.wikipedia.org/wiki/Race_condition#Example)





# Sequential

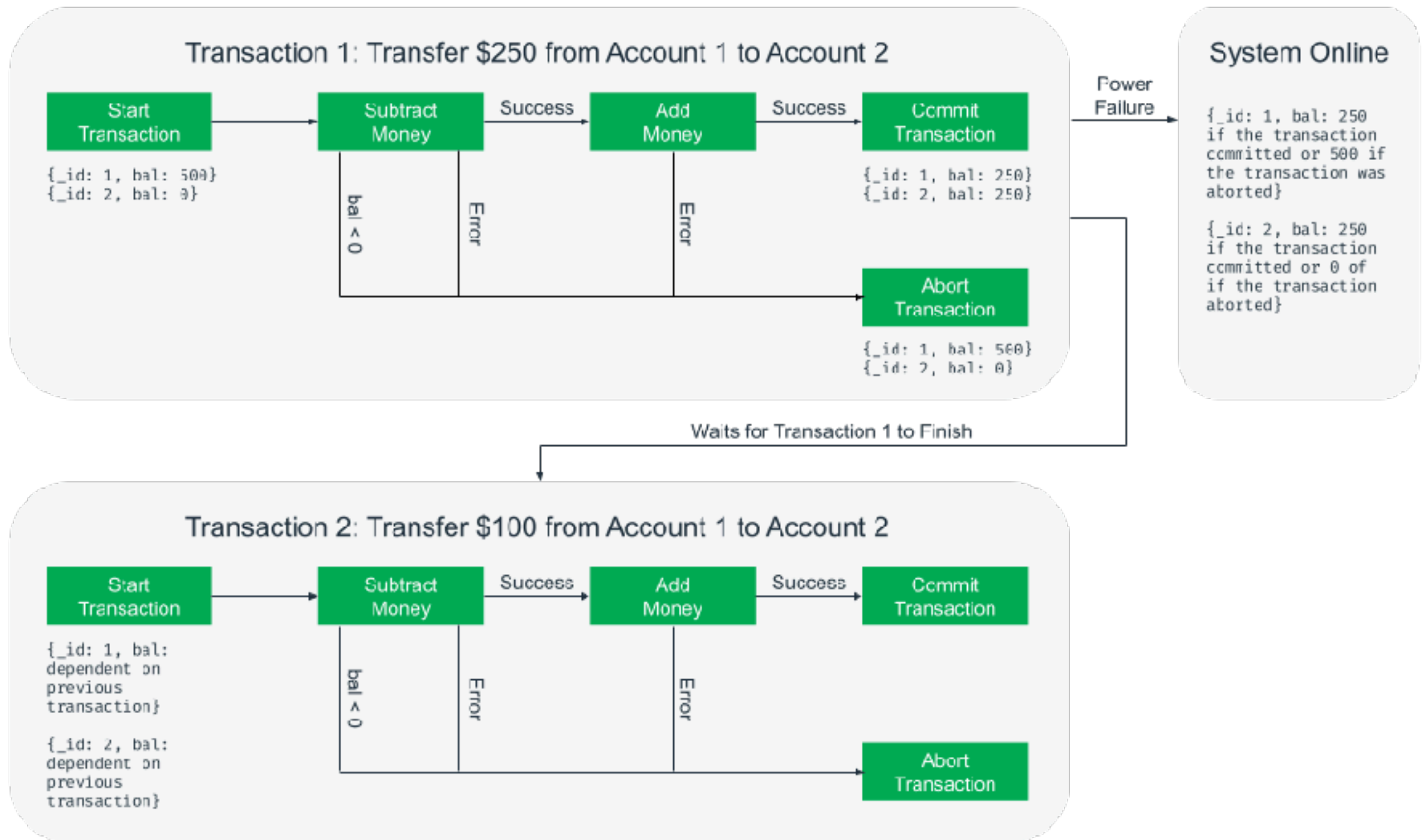


<https://www.mongodb.com/basics/acid-transactions>





# ACID in RDBMS



<https://www.mongodb.com/basics/acid-transactions>



# SQL

# Structure Query Language



# SQL

Maintain data in **tables**

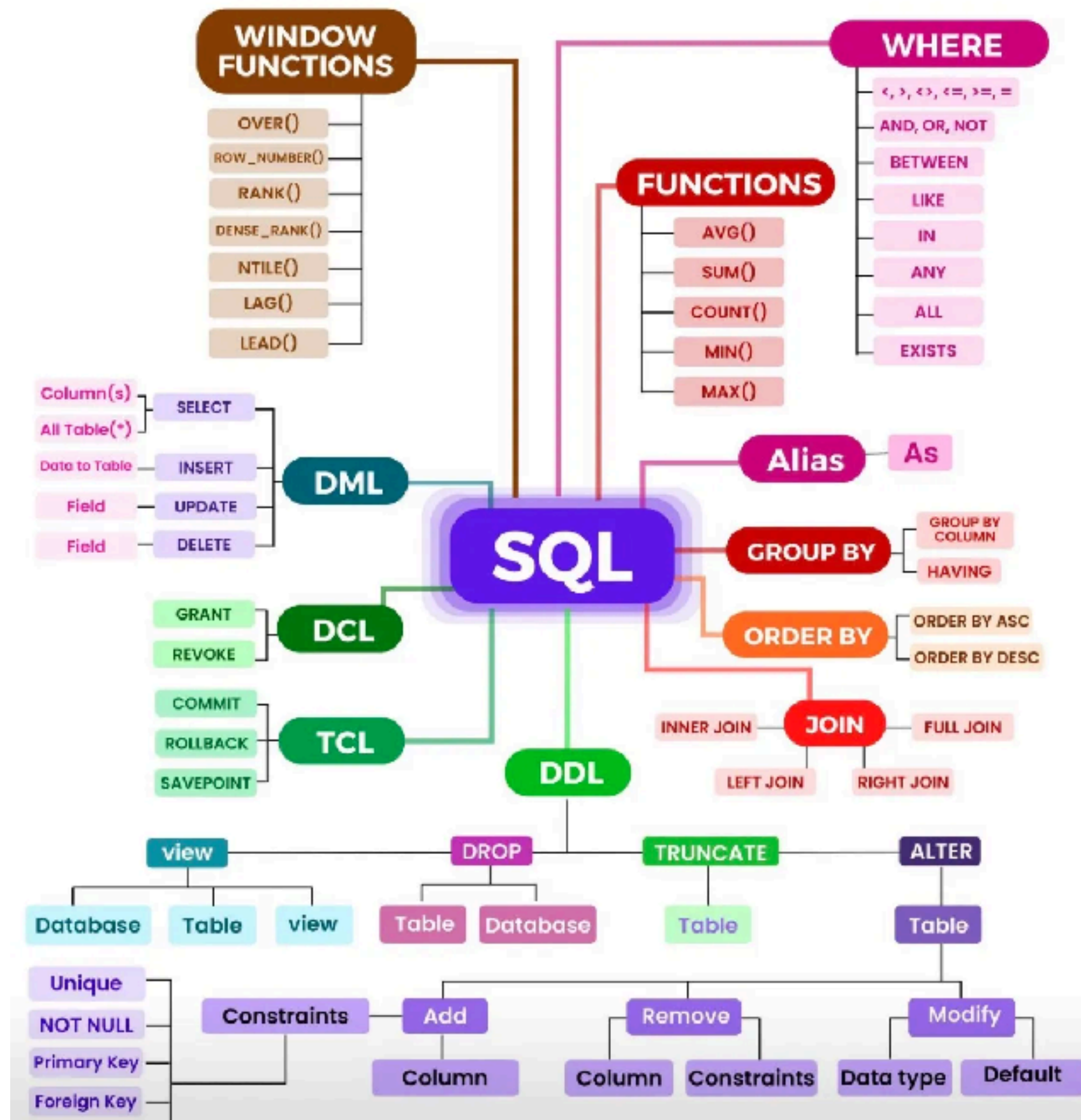
**Pre-define** structure

Each table has **Primary key**

Many **columns**

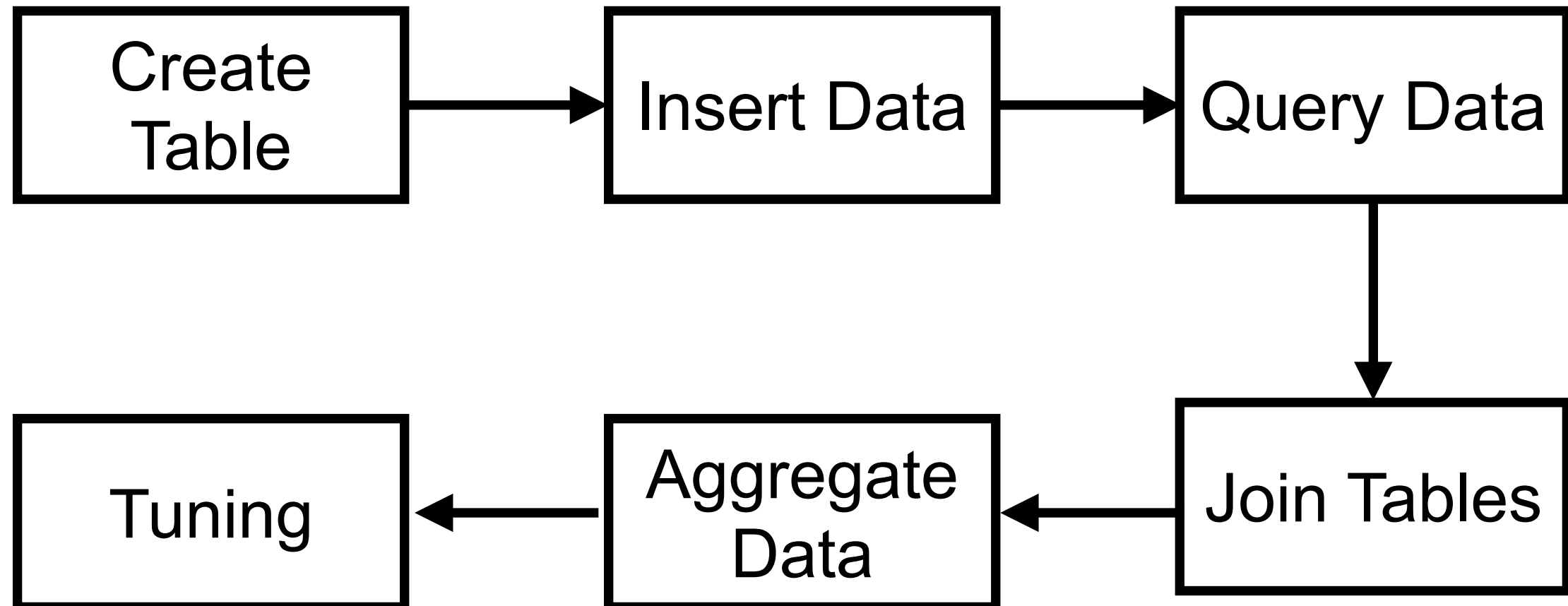
Data is represented in term of **row**





# Learning paths

## CRUD operations



# Query Data



**SELECT** columns  
**FROM** table  
**WHERE** condition A  
**AND** condition B  
**GROUP BY** columns  
**ORDER BY** columns **ASC|DESC**





# Select Data process

Table A

Table B

Table C





# Select Data process

Table A

Table B

Table C

FROM + JOIN  
(merge)

Table A, B, C



# Select Data process

Table A

Table B

Table C

FROM + JOIN  
(merge)

Table A, B, C

WHERE  
(filter)

Table A, B, C



# Select Data process

Table A

Table B

Table C

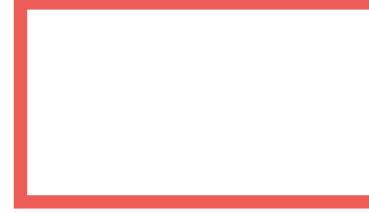
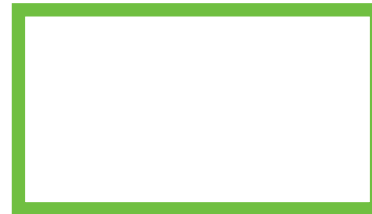
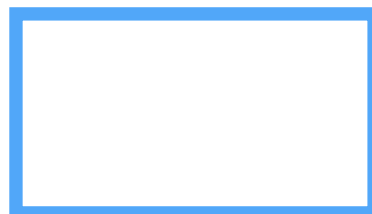
FROM + JOIN  
(merge)

Table A, B, C

WHERE  
(filter)

Table A, B, C

Group By  
(Group)



# Select Data process

Table A

Table B

Table C

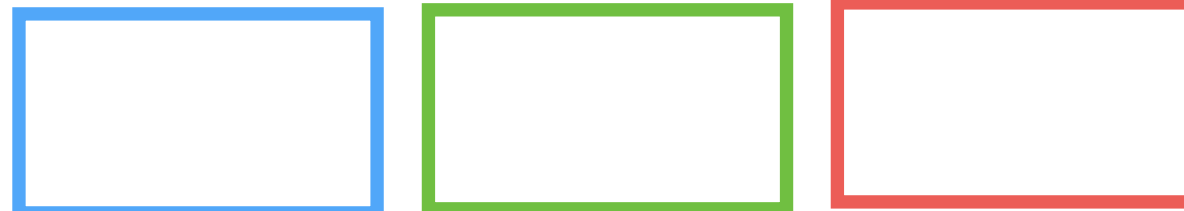
FROM + JOIN  
(merge)

Table A, B, C

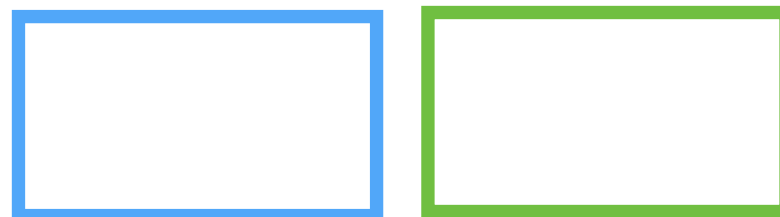
WHERE  
(filter)

Table A, B, C

Group By  
(Group)



Having  
(filter)

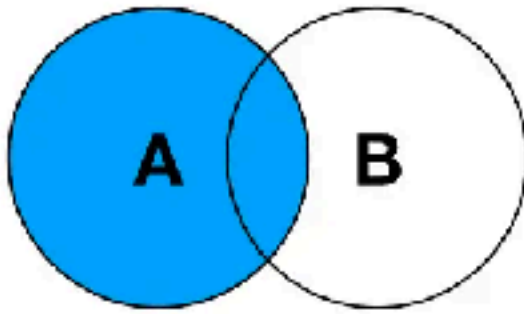


# Joining

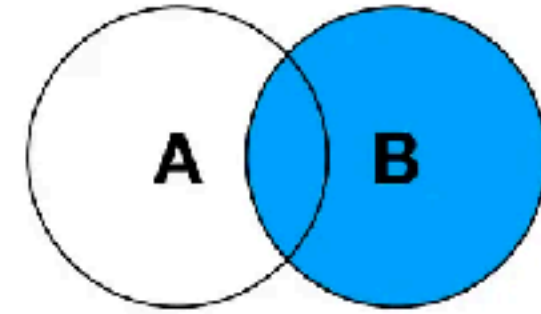
<https://www.postgresql.org/docs/current/tutorial-join.html>



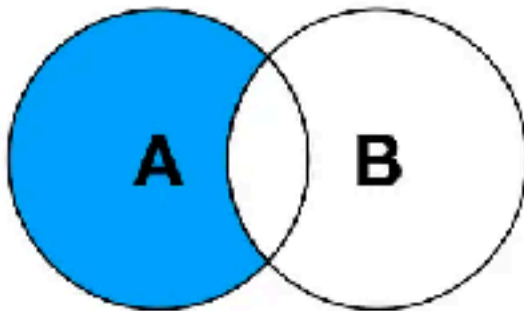
# SQL JOINS



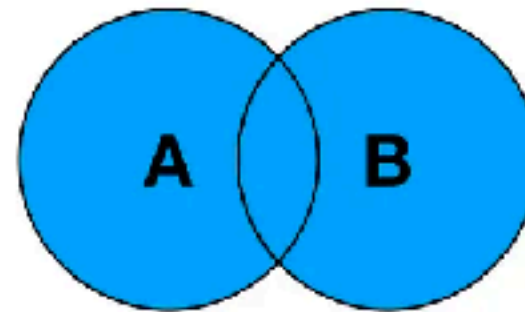
**LEFT JOIN**



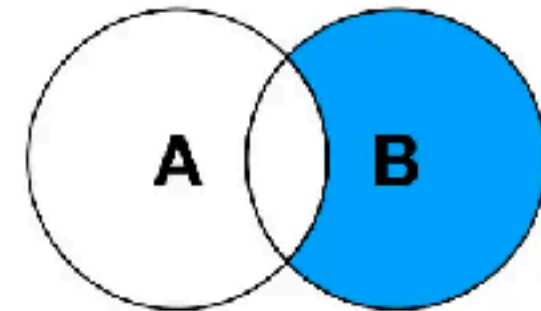
**RIGHT JOIN**



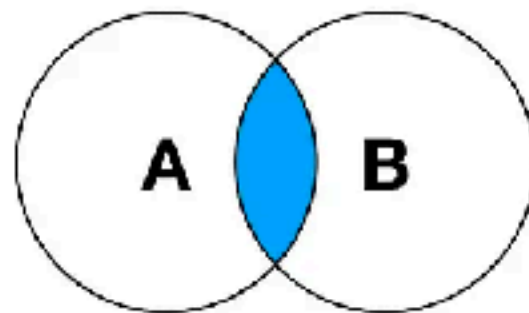
**LEFT JOIN EXCLUDING  
INNER JOIN**



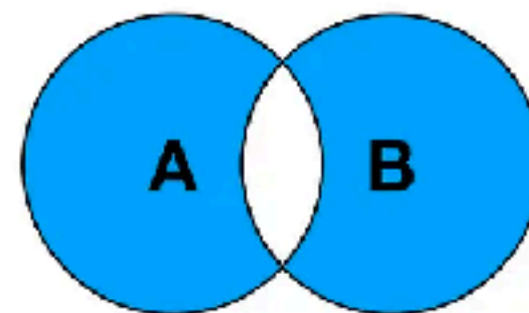
**FULL OUTER JOIN**



**RIGHT JOIN EXCLUDING  
INNER JOIN**



**INNER JOIN**



**FULL OUTER JOIN EXCLUDING  
INNER JOIN**



Self-join

# Join in PostgreSQL

Left join

Inner join

Right join

Full outer  
join





# Workshop

Basket A

Apple  
Orange  
Banana  
Cucumber

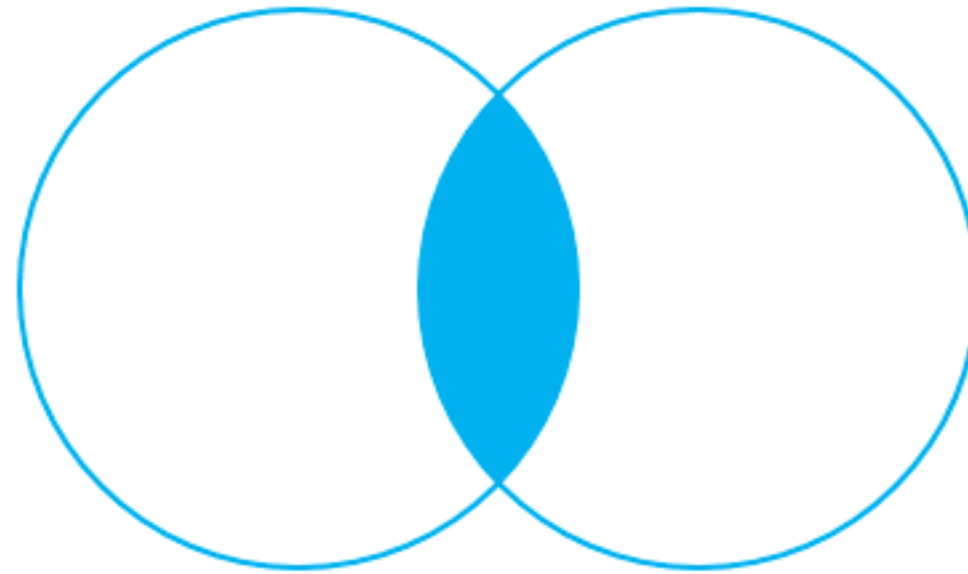
Basket B

Orange  
Apple  
Watermelon  
Pear





# Inner join



INNER JOIN



# Inner join

Basket A

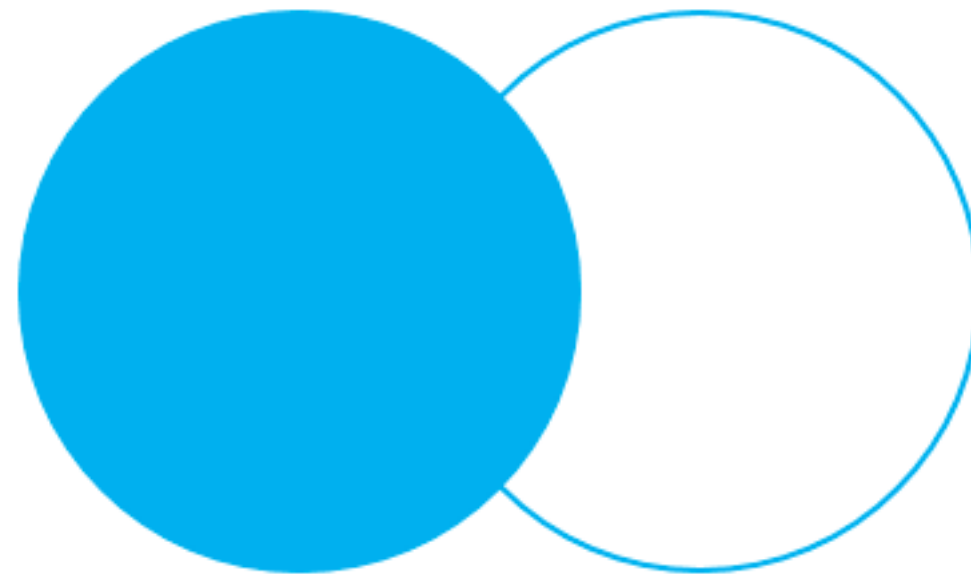
**Apple**  
**Orange**  
Banana  
Cucumber

Basket B

**Orange**  
**Apple**  
Watermelon  
Pear



# Left join (1)



LEFT OUTER JOIN



# Left join (1)

Basket A

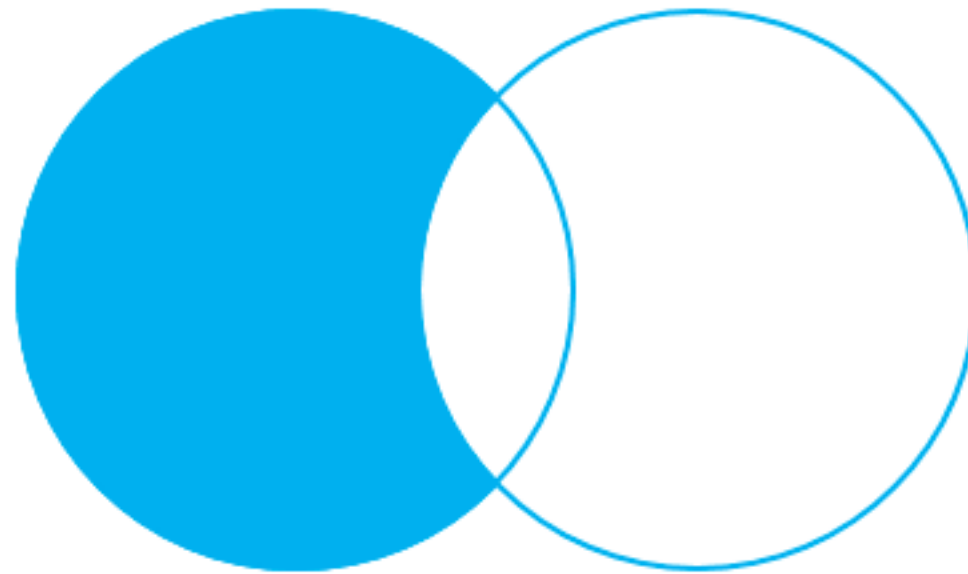
**Apple**  
**Orange**  
**Banana**  
**Cucumber**

Basket B

**Orange**  
**Apple**  
**Watermelon**  
**Pear**



# Left join (2)



LEFT OUTER JOIN – only  
rows from the left table



# Left join (2)

Basket A

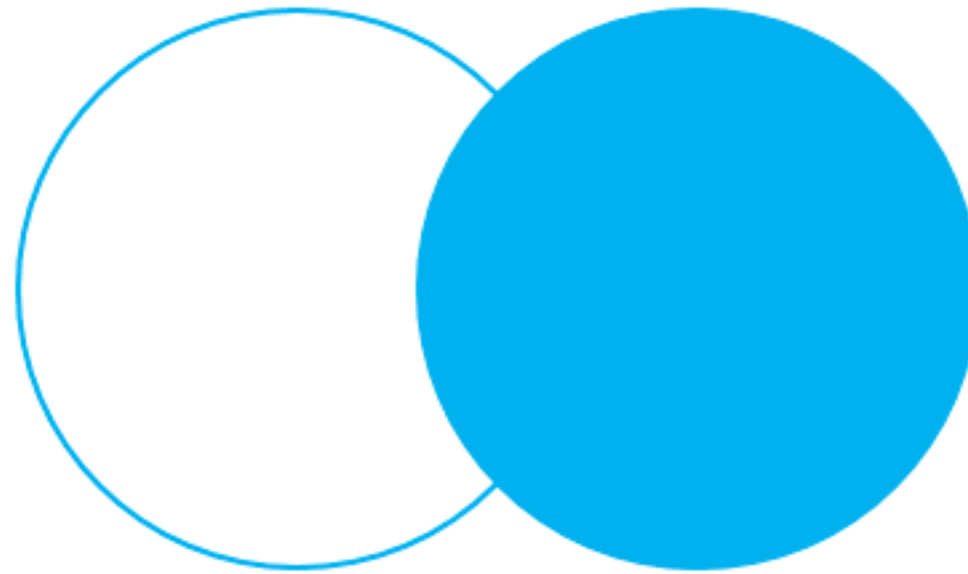
Apple  
Orange  
**Banana**  
**Cucumber**

Basket B

Orange  
Apple  
Watermelon  
Pear



# Right join (1)



RIGHT OUTER JOIN





# Right join (1)

Basket A

Apple  
Orange  
**Banana**  
**Cucumber**

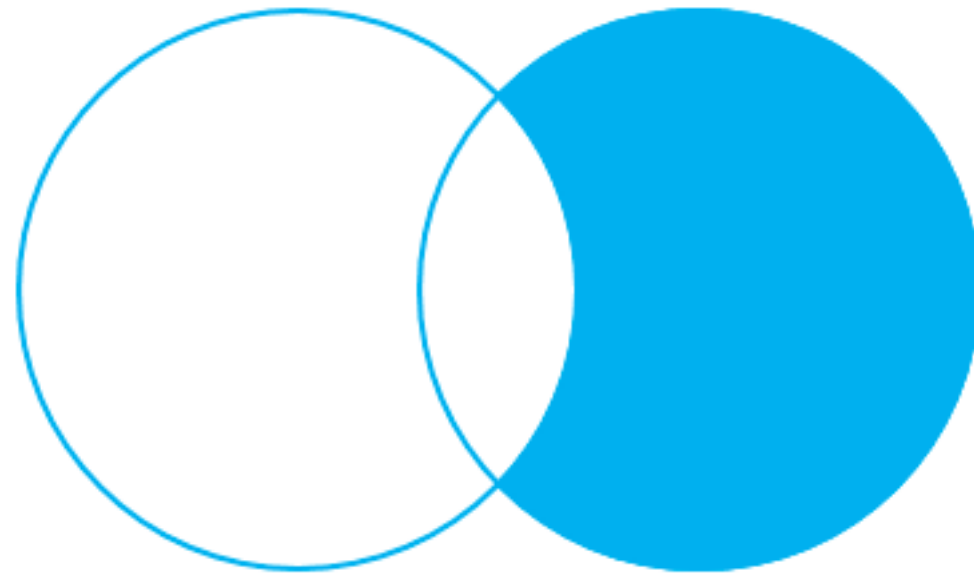
Basket B

**Orange**  
**Apple**  
**Watermelon**  
**Pear**





# Right join (2)



RIGHT OUTER JOIN – only  
rows from the right table



# Right join (2)

Basket A

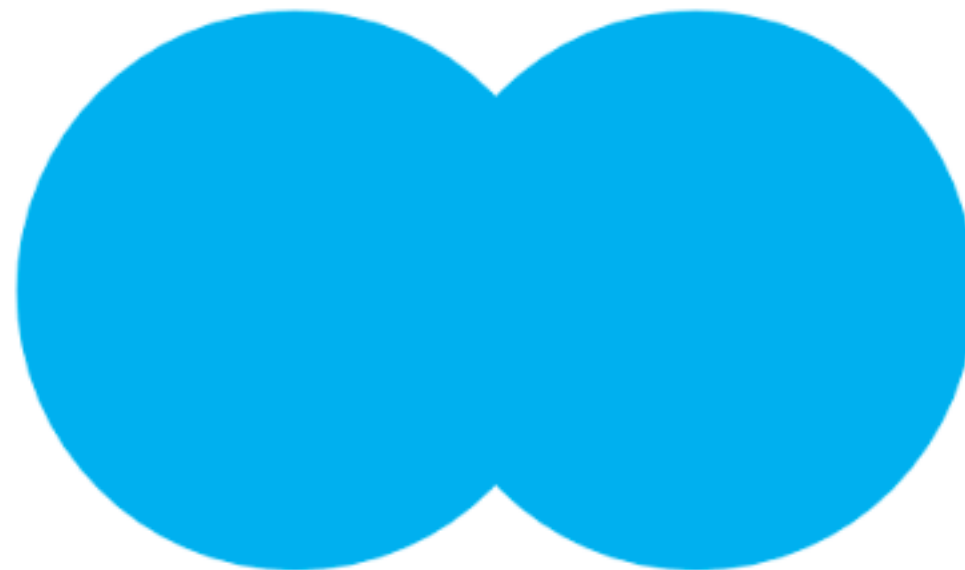
Apple  
Orange  
Banana  
Cucumber

Basket B

Orange  
Apple  
**Watermelon**  
**Pear**



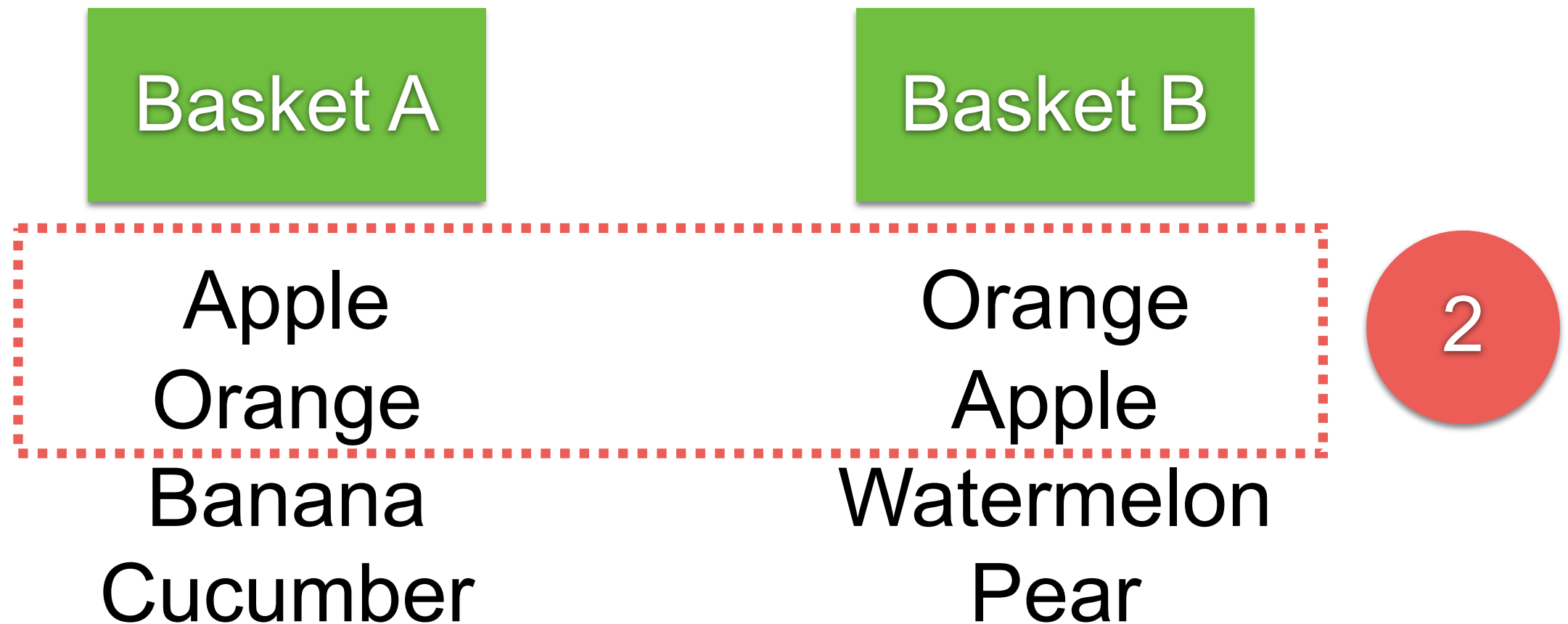
# Full outer join (1)



FULL OUTER JOIN



# Full outer join (1)



# Full outer join (1)

Basket A

Apple  
Orange  
Banana  
Cucumber

Basket B

Orange  
Apple  
Watermelon  
Pear

2



# Full outer join (1)

Basket A

Apple  
Orange  
Banana  
Cucumber

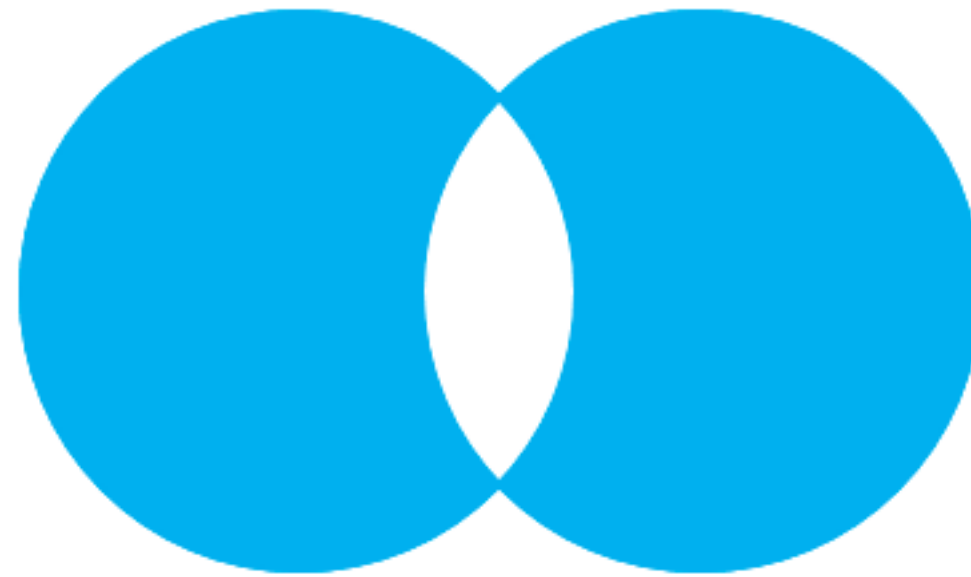
Basket B

Orange  
Apple  
Watermelon  
Pear

2



# Full outer join (2)



FULL OUTER JOIN – only  
rows unique to both tables





# Full outer join (2)

Basket A

Apple  
Orange  
Banana  
Cucumber

Basket B

Orange  
Apple  
Watermelon  
Pear

2





# Full outer join (2)

Basket A

Apple  
Orange  
Banana  
Cucumber

Basket B

Orange  
Apple  
Watermelon  
Pear

2



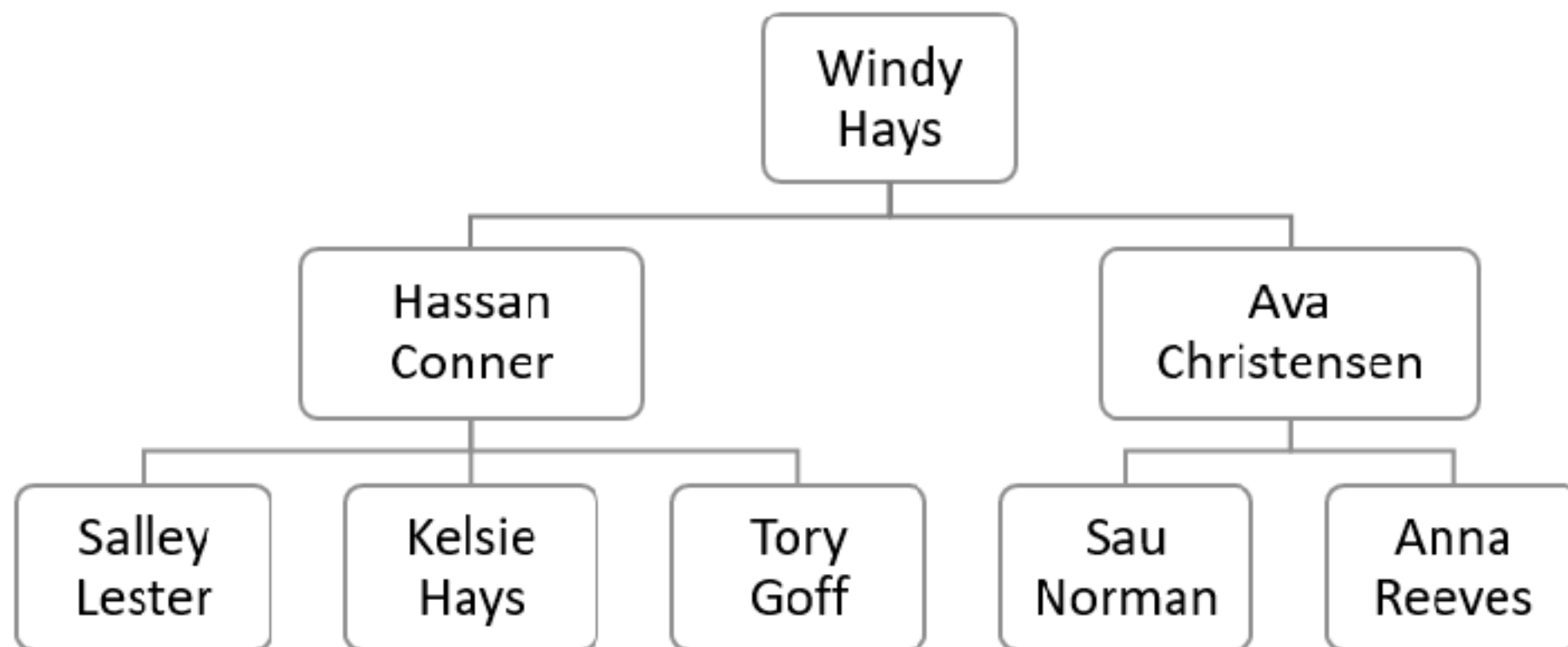
# Workshop with Joins

<https://github.com/up1/workshop-database/blob/main/workshop/joins.md>



# Self-join !!!

Regular join that joins a table to itself  
Query hierarchical data



# Workshop with Self-Join

<https://github.com/up1/workshop-database/blob/main/workshop/self-join.md>



# Problems ?



# Problems

Slow query

Accurate knowledge is required

Add complexity

Need denormalization ?

Rewrite query ?

Database vs application caching ?



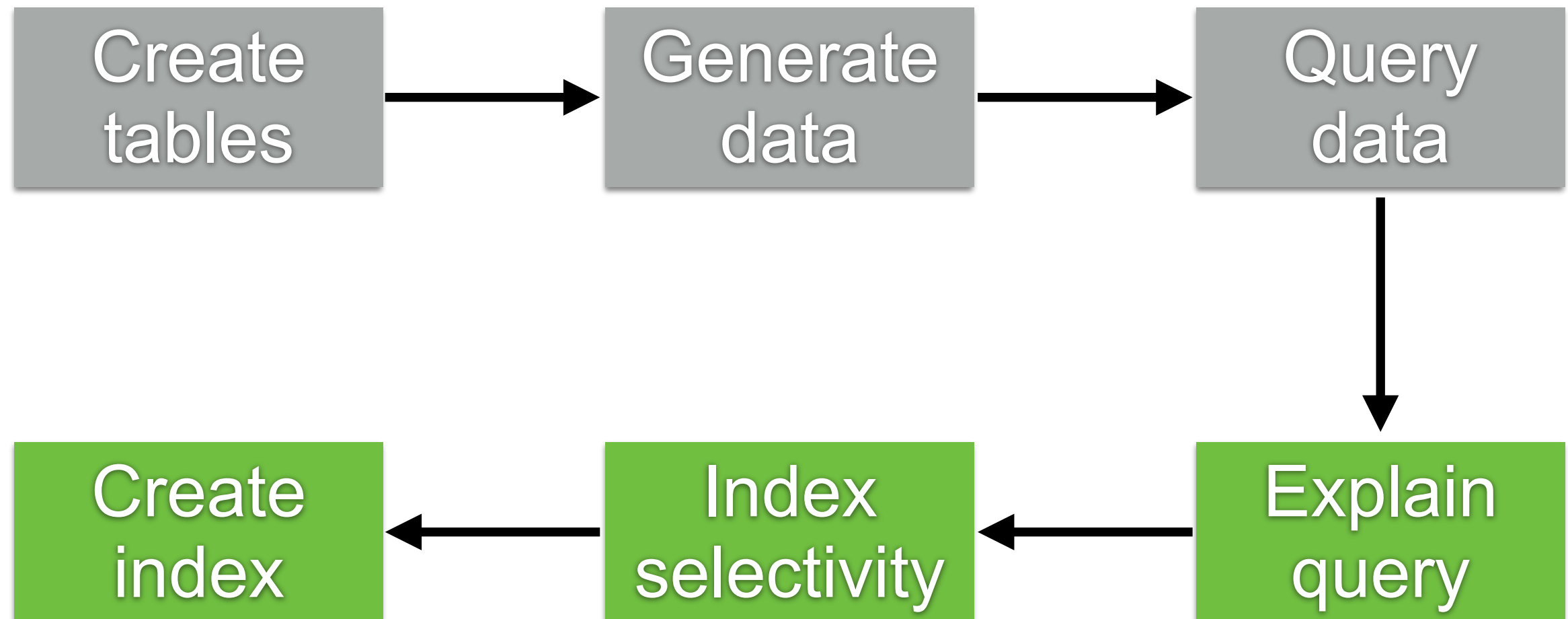
# Improvement for query data

**When ? Why ? How ? Who ?**





# Step in workshop





# Tips for SQL Optimization

| Bad                                   | Good                                     |
|---------------------------------------|------------------------------------------|
| SELECT *                              | SELECT column1, column2                  |
| SELECT ALL                            | SELECT with Limit, Offset, Top           |
| Many joins !!                         | Use appropriate joins                    |
| SELECT distinct !! For large datasets | Use group by, union , exists, row_number |
| %column%                              | Use full text search feature             |
| count(*) > 0 or IN                    | exists (select 1 from table WHERE ...)   |
| Don't use subquery                    | Use derived table or pre-joined tables   |



**Always  
Explain Analyze Query !!**



# Query plan in PostgreSQL

## EXPLAIN ANALYZE <SQL>

The screenshot shows a PostgreSQL query editor interface. At the top, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, displaying the SQL query: `1 EXPLAIN ANALYZE select * from book;`. Below the query editor, there are tabs for 'Data Output', 'Messages', 'Graph Visualiser', 'X', 'Explain', 'X', and 'Notifications'. The 'Data Output' tab is selected, showing a table with the query plan results. The table has three rows: 1. 'Seq Scan on book (cost=0.00..2334.00 rows=100000 width=89) (actual time=0.007..8.479 rows=100000 loops...)', 2. 'Planning Time: 0.209 ms', and 3. 'Execution Time: 12.950 ms'.

|   | QUERY PLAN                                                                                                 |
|---|------------------------------------------------------------------------------------------------------------|
| 1 | Seq Scan on book (cost=0.00..2334.00 rows=100000 width=89) (actual time=0.007..8.479 rows=100000 loops...) |
| 2 | Planning Time: 0.209 ms                                                                                    |
| 3 | Execution Time: 12.950 ms                                                                                  |

<https://github.com/up1/workshop-database/tree/main/workshop#query-plan>



# Indexing Strategies

<https://www.postgresql.org/docs/current/indexes-types.html>



# Index

## Symbols

2PC, 219, 346, 347, 349–351

## A

A\* pathfinding algorithms, 64, 84

ACID, 197, 208, 217, 219, 321

ActiveMQ, 92

adjacency lists, 77

Advanced Message Queuing Protocol,  
125

Aeron, 402

aggregation window, 164, 176

Airbnb, 193, 199, 337

Amazon, 137, 199, 317

Amazon API Gateway, 303

Amazon Web Services, 251, 302

AML/CFT, 318

AMM, 409

AMQP, 125

Apache James, 231

append-only, 362, 365

Apple, 393

Apple Pay, 315

application loop, 398

ask price, 380

asynchronous, 328

At-least once, 122

at-least once, 93, 122

at-least-once, 331

at-most once, 93, 122

at-most-once, 331

atomic commit, 167

atomic operation, 218

audit, 360

Automatic Market Making, 409

Availability Zone, 253

Availability Zones, 268

availability zones, 27

AVRO, 165

AWS, 251, 302, 303

AWS Lambda, 303

AZ, 268

## B

B+ tree, 267

Backblaze, 272

base32, 11

BEAM, 55

bid price, 380

Bigtable, 137, 235, 243

Blue/green deployment, 18

brokers, 95, 96, 98, 102, 105–107, 113,  
118, 120, 122

buy order, 393

## C

California Consumer Privacy Act,  
2

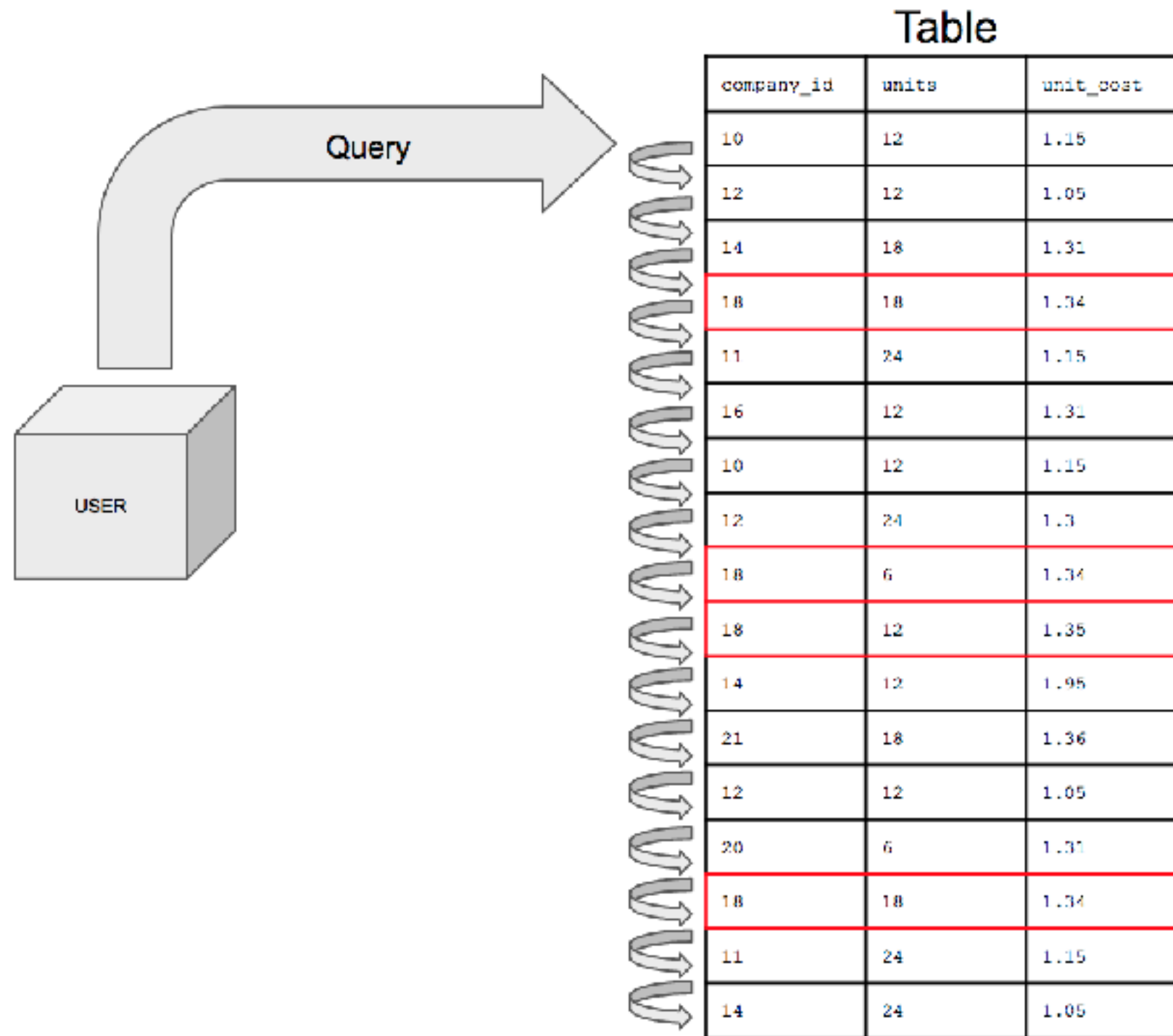
candlestick chart, 381

candlestick charts, 384, 387, 396,  
407

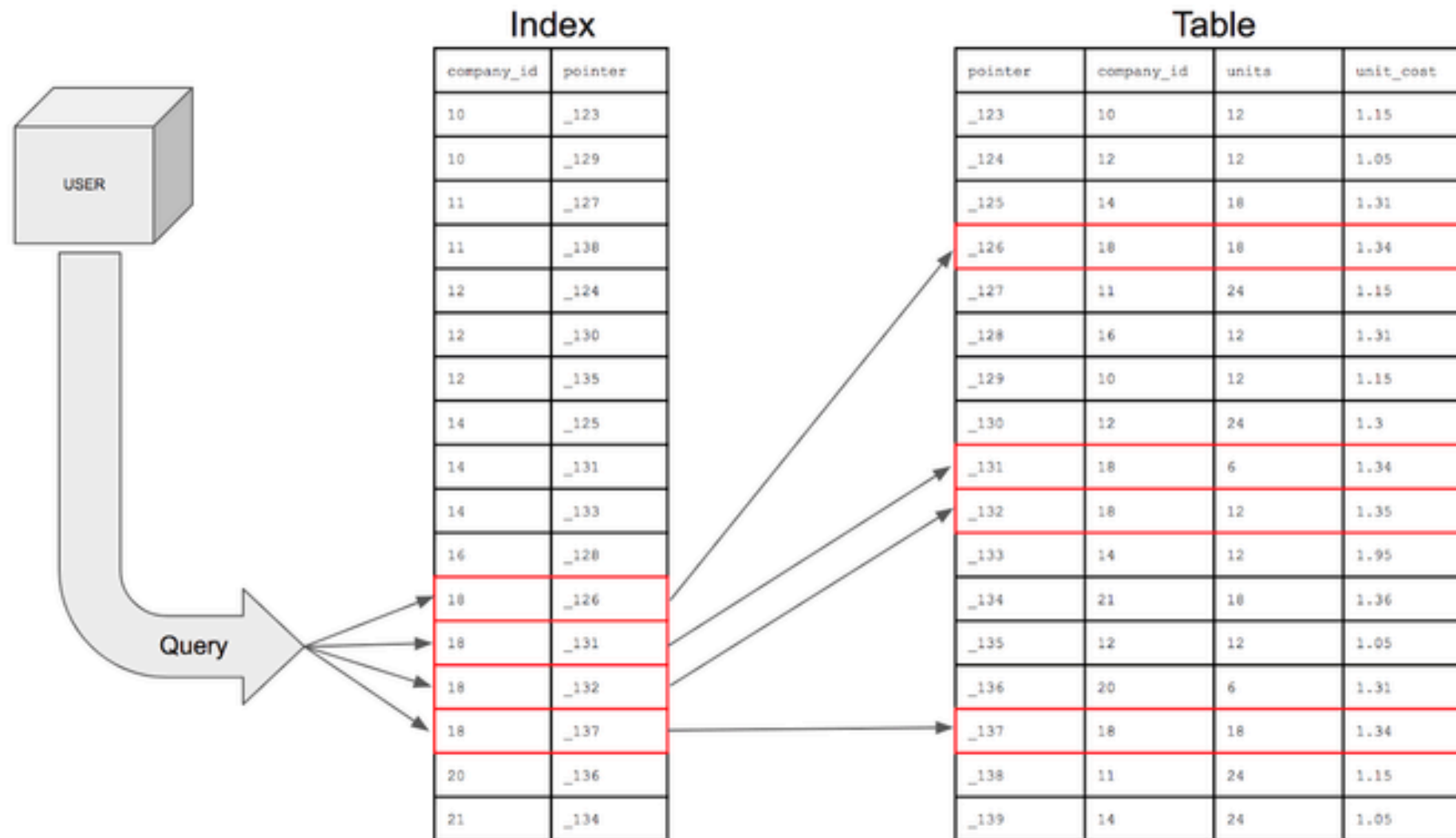
CAP theorem, 79



# Without Index (full table scan)



# With Index





# With Index

Table

|         | Column A     | Column B             |
|---------|--------------|----------------------|
| RowId 1 | Some value 1 | Some another value 1 |
| RowId 2 | Some value 2 | Some another value 2 |
| RowId 3 | Some value 3 | Some another value 3 |
| RowId 4 | Some value 1 | Some another value 4 |
| RowId 5 | Some value 5 | Some another value 5 |
| RowId 6 | Some value 6 | Some another value 6 |
| RowId 7 | Some value 1 | Some another value 7 |
| RowId 8 | Some value 8 | Some another value 8 |

Index

| Some value 1 | "RowId 1" "RowId 4" "RowId 7" |
|--------------|-------------------------------|
| Some value 2 | RowId 2                       |
| Some value 3 | RowId 3                       |
| Some value 5 | RowId 5                       |
| Some value 6 | RowId 6                       |
| Some value 8 | RowId 8                       |
|              |                               |
|              |                               |

Index by column "Column A"





# Main factors for good index ?

How much space does the index use ?

How fast does it read ?

How much overhead does it add to writes ?

How well does it handle concurrency ?

Quick lookup, fast insert, reliable delete



# Index selectivity

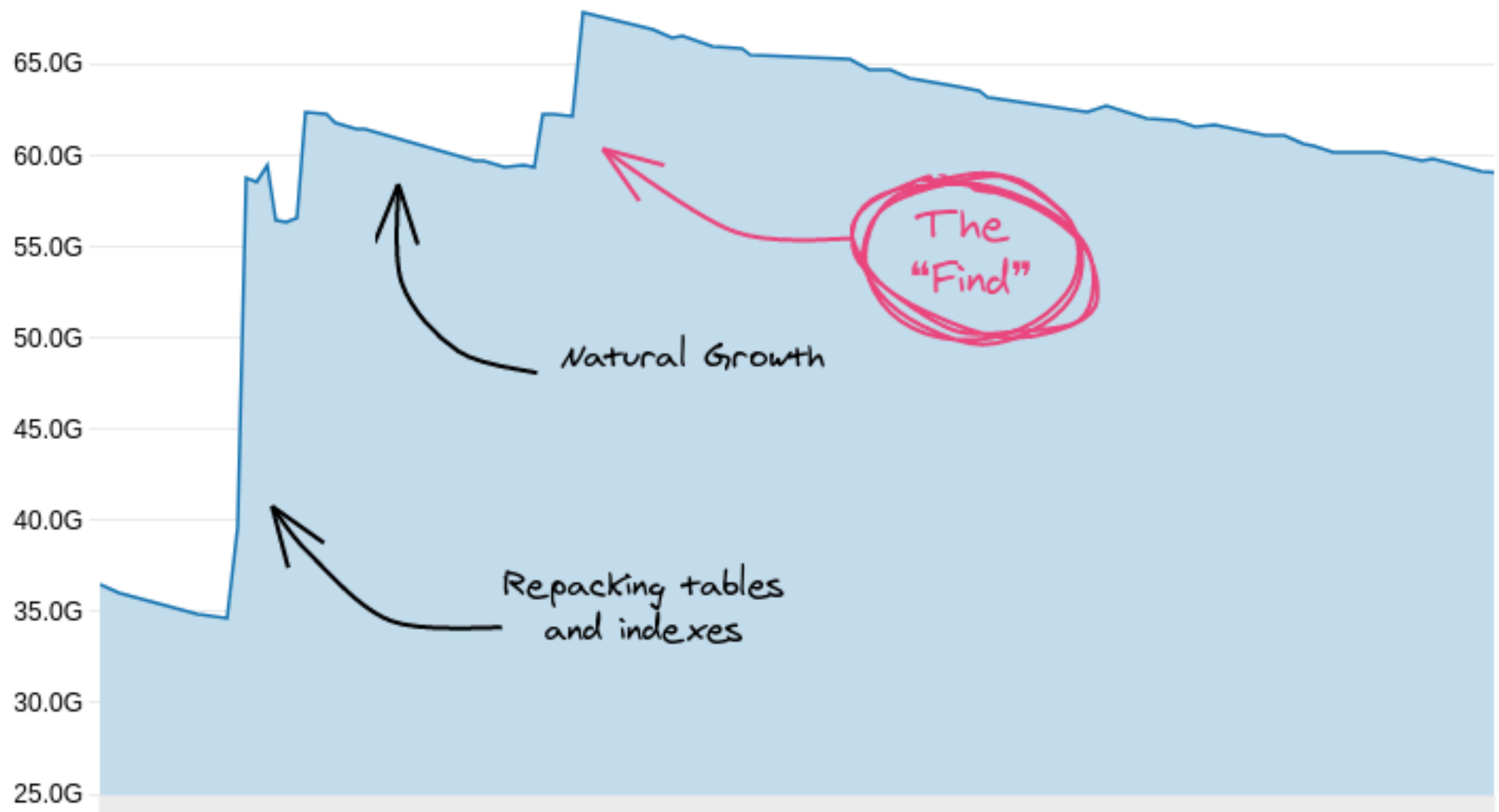
Factor to select a column to create an index

The number of **distinct values** in the indexed column divided by the number of records in the table is called a selectivity of an index

***Prefer indexing columns with selectivity greater than  $> 0.85$  (use Index scan)***



# Unused Index ?



<https://hakibenita.com/postgresql-unused-index-size>



# Types Index in PostgreSQL

B-Tree

Hash

Block range indexes (BRIN)

Generalized inverted index (GIN)

Generalized inverted search tree index (GiST)

Space partitioned GiST (SP-GiST)

<https://www.postgresql.org/docs/current/indexes-types.html>

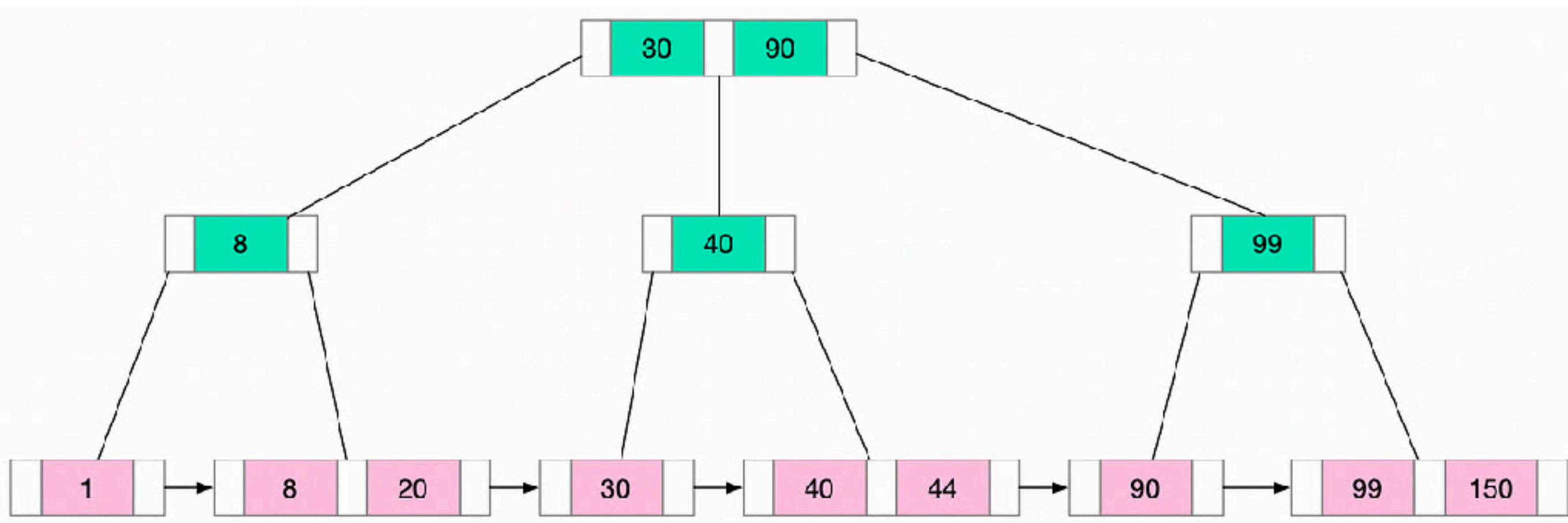


# PostgreSQL Index Types

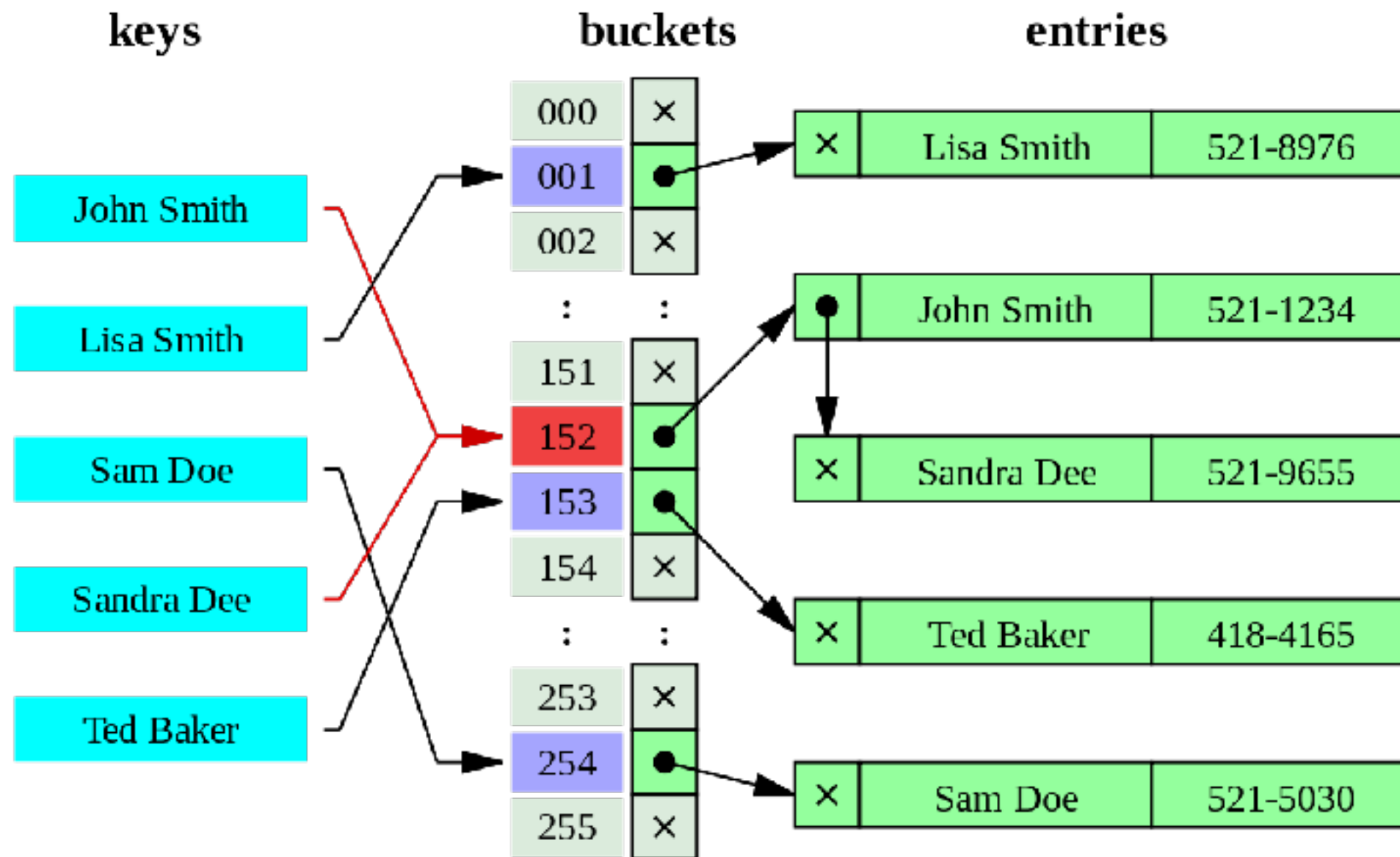
| Index type | Data                                       | Target                                           | Use case                                                 |
|------------|--------------------------------------------|--------------------------------------------------|----------------------------------------------------------|
| B-Tree     | High-cardinality scalar columns            | Fast equality, range, between operation          | PK, FK, timestamp/range scan, sorting                    |
| GIN        | Composite values (arrays, JSONB, tsvector) | Efficient contains, full-text, array membership  | Full text search, JSONB containment, array lookup        |
| GiST       | Complex or non-scalar types                | Flexible tree, Range and text                    | PostGIS spatial queries, trigram similarity, range types |
| BRIN       | Very large data, ordered table             | Lightweight index for append-only and range scan | Time-series data<br>Log data                             |
| Bloom      | Many low-cardinality columns               | Compact multi-column equality filters            | Filter data, check data existing                         |



# B-Tree +

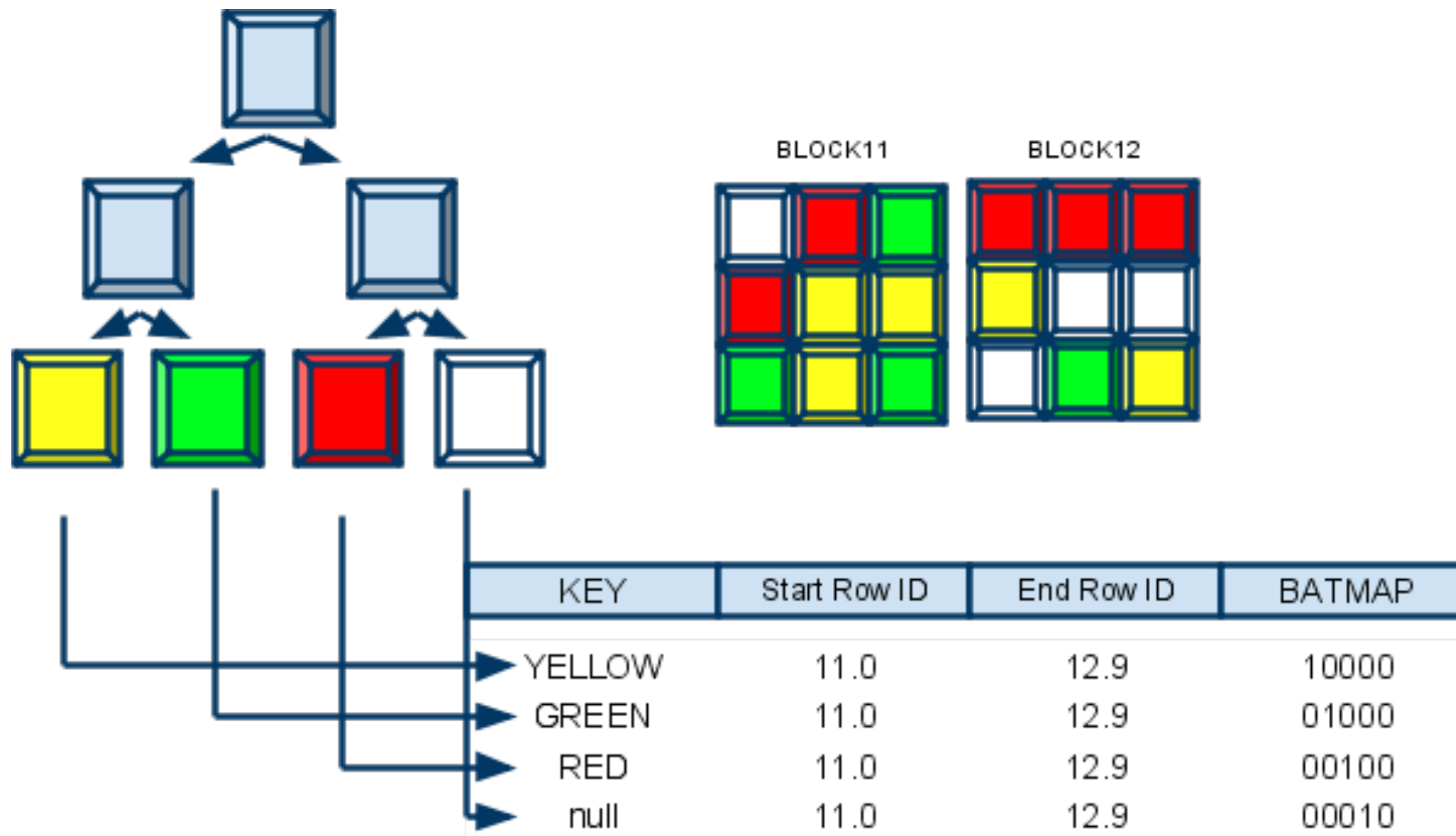


# Hash





# Bitmap

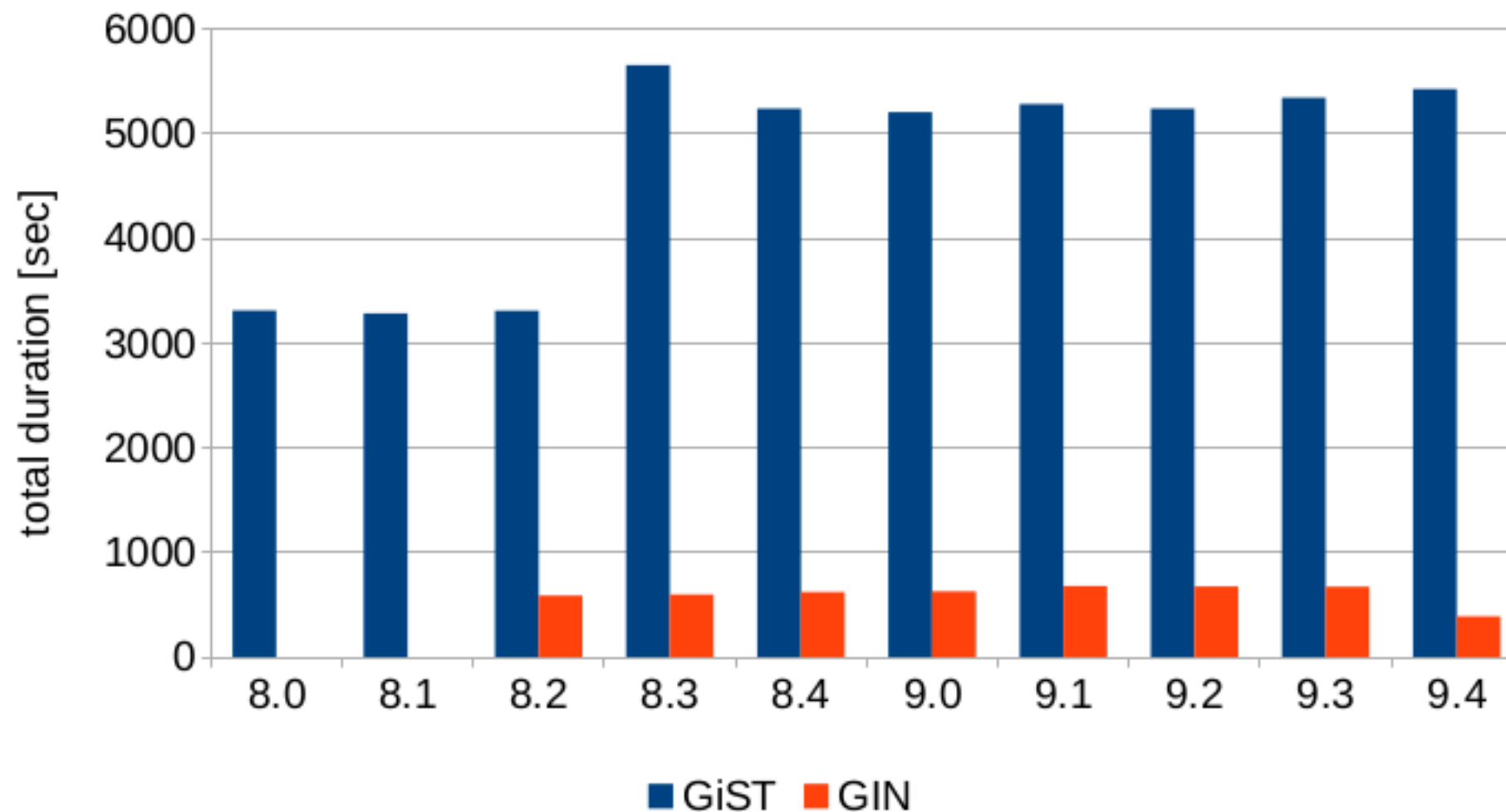




# GIN vs GiST

## Fulltext benchmark - GiST vs. GIN

33k queries from postgresql.org [TOP 100]



# Tuning and Use Index ?

Size vs Speed

Extension setup

Combine indexes

Measure impact with **EXPLAIN** (before and after)



# Don't over-index Unused index

Size of index ?  
Usages ?



# Workshop with Indexing

<https://github.com/up1/workshop-database/tree/main/workshop/basic-workshop>



# Tuning Performance



# Scalability metrics

Throughput (transactions per second)

Latency (response time)

Availability (uptime)

Durability (data safety)



# Tuning performance

Normalization vs Denormalization

**Partition**

**Sharding**

Database architecture

Database parameteres





# Partition

<https://hakibenita.com/postgresql-unused-index-size>





# Partition

Table partitioning refers to **splitting** what is logically one large table into **smaller physical pieces**

Reduce index size



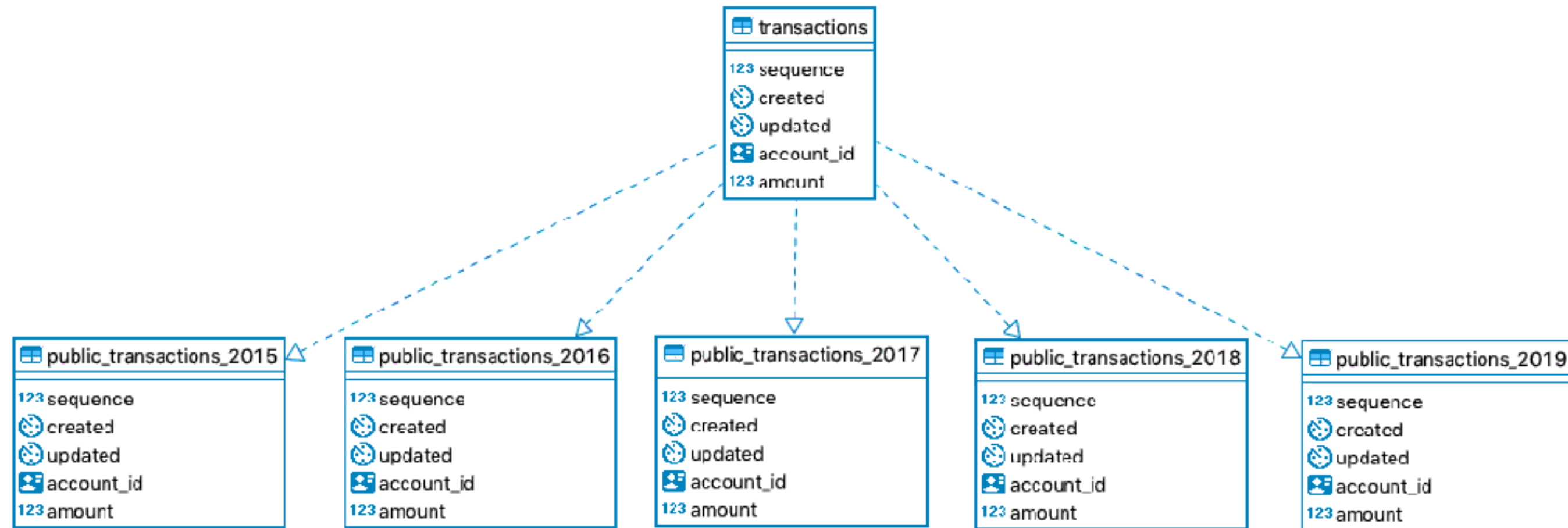
# Partition

Horizontal partitioning  
Vertical partitioning



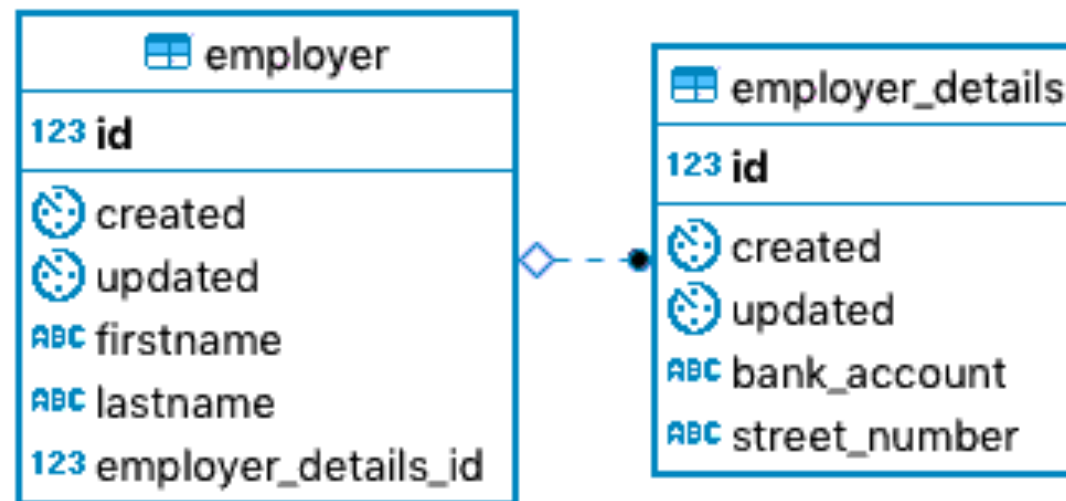
# Horizontal partitioning

Putting different rows into different table



# Vertical partitioning

## Horizontal partitioning

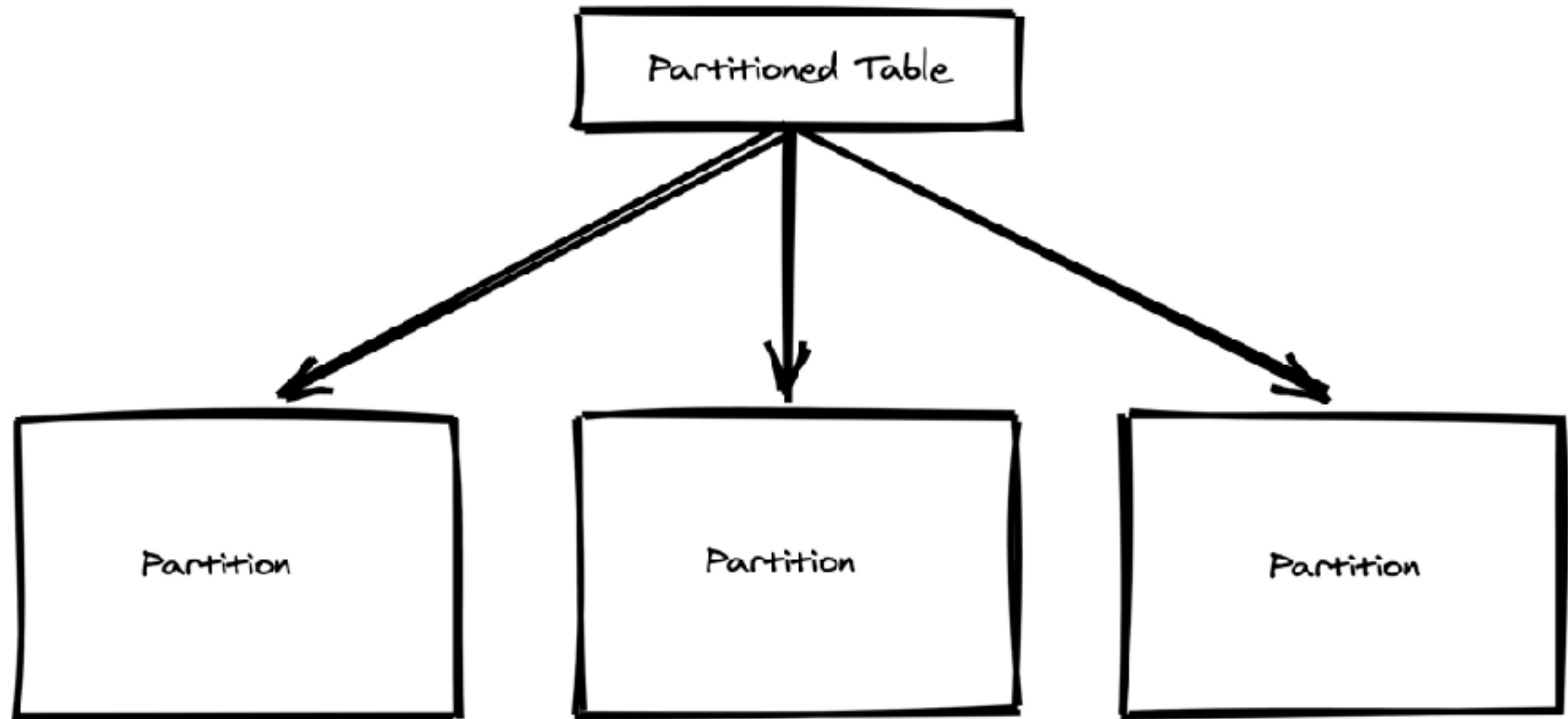


# Types Horizontal partitioning

Range  
List



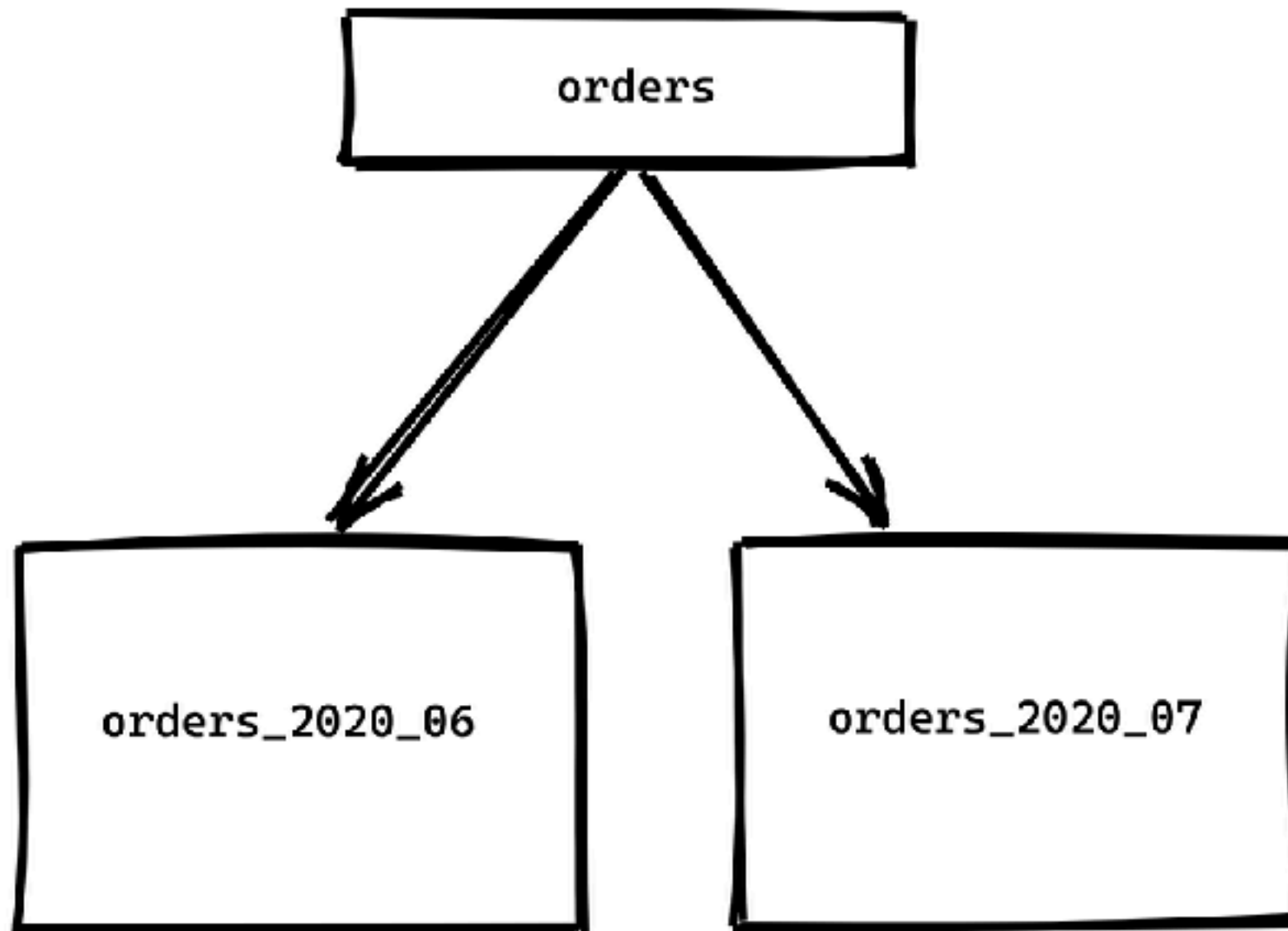
# Partition tables



<https://chrisherwin.com/table-partitioning/>



# Orders table

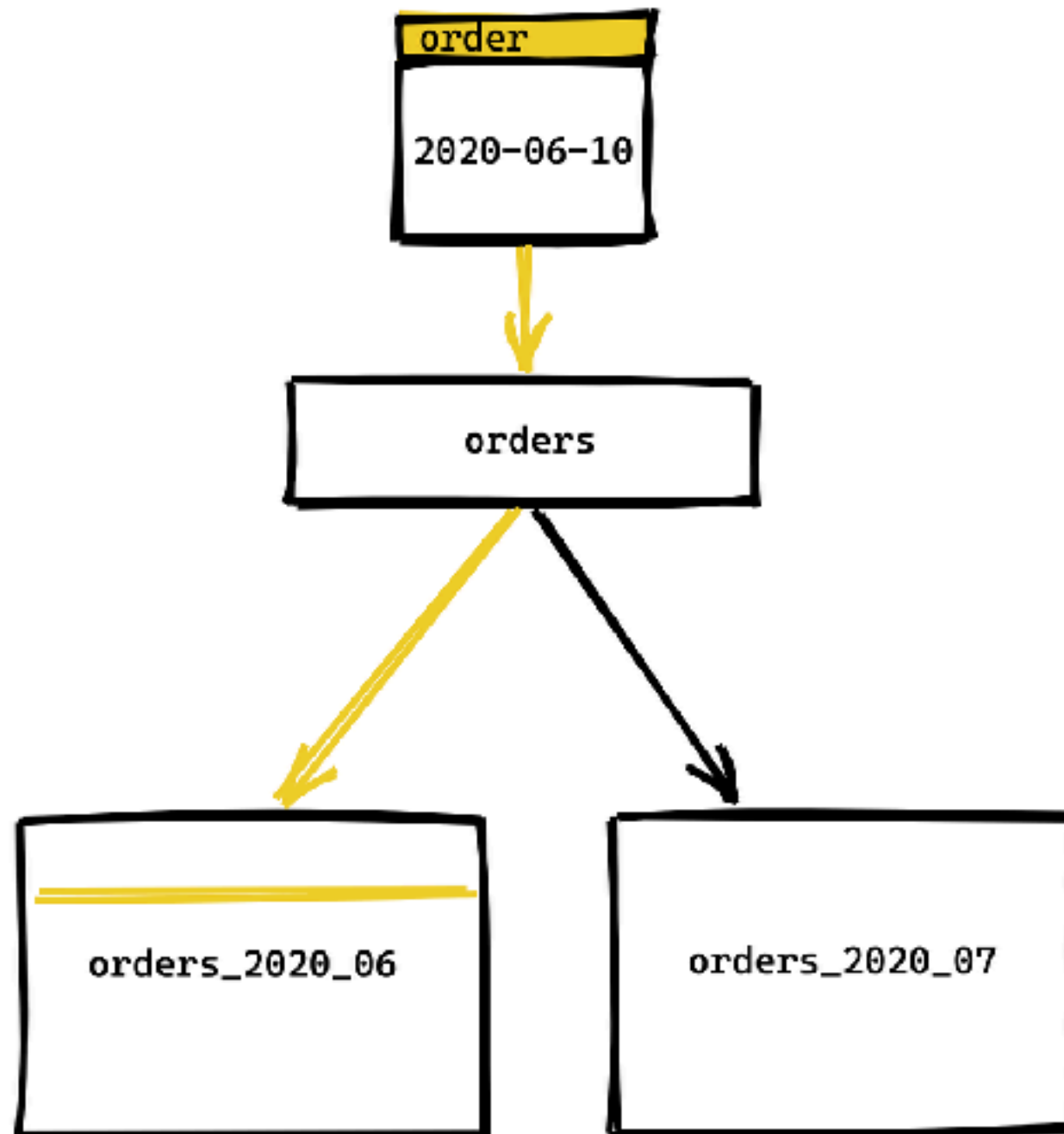


<https://chriserwin.com/table-partitioning/>





# Insert data into orders

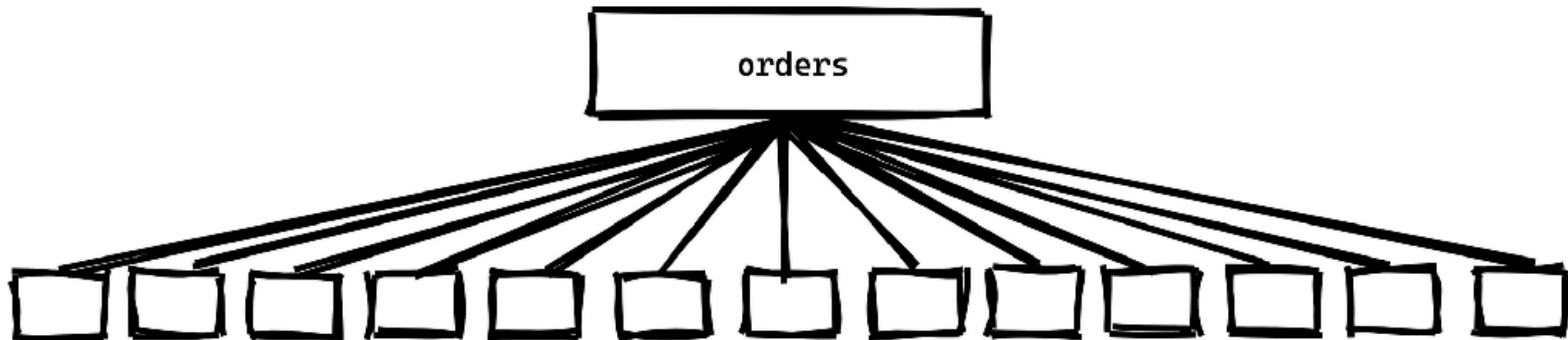


<https://chriserwin.com/table-partitioning/>





...



<https://chriserwin.com/table-partitioning/>



# Workshop with Partition

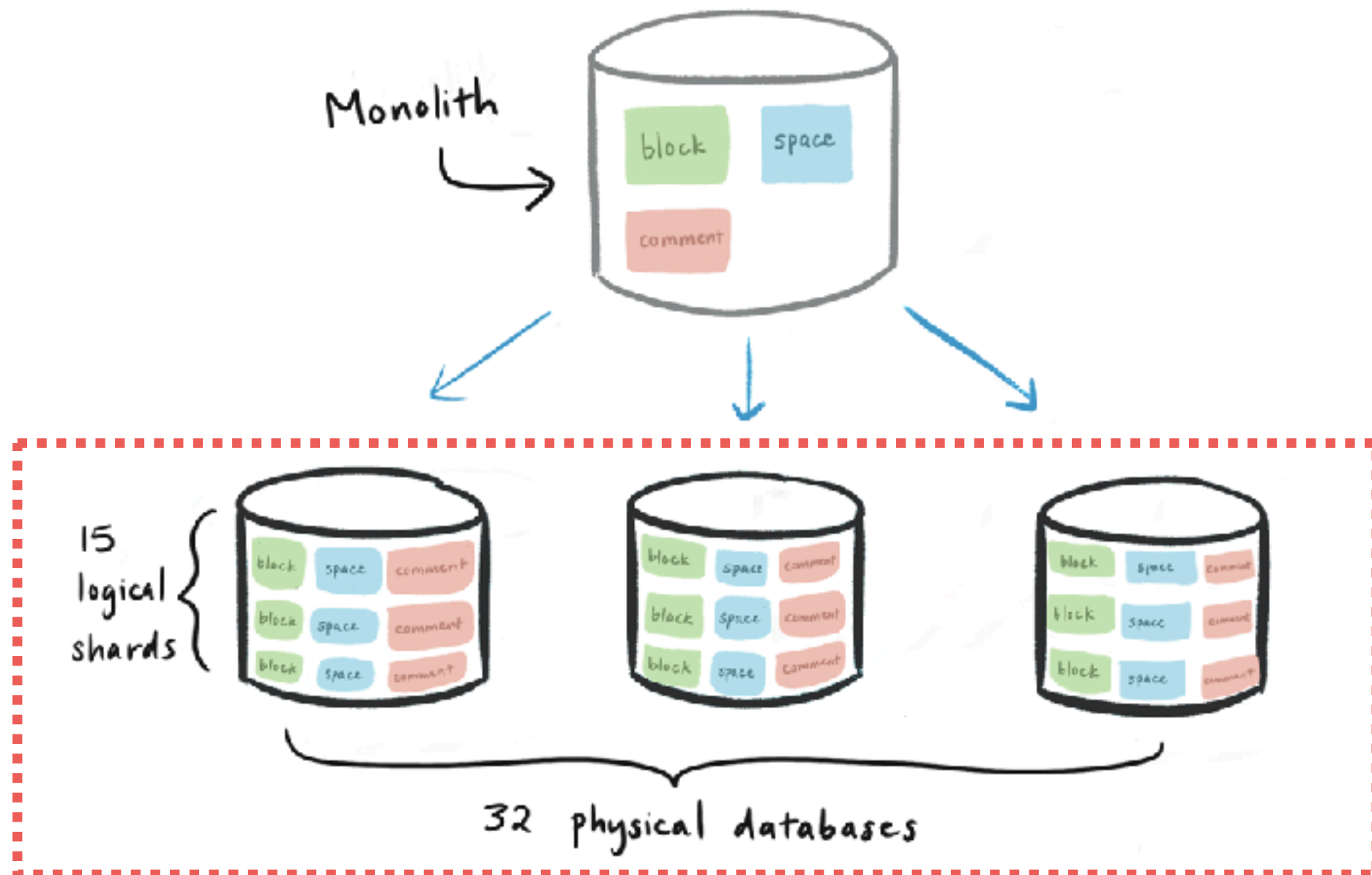
<https://github.com/up1/workshop-database/blob/main/workshop/basic-workshop/partition.md>



# Sharding



# Sharding (Physical server)



<https://www.notion.so/blog/sharding-postgres-at-notion>



# Sharding techniques

Hash

Range

Location

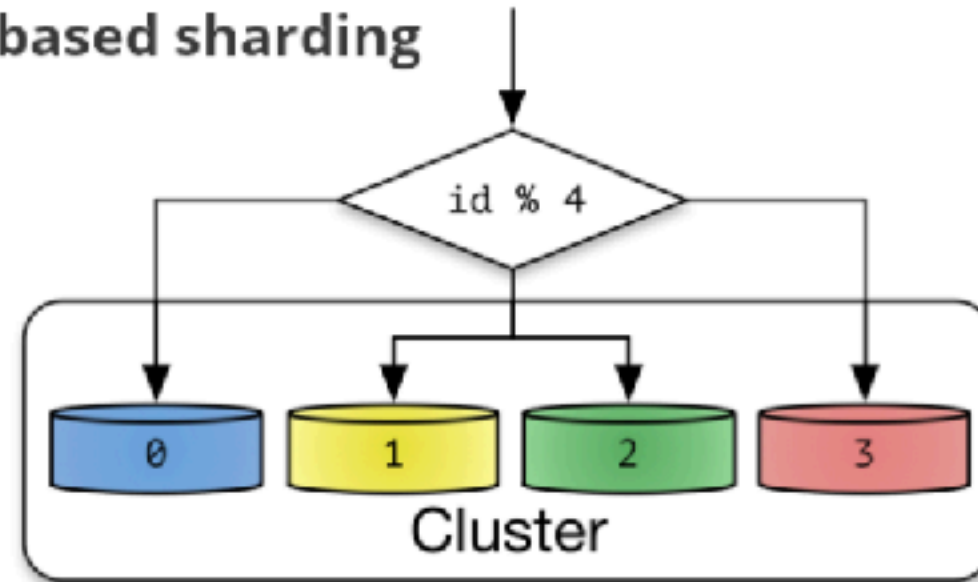
Key/Directory

<https://www.somkiat.cc/introduction-database-sharding/>

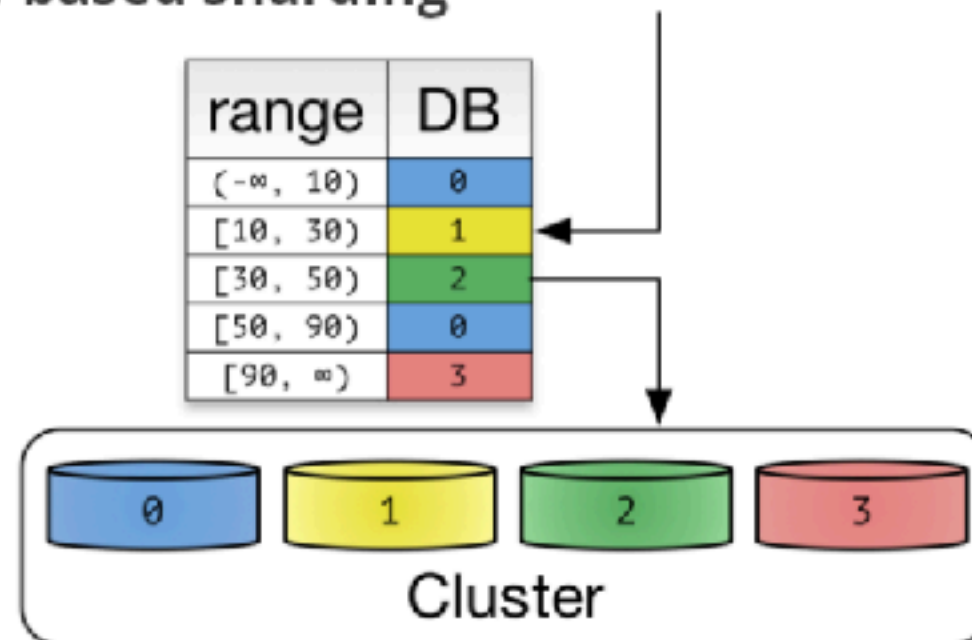


# Sharding

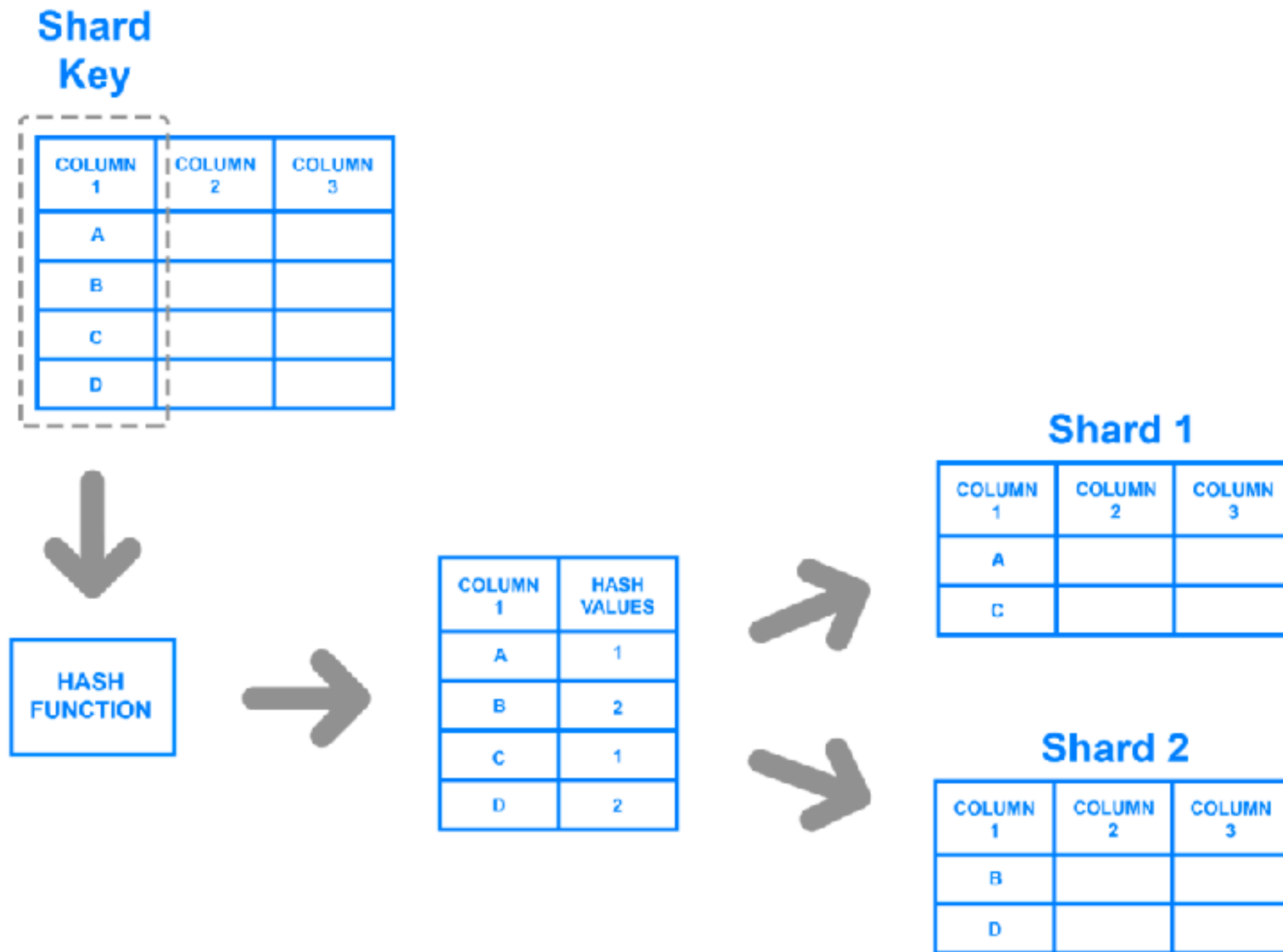
(a) Hash-based sharding



(b) Range-based sharding



# Shard Key





# Range

| PRODUCT     | PRICE |
|-------------|-------|
| WIDGET      | \$118 |
| GIZMO       | \$88  |
| TRINKET     | \$37  |
| THINGAMAJIG | \$18  |
| DOODAD      | \$60  |
| TCHOTCHKE   | \$999 |



(\$0-\$49.99)

| PRODUCT     | PRICE |
|-------------|-------|
| TRINKET     | \$37  |
| THINGAMAJIG | \$18  |

(\$50-\$99.99)

| PRODUCT | PRICE |
|---------|-------|
| GIZMO   | \$88  |
| DOODAD  | \$60  |

(\$100+)

| PRODUCT   | PRICE |
|-----------|-------|
| WIDGET    | \$118 |
| TCHOTCHKE | \$999 |





# Directory

Pre-Sharded Table

Shard Key

| DELIVERY ZONE | FIRST NAME | LAST NAME |
|---------------|------------|-----------|
| 3             | DARCY      | CLAY      |
| 1             | DENISE     | LASALLE   |
| 2             | HIROSHI    | YOSHIMURA |
| 4             | KIRSTY     | MACCOLL   |



Shard Key

| DELIVERY ZONE | SHARD ID |
|---------------|----------|
| 1             | S1       |
| 2             | S2       |
| 3             | S3       |
| 4             | S4       |



S1

|   |        |         |
|---|--------|---------|
| 1 | DENISE | LASALLE |
|---|--------|---------|

S2

|   |         |           |
|---|---------|-----------|
| 2 | HIROSHI | YOSHIMURA |
|---|---------|-----------|

S3

|   |       |      |
|---|-------|------|
| 3 | DARCY | CLAY |
|---|-------|------|

S4

|   |        |         |
|---|--------|---------|
| 4 | KIRSTY | MACCOLL |
|---|--------|---------|



# Sharding

Shard 1

| Stores |                |
|--------|----------------|
| id     | name           |
| 1      | my book store  |
| 5      | my other store |

| Products |      |          |
|----------|------|----------|
| id       | name | store_id |
| 1        | foo  | 1        |
| 2        | bar  | 1        |
| 3        | baz  | 1        |

| Purchases |            |          |       |
|-----------|------------|----------|-------|
| id        | product_id | store_id | price |
| 1         | 2          | 1        | 1000  |
| 2         | 1          | 1        | 1200  |
| 3         | 3          | 1        | 1199  |

Shard 2

| Stores |               |
|--------|---------------|
| id     | name          |
| 2      | my sock store |
| 6      | old things    |

| Products |           |          |
|----------|-----------|----------|
| id       | name      | store_id |
| 33       | new socks | 2        |
| 34       | old socks | 6        |
| 35       | old tie   | 6        |

| Purchases |            |          |       |
|-----------|------------|----------|-------|
| id        | product_id | store_id | price |
| 102       | 35         | 6        | 600   |
| 43        | 33         | 2        | 800   |

<https://www.citusdata.com/blog/2016/08/10/sharding-for-a-multi-tenant-app-with-postgres/>



# Challenge of Large datasets



# Challenge of Large datasets

Disk I/O, CPU, Memory pressure

Index size and maintenance

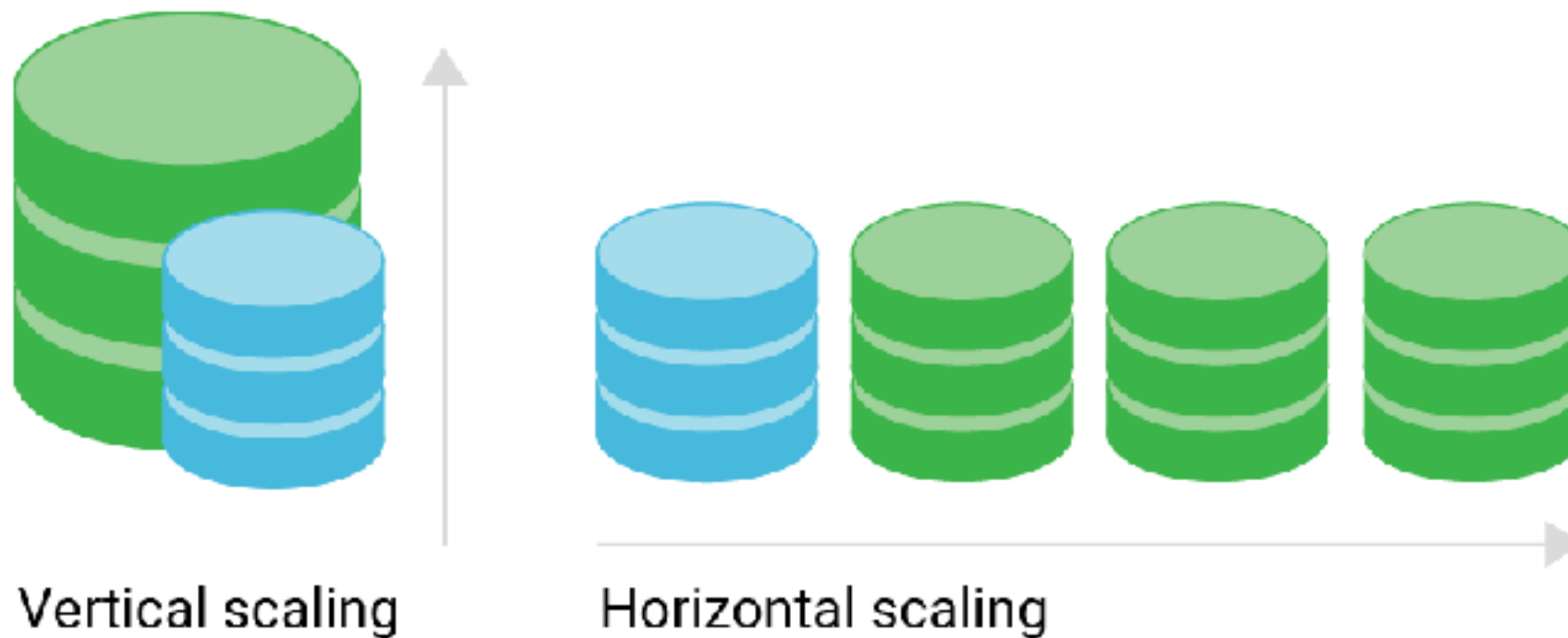
Locking and concurrency

Why database slow down  
with more data ?



# Scaling Database ?

## Vertical vs Horizontal



# Scalability metrics

Throughput (transactions per second)

Latency (response time)

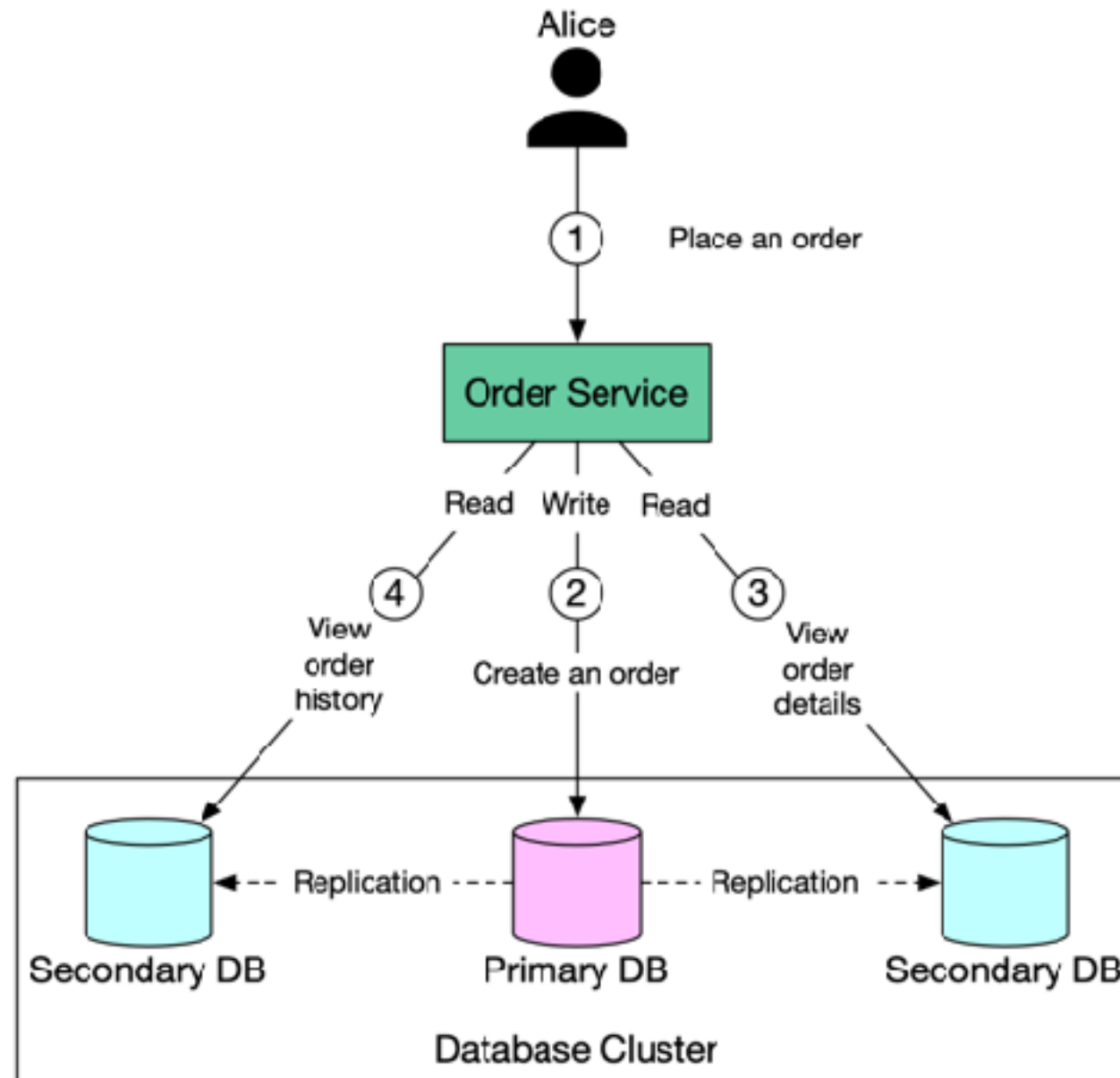
Availability (uptime)

Durability (data safety)



# Read replicas

## Read-scaling solution

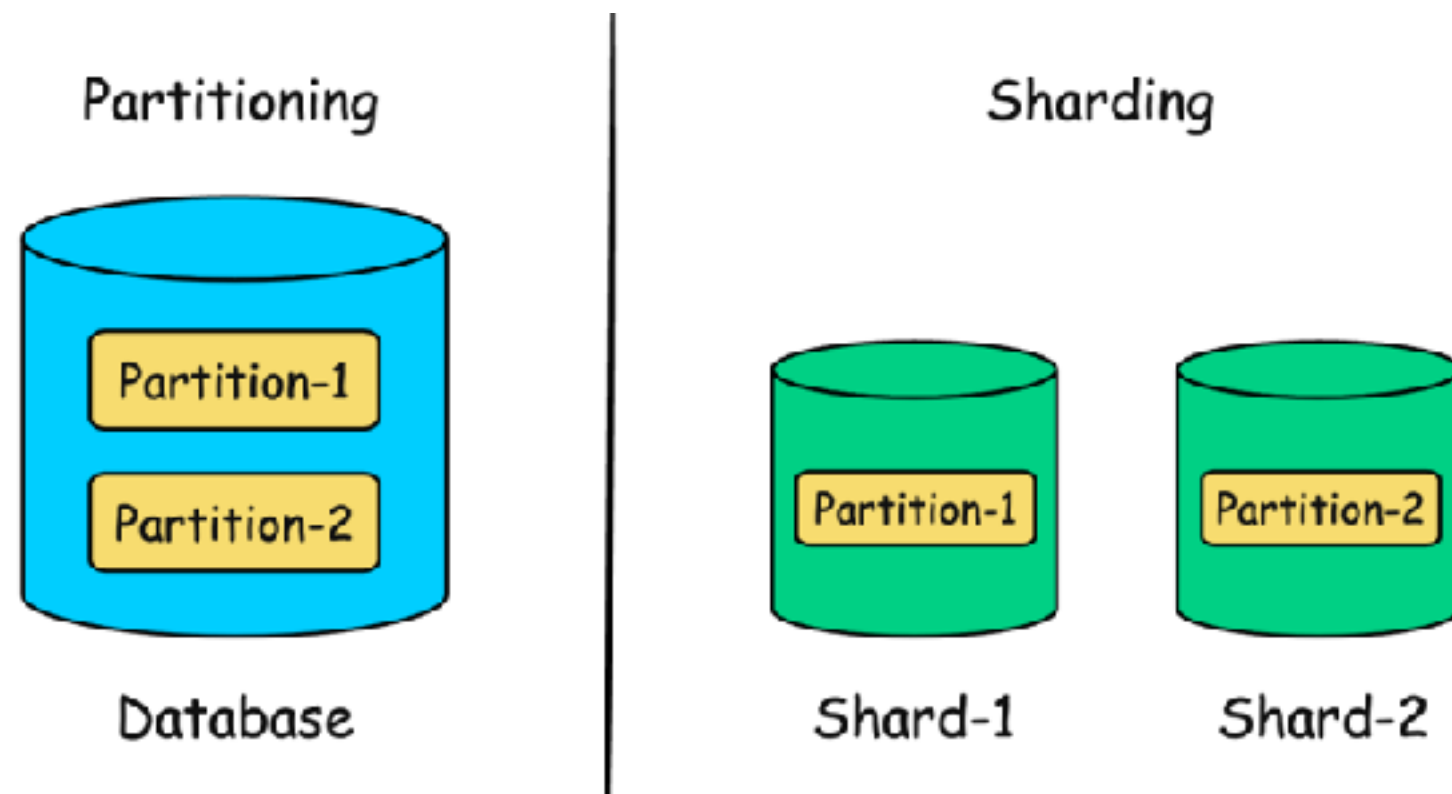




# Data distribution strategies

Partitioning (within a single instance)

Sharding (across multiple instances)





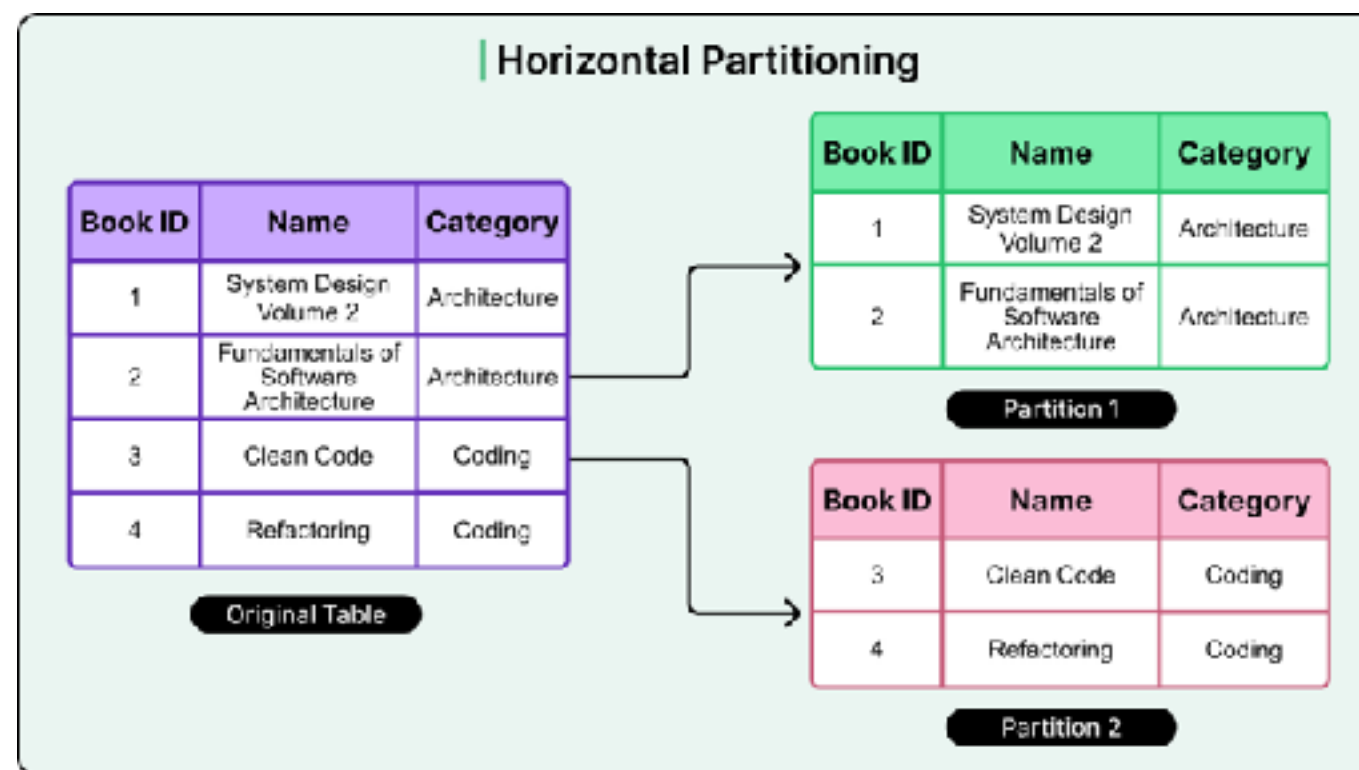
# Database sharding

Design for large dataset by split into smaller

Divided across multiple instance

Based-on horizontal partitioning

Use when to overcome single instance limitations



# Sharding techniques

Range

Hash or key-based

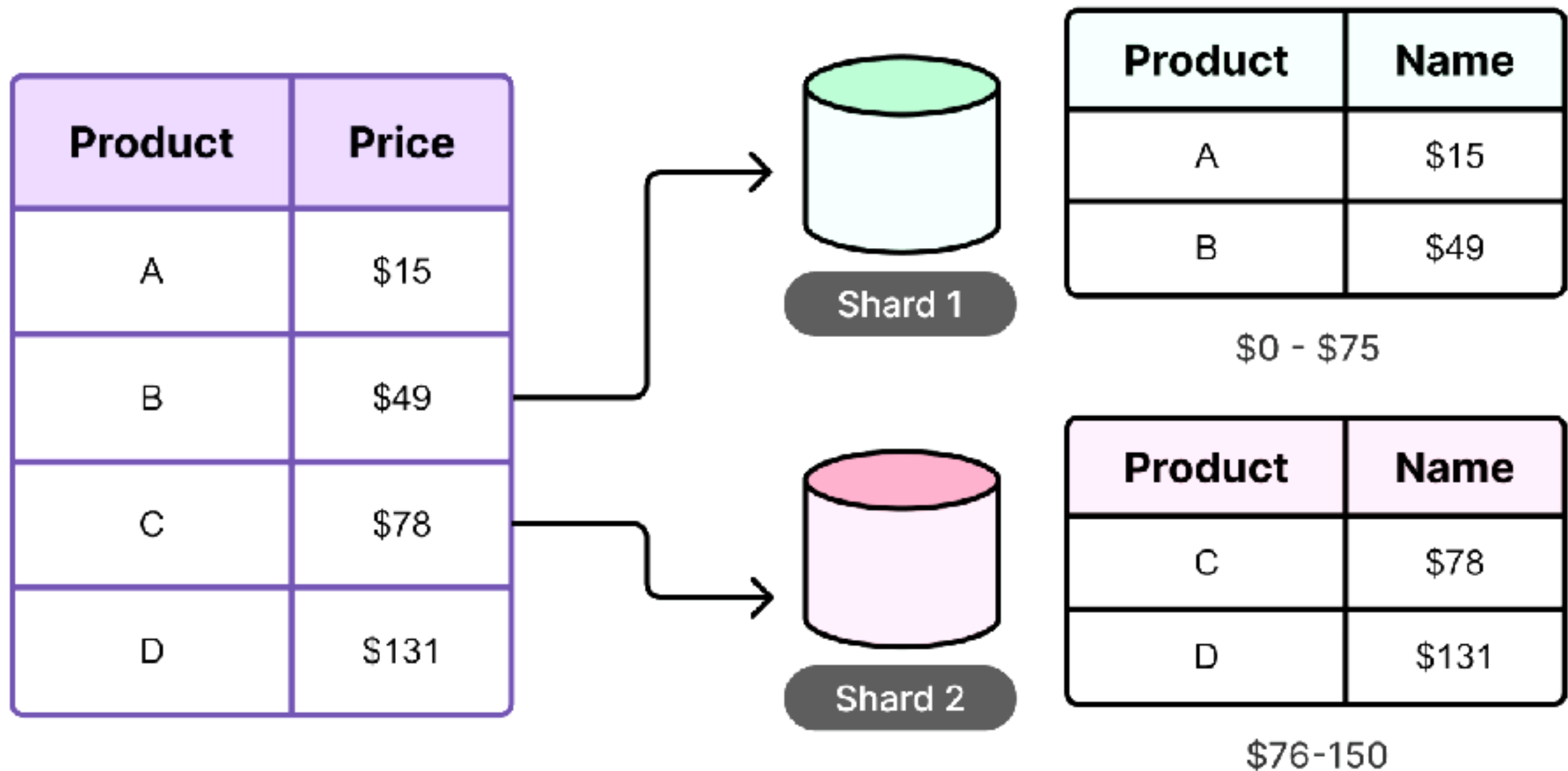
Directory-based

Request routing

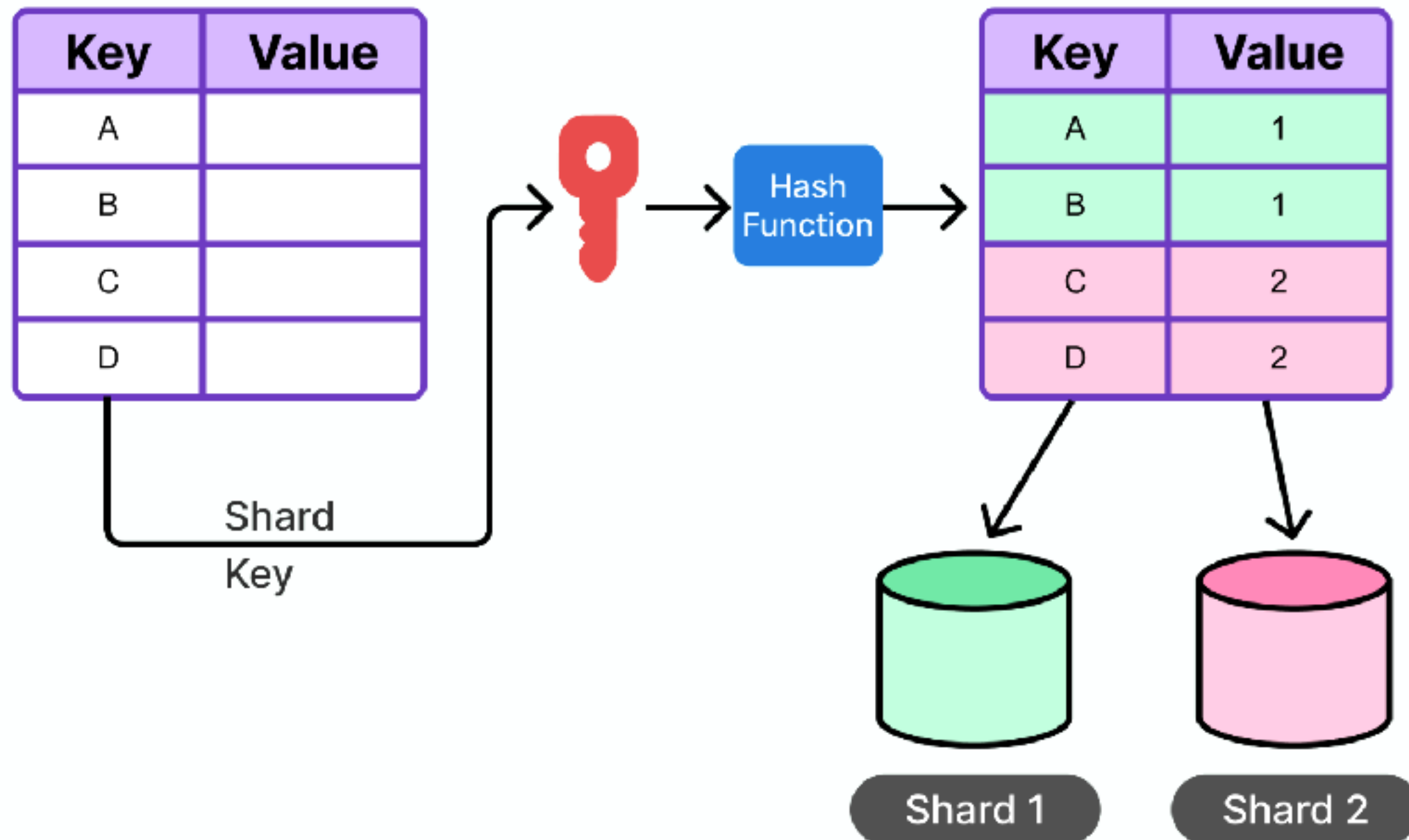
<https://blog.bytebytego.com/p/a-guide-to-database-sharding-key>



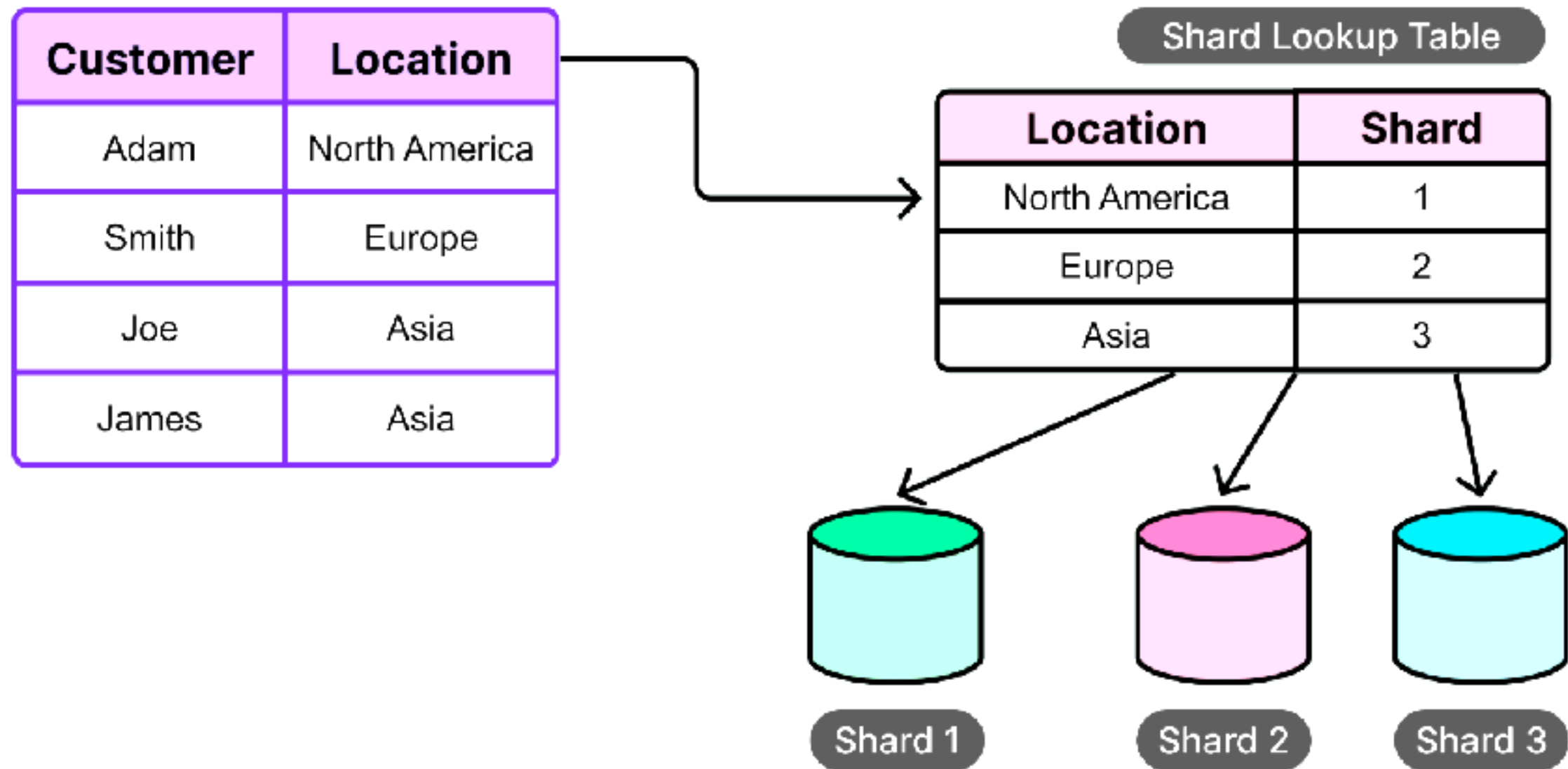
# Range-based sharding



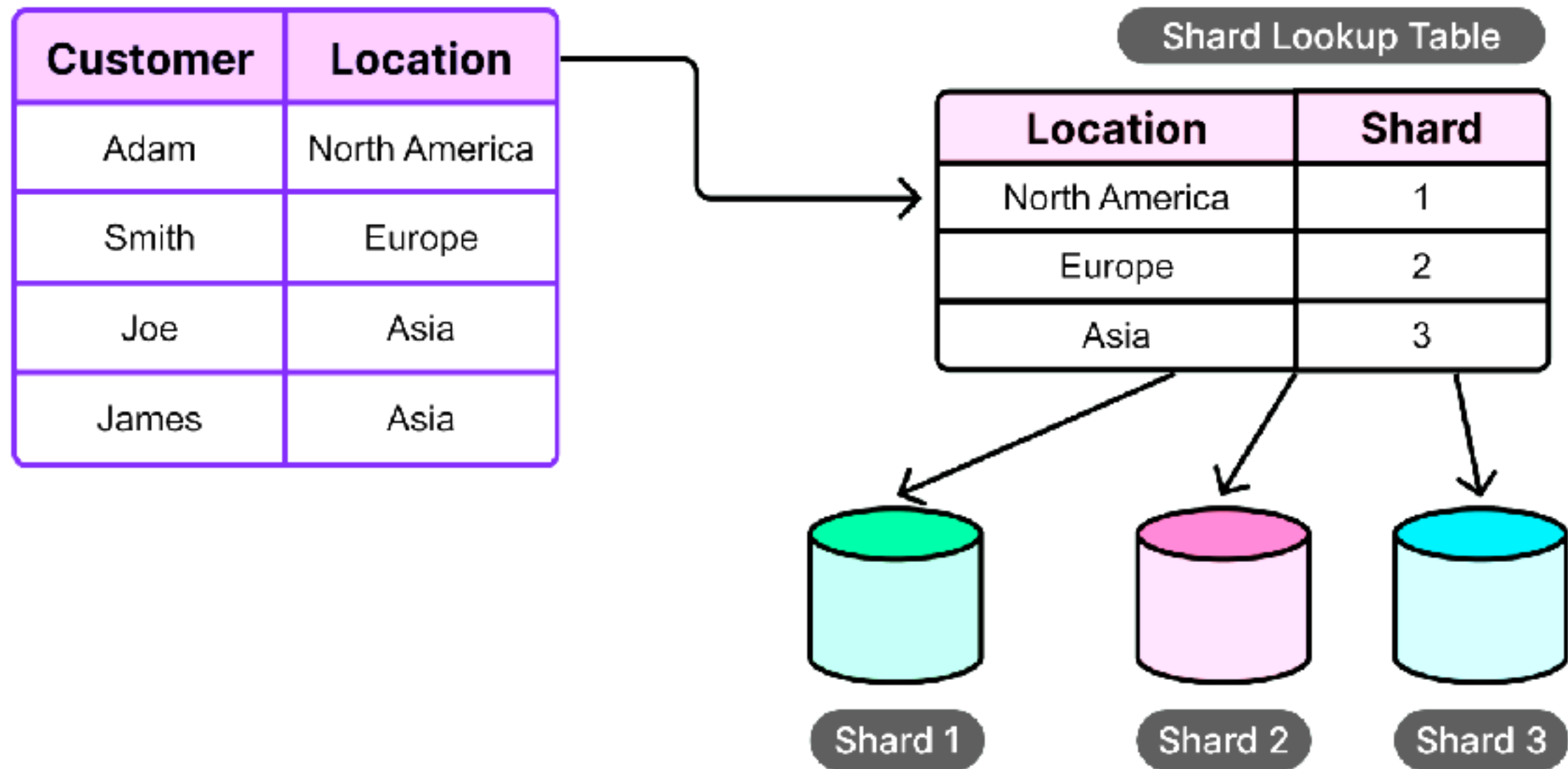
# Key-based sharding



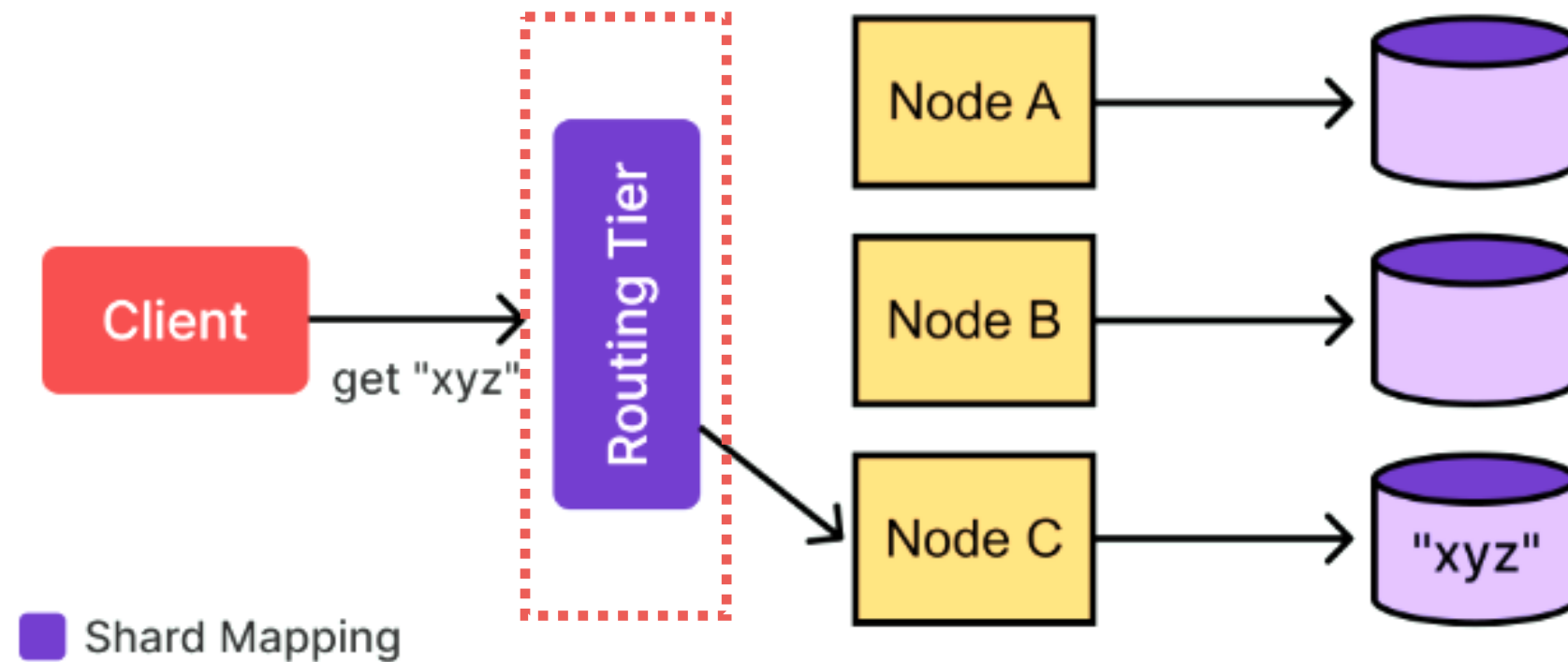
# Directory-based sharding



# Directory-based sharding



# Request routing





# Workshop with Sharding

<https://github.com/up1/workshop-database/blob/main/workshop/basic-workshop/partition.md>





# Denormalization



# Denormalization techniques

Redundant column/Pre-joining tables

Table splitting (horizontal/vertical)

Add derived columns

Mirrored tables

View vs Materialized views

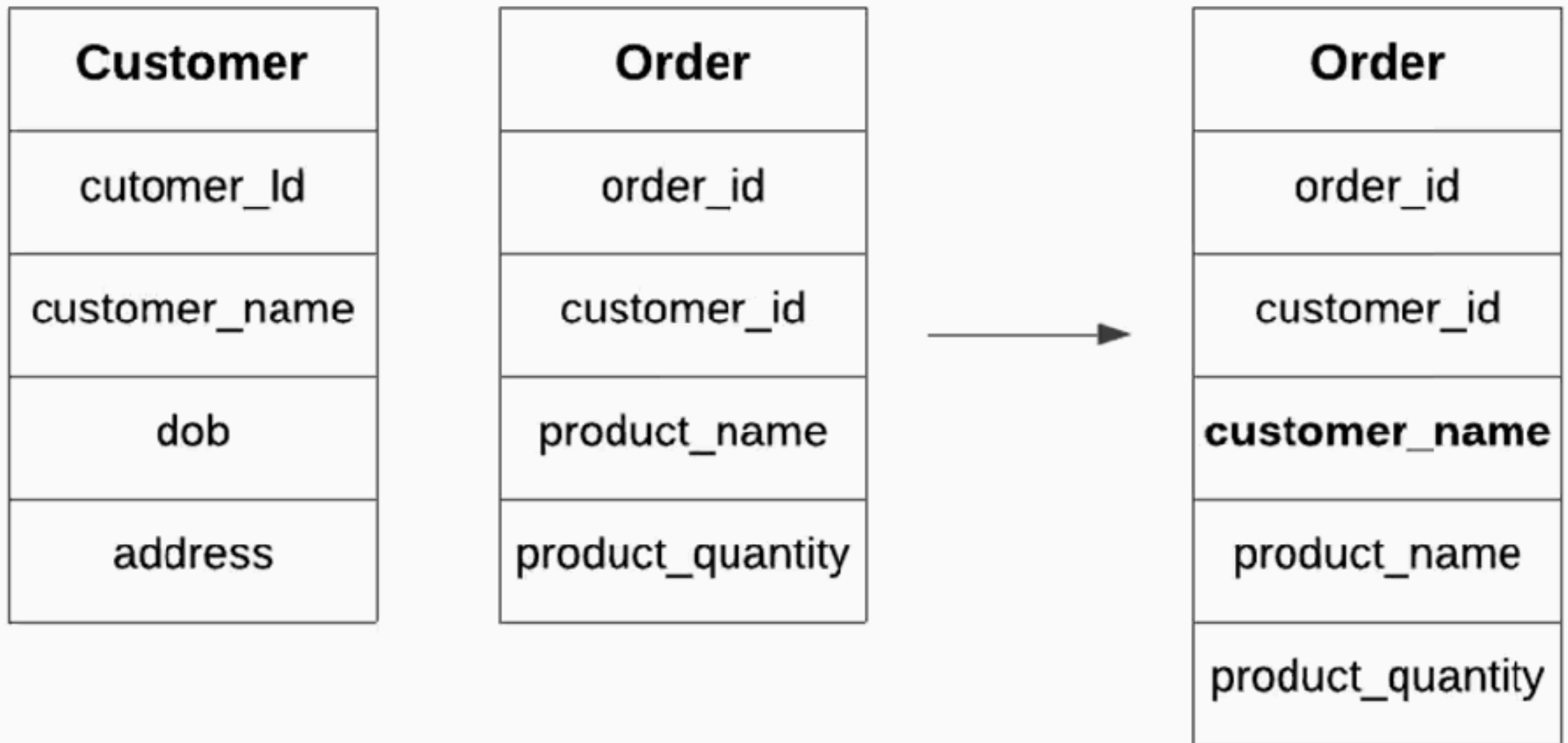


# View vs Materialized view ?

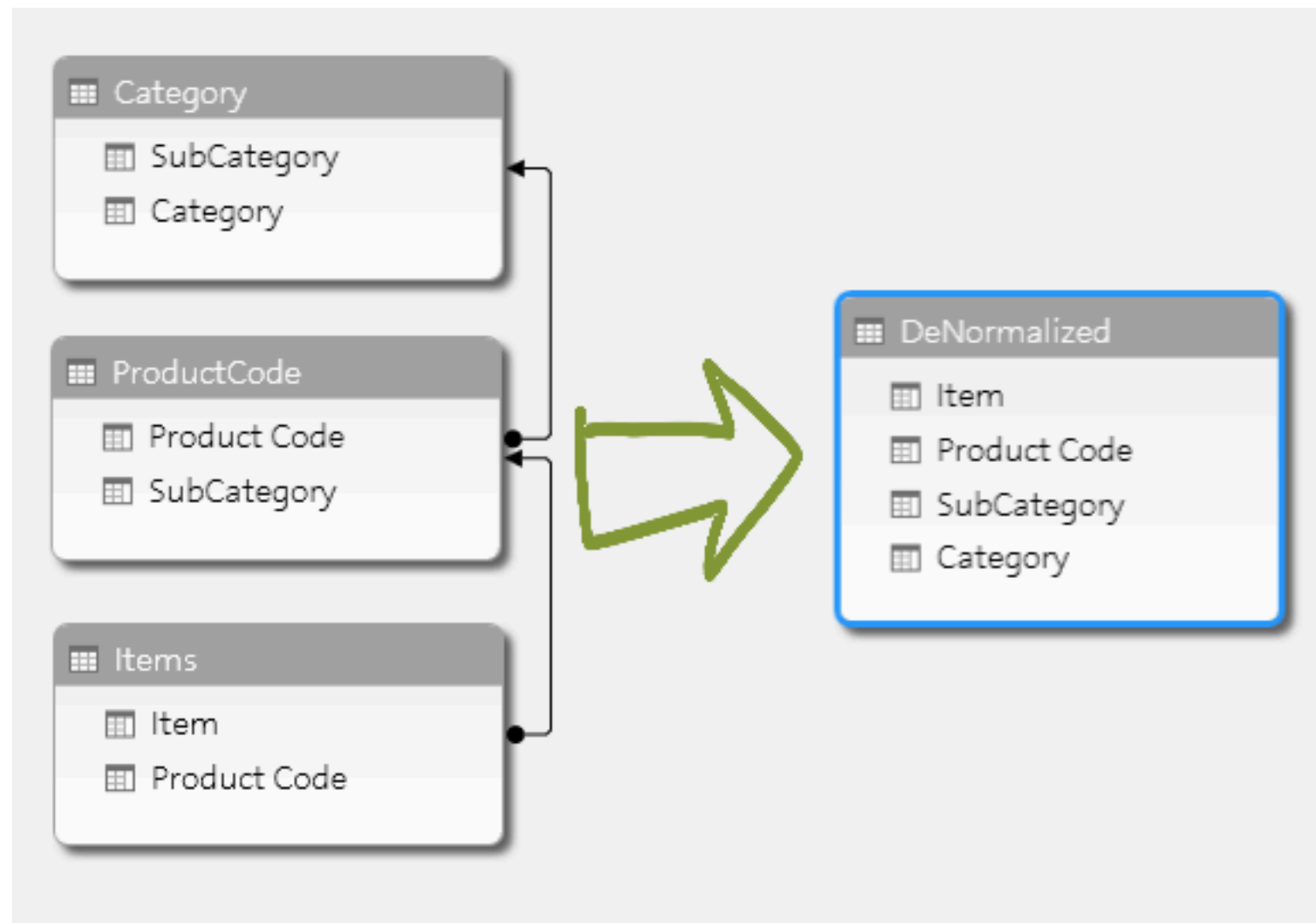
| Features       | View                   | Materialized view        |
|----------------|------------------------|--------------------------|
| Storage        | Virtual                | Physical                 |
| Query speed    | Slow for complex query | Fast                     |
| Data freshness | Realtime               | Snapshot, manual refresh |
| Index          | Can not be indexed     | Can be indexed           |
| Maintenance    | None                   | Require REFRESH command  |



# Redundant column/Pre-joining tables



# Redundant column/Pre-joining tables



# Horizontal splitting

| Student_ID | Department_ID | Student_Name |
|------------|---------------|--------------|
| 1          | 1             | Alex         |
| 2          | 2             | Marie        |
| 3          | 1             | Sam          |
| 4          | 3             | Sara         |



Computer Science

| Student_ID | Department_ID | Student_Name |
|------------|---------------|--------------|
| 1          | 1             | Alex         |
| 3          | 1             | Sam          |

Chemistry

| Student_ID | Department_ID | Student_Name |
|------------|---------------|--------------|
| 2          | 2             | Marie        |

Botany

| Student_ID | Department_ID | Student_Name |
|------------|---------------|--------------|
| 4          | 3             | Sara         |



# Vertical splitting





# Add derived columns

| Student_ID | Name  |
|------------|-------|
| 1          | Alex  |
| 2          | Marie |

| Student_ID | Assignment_ID | Mark  |
|------------|---------------|-------|
| 1          | 1             | 20    |
| 1          | 2             | 35.50 |
| 2          | 1             | 45    |
| 2          | 2             | 45    |



| Student_ID | Name  | Total_Marks |
|------------|-------|-------------|
| 1          | Alex  | 55.5        |
| 2          | Marie | 90          |





# Pros of data denormalization

Improve query performance (UX)

Reduce complexity of data model

Improve application scalability

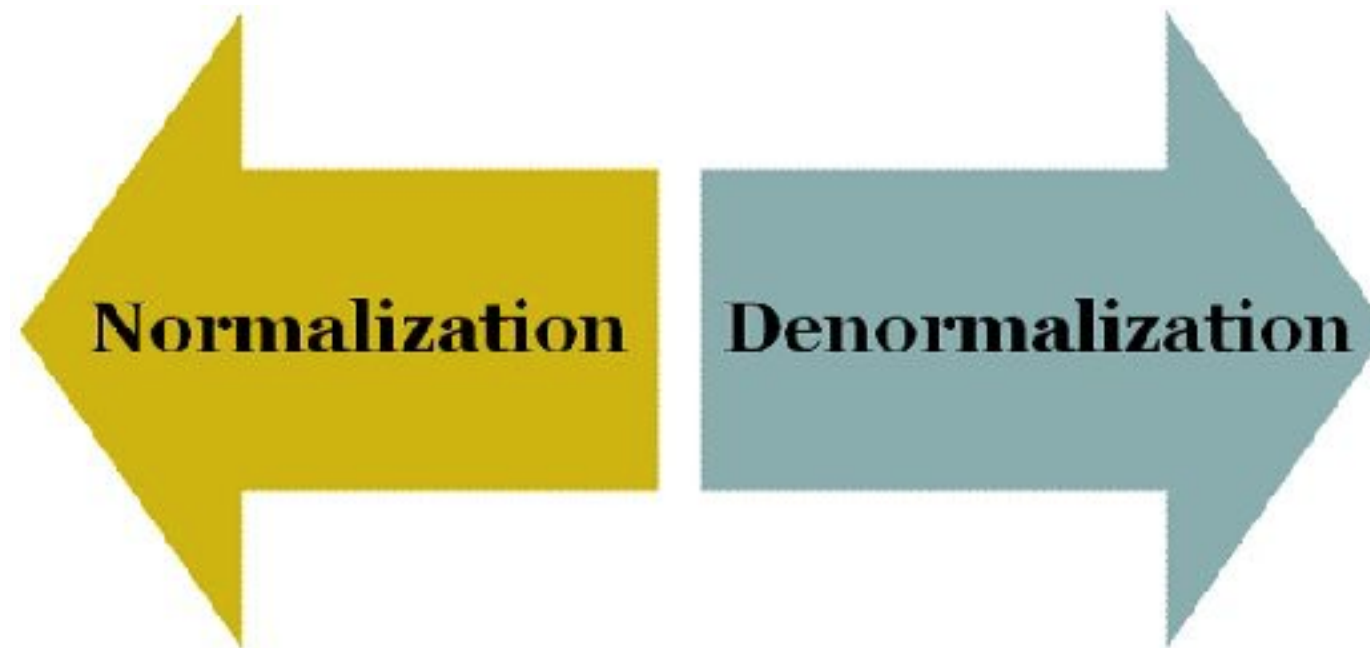
Generate data report faster



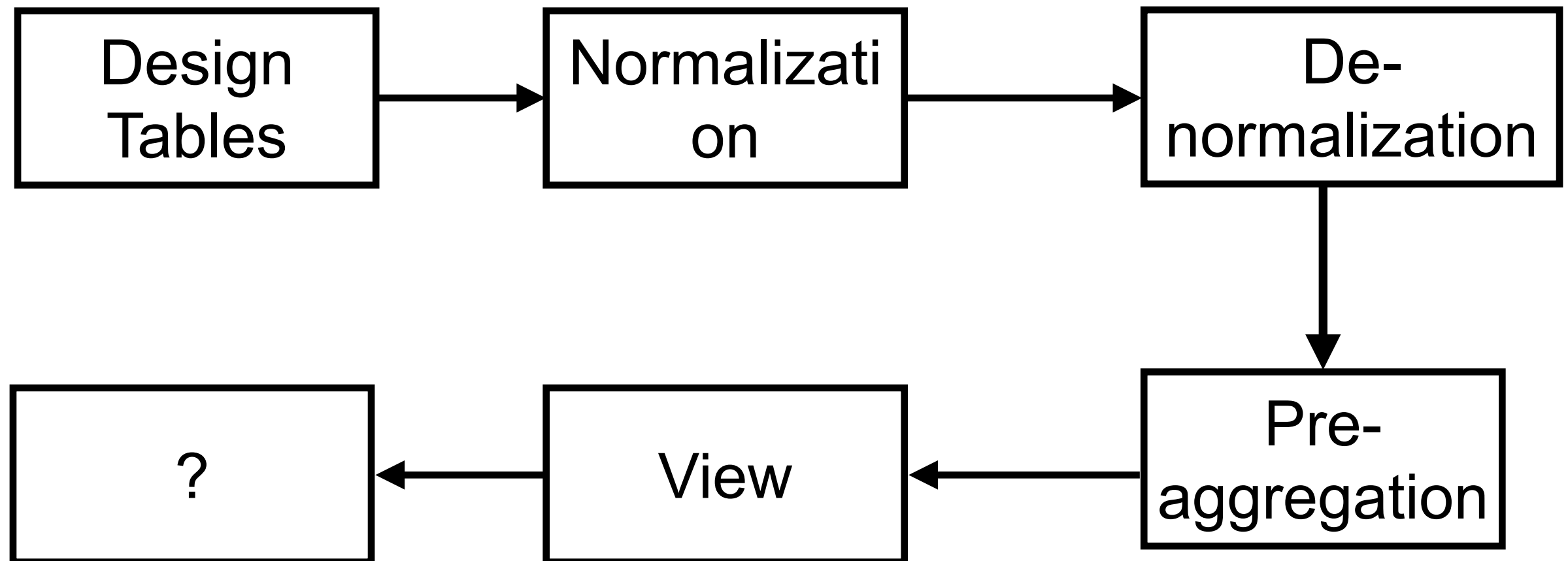
# Cons of data denormalization

- Increase data redundancy
- Inconsistency between data sets
- Require more disk space (split tables)
- More data model (hard to maintain)
- Difficult to insert and update data
- Maintenance costs





# Workshop



# Q/A

